

STAR Robot Control System

Comprehensive Technical Documentation

Complete Reference for Software Engineers

Version 1.0
December 2025

STAR Development Team
Locked Inc.

Contents

1	Nanopb (Protobuf) Protocol with ARQ and CRC for SPI Communication	9
1.1	Executive Summary	9
1.1.1	Control Loop Frequency Rationale	9
1.2	Key Design Decisions	10
1.2.1	Protocol Stack	10
1.2.2	Byte Order and Standards Compliance	10
1.2.3	Memory Budget	11
1.3	Implementation Requirements	12
1.3.1	Protocol Stack Architecture	12
1.3.2	ESP32 (SPI Peripheral) Implementation	12
1.3.3	RPi5 (SPI Controller) Implementation	13
1.3.4	ARQ Symmetry and State Management	14
1.3.5	Example Transaction: RPi5 Sends Velocity Command	15
1.4	ARQ Communication Flow	15
1.4.1	Normal Operation (No Retransmission)	16
1.4.2	Error Recovery Scenarios	16
1.5	Error Handling Strategies	18
2	Protocol Buffer Schema Architecture	19
2.1	Architecture Overview	19
2.2	Service Specifications	19
2.2.1	MotorControlService	19
2.2.2	TelemetryService	20
2.2.3	BatteryManagementService	21
2.2.4	ConfigurationService	21
2.2.5	FirmwareUpdateService	22
2.3	common.proto – Shared Types	22
2.4	Code Generation and Integration	23
3	Hardware Pin Connections	24
3.1	RX72N Microcontroller Pinout (R5F572NNHGFP#30)	24
3.1.1	Package Information	24
3.1.2	Motor Control Pin Assignments	24
3.1.3	Communication Interfaces	25
3.1.4	Status and Safety	26
3.1.5	System Clocks	26
3.2	Comparison: RX72N vs ESP32-S3 Architecture	26
3.3	Raspberry Pi 5 Pinout	26
3.3.1	RPi5 Pin Function Categories	27
3.4	Additional Peripheral Connections	28
3.4.1	IMU (Inertial Measurement Unit)	28
3.4.2	Stereo Cameras	28
3.4.3	Temperature Sensor	29
4	Protocol Buffer Style Guide	30
4.1	Terminology Standard	30

4.2	Protocol Buffer Naming Conventions (Boston Dynamics Style)	30
4.3	Unit Suffixes (MKS System)	30
4.4	General Documentation Standards	31
5	C Style Guide	32
5.1	General Guidelines for ASM and Inline ASM	32
5.2	When to Use ASM and Inline ASM	33
5.3	Guidelines	33
5.4	Platform-Specific Error Checking Macros	33
5.5	General Rules	34
5.6	Control Structures	34
5.6.1	For Loops	35
5.6.2	While Loops	35
5.7	Functions	35
5.8	Structs and Enums	36
5.9	Initializer Lists	37
5.10	Closing Brace Placement	38
5.11	Summary Table	38
5.12	General Rules	38
5.13	Basic Switch Structure	39
5.14	Case Block Braces	39
5.15	Break Statement Requirements	40
5.16	Default Case Handling	41
5.17	Switch on Enum Types	42
5.18	Indentation	43
5.19	Summary	43
5.20	General Commenting Guidelines	44
5.21	File Header Comments	44
5.22	Source File Header	45
5.23	Doxygen Documentation	45
5.23.1	Function Documentation	45
5.23.2	Parameter Direction Markers	46
5.23.3	Code Examples with @code/@endcode	47
5.23.4	Struct Member Documentation	47
5.24	Special Comment Keywords	48
5.24.1	Keyword Definitions	48
5.25	Section Comments	48
5.26	Aligned Comments	49
5.27	Implementation Comments	49
5.28	Private Function Documentation	50
5.29	Summary	51
5.30	RTOS Types, Mutexes, Semaphores, and Atomic Operations	51
5.31	General Rules	51
5.32	Basic Enum Structure	52
5.33	Explicit Value Assignment	52
5.34	Documentation	52
5.35	Enum to String Functions	53
5.36	Using the Count Sentinel	53

5.37	Switch Statement Pattern	54
5.38	Forward Declaration	54
5.39	Naming Convention Summary	55
5.40	General Principles	55
5.41	Platform Error Codes	55
5.42	Validation Macros	56
5.42.1	RX_RETURN_ON_FALSE	56
5.42.2	ESP_GOTO_ON_ERROR	57
5.42.3	ESP_ERROR_CHECK	58
5.43	NULL Safety Patterns	58
5.44	Mutex Error Handling	59
5.45	Error Interface Dependency Injection	59
5.46	Cleanup Patterns	60
5.47	Error Logging	61
5.48	Summary	61
5.49	Return Value Conventions	62
5.50	Private and Exposed Private Functions	62
5.50.1	Private Functions (static)	62
5.50.2	Exposed Private Functions (priv_)	63
5.51	Parameter Validation	63
5.52	Function Declaration Formatting	64
5.53	Doxygen Documentation	65
5.54	Private Function Documentation	65
5.55	Function Implementation Patterns	65
5.55.1	Single Exit Point with Cleanup	65
5.55.2	Factory Function Pattern	66
5.56	Callback Functions	67
5.57	Summary	68
5.58	General Principles	68
5.59	Static File-Scoped Variables (s_ prefix)	68
5.60	Why static const Over #define	69
5.61	When to Use #define	69
5.62	Explicit Type Usage - Always Use Fixed-Width Integer Types	70
5.62.1	Why Explicit Types Matter	70
5.62.2	Correct Type Usage	70
5.62.3	Special Case: Characters and Strings	71
5.62.4	Loop Counters and Array Indices	71
5.62.5	Function Parameters and Return Types	71
5.62.6	Type Casting for Enum Comparisons	72
5.62.7	Summary: Type Usage Rules	72
5.63	True Global Variables (g_ prefix)	72
5.64	Dependency Injection Alternative	73
5.65	Static Variables for Module State	73
5.66	Thread Safety for Shared State	74
5.67	Summary	74
5.68	General Principles	75
5.69	Include Guard Naming Convention	75
5.70	C++ Compatibility	76

5.71	Header File Structure	76
5.72	Include Order	77
5.73	Forward Declarations	78
5.74	Separating Types from Functions	78
5.75	Doxygen File Documentation	79
5.76	Complete Header Template	79
5.77	Summary	81
5.78	General Rules	81
5.79	Space After Control Keywords	81
5.80	No Space After Function Names	82
5.81	Binary Operators	82
5.82	Unary Operators	82
5.83	Pointer Declarations	83
5.84	Comma and Semicolon Spacing	83
5.85	Parentheses Spacing	84
5.86	Parameter Alignment	84
5.87	Variable and Comment Alignment	84
5.88	Struct Initializer Alignment	85
5.89	Summary	86
5.90	General Rules	86
5.91	Include Order	86
5.92	Complete Example from Codebase	87
5.93	FreeRTOS Include Order	87
5.94	System Includes	88
5.95	Platform HAL Includes	88
5.96	Project Includes	88
5.97	Header File Includes	89
5.98	Conditional Includes	89
5.99	Include Comments	89
5.100	Avoiding Circular Dependencies	90
5.101	Summary	90
5.102	General Rules	91
5.103	Functions and Variables	91
5.104	Module Prefixes	91
5.105	Constants and Macros	92
5.106	Type Names	92
5.107	Enum Members	93
5.108	Static Variables (File-Scoped)	93
5.109	Global Variables	94
5.110	Private Functions	94
5.110.1	File-Scoped Static Functions (internal_ prefix)	94
5.110.2	Exposed Private Functions (priv_ prefix)	95
5.111	Summary Table	96
5.112	Typedef Struct Pattern	96
5.112.1	Standard Pattern	96
5.112.2	Error Handler Example	97
5.113	Union Usage for Protocol Configurations	97
5.113.1	Protocol-Specific Structures	98

5.114	Callback Function Types	99
5.115	Interface Patterns (Dependency Injection)	100
5.115.1	Interface Usage Pattern	101
5.116	Opaque Handle Patterns	101
5.117	Operations Structure Pattern	101
5.118	Manager Structure Pattern	102
5.119	Best Practices	103
5.120	Library Directory Structure	103
5.120.1	Example: star_bus Library	103
5.121	CMakeLists.txt Structure	104
5.122	Header File Organization	104
5.122.1	Header File Template	104
5.122.2	Header Section Order	106
5.123	Include Guard Naming Convention	106
5.124	Source File Organization	107
5.124.1	Source File Template	107
5.124.2	Source Section Order	108
5.125	Types Header Separation	109
5.125.1	Controller Header Pattern	109
5.126	Project Root Structure	110
5.127	Best Practices	110
5.128	Overview	110
5.129	SPI Bus Terminology	111
5.129.1	COPI/CIPO (Not MOSI/MISO)	111
5.130	I2C and General Bus Terminology	111
5.130.1	Controller/Peripheral (Not Master/Slave)	112
5.131	Mapping Platform Legacy APIs	112
5.132	Documentation Guidelines	113
5.133	Code Review Checklist	114
5.134	Common Patterns	114
5.135	Summary	115
5.136	Dependency Injection (DIP)	115
5.136.1	Interface Definition	115
5.136.2	Interface Implementation	116
5.136.3	Dependency Injection Pattern	116
5.136.4	NULL Interface Handling	117
5.137	Bus Abstraction Pattern	117
5.137.1	Unified Configuration	117
5.137.2	Factory Functions	118
5.137.3	Manager Pattern	118
5.138	Operations Structure Pattern	119
5.139	Thread-Safe Callback Pattern	119
5.140	Thread Safety Patterns	120
5.140.1	Mutex Timeout Convention	120
5.140.2	Mutex with Cleanup (goto pattern)	120
5.141	Error Handler Ownership Pattern	121
5.142	Linked List Management	122
5.143	Best Practices Summary	123

5.144	Prefer Alternatives to Macros	123
5.145	When Macros Are Necessary	124
5.146	Macro Safety Rules	124
5.146.1	Wrap Statement-Like Macros in <code>do-while(0)</code>	124
5.146.2	Parenthesize All Macro Arguments	125
5.146.3	Beware of Side Effects and Multiple Evaluation	125
5.147	Multi-line Macro Formatting	125
5.148	Naming Conventions	126
5.149	Include Guards	126
5.150	Conditional Compilation	127
5.151	Overview	127
5.152	Benefits	127
5.153	Standard Unit Suffixes	128
5.154	Usage Examples	128
5.154.1	Distance and Velocity	128
5.154.2	Angular Measurements	128
5.154.3	Temperature	129
5.154.4	Time	129
5.154.5	Electrical Measurements	129
5.154.6	Percentage	130
5.155	Protocol Buffer Integration	130
5.156	Conversion Functions	130
5.157	Summary	131
5.158	No Dynamic Allocation	131
5.158.1	Rationale	131
5.158.2	Alternatives	132
5.158.3	Platform Heap Functions	132
5.159	Avoid Large Stack Allocations	133
5.159.1	Stack Size Constraints	133
5.159.2	Examples	133
5.159.3	Stack Usage Guidelines	134
5.160	Use <code>const</code> for Flash Storage	134
5.160.1	Rationale	134
5.160.2	Examples	134
5.160.3	Pointer Declarations	135
5.161	Validate All Buffers	135
5.161.1	Rationale	136
5.161.2	Validation Patterns	136
5.161.3	DMA Buffer Validation	136
5.161.4	Ring Buffer Validation	137
5.162	Summary	138
5.163	Interrupt Service Routines (ISRs)	138
5.163.1	Minimize ISR Work	138
5.163.2	ISR Constraints	139
5.164	No Blocking Delays	139
5.165	Real-Time and Safety	140
5.165.1	Timing Constraints	140
5.165.2	Watchdog Timers	141

5.165.3	Sensor Timeouts	141
5.165.4	Fail-Safe Design	142
5.165.5	CRC Validation	143
5.166	Hardware Abstraction	143
5.166.1	Use Abstract Bus Interfaces	143
5.166.2	Debounce Mechanical Inputs	144
5.166.3	Enable Brown-Out Detection	144
5.166.4	Flash Wear-Leveling	145
5.166.5	Always Use Pull-Up/Pull-Down	145
5.167	Performance and Real-Time	145
5.167.1	Benchmark, Don't Guess	145
5.167.2	Avoid Blocking Operations	146
5.168	Power Management	146
5.168.1	Use Sleep Modes	146
5.168.2	Use DMA to Reduce CPU Load	147
5.169	Testing and Debugging	147
5.169.1	Use Mock Drivers	147
5.169.2	Meaningful Logging	147
5.170	Reliability and Fault Tolerance	147
5.170.1	Reset Peripherals if Stuck	147
5.171	Summary	148
5.172	nanopb Overview	148
5.173	Why Static Allocation?	148
5.174	Field Size Configuration	149
5.175	Standard Field Size Constraints	149
5.176	Examples	149
5.176.1	String Fields	149
5.176.2	Repeated Fields	149
5.176.3	Bytes Fields	150
5.177	Sizing Guidelines	150
5.177.1	String Sizes	150
5.177.2	Repeated Field Counts	150
5.177.3	Bytes Field Sizes	151
5.178	Memory Impact	151
5.178.1	Buffer Size Calculation	151
5.178.2	RAM Usage Example	151
5.179	Best Practices	151
5.179.1	Right-Size Your Buffers	151
5.179.2	Use Fixed-Size Types	152
5.179.3	Validate Message Sizes	152
5.180	Integration with Code Generation	152
5.180.1	buf.gen.yaml Configuration	152
5.180.2	Options File Naming	153
5.181	Common Pitfalls	153
5.181.1	Missing Options File	153
5.181.2	Mismatched Field Names	153
5.181.3	Forgetting Array Count	153
5.182	Summary	153

5.183	NASA Power of 10: Rules for Safety-Critical Code	154
5.183.1	Compliance Status Legend	154
5.183.2	Rule 1: Simplify Control Flow	154
5.183.3	Rule 2: Fixed Loop Upper-Bounds	155
5.183.4	Rule 3: No Dynamic Memory After Initialization	155
5.183.5	Rule 4: Keep Functions Short	156
5.183.6	Rule 5: Use Assertions Generously	156
5.183.7	Rule 6: Declare Data at Smallest Scope	157
5.183.8	Rule 7: Check All Return Values and Parameters	157
5.183.9	Rule 8: Limit Preprocessor Use	158
5.183.10	Rule 9: Restrict Pointer Use	159
5.183.11	Rule 10: Compile with Maximum Warnings	159
5.183.12	Summary: STAR Compliance Matrix	160
5.183.13	References	160
6	Gateway Architecture	161
6.1	Overview	161
6.2	Protocol Stack Implementation	161
6.3	Directory Structure	161
6.4	Frame Package	162
6.4.1	Frame Constants	162
6.4.2	Frame Types	162
6.5	Transport Package	163
6.5.1	SPI Configuration	163
6.5.2	Transport Interface	163
6.6	ARQ Package	163
6.6.1	ARQ Parameters	163
6.6.2	ARQ State Machine	163
6.7	gRPC Services	164
6.8	Build and Test	164
6.9	Dependencies	164

1 Nanopb (Protobuf) Protocol with ARQ and CRC for SPI Communication

1.1 Executive Summary

Overview: Protocol Buffers (nanopb) encapsulated in frames with CRC-32 and Automatic Repeat Request (ARQ) for reliable SPI communication between RPi5 and ESP32-S3.



System Constraints:

- ESP32-S3-WROOM-1-N16: 16MB flash, 512KB SRAM, NO PSRAM
- 100Hz SPI communication (10ms period)
- 250Hz motor control loop (4ms period)
- Static allocation only (no malloc)

1.1.1 Control Loop Frequency Rationale

Why different loop rates?

The nested loop architecture follows classic control systems design. The outer loop (SPI at 100Hz) handles command distribution and telemetry collection, while the inner loop (PID at 250Hz) responds quickly to motor dynamics. Running PID at 2.5 times the communication rate ensures smooth control between commands without wasting CPU on excessive computation.

Why 250Hz for motor control?

This is a Nyquist sampling consideration. With DC gearmotors at 210 RPM (3.5 Hz mechanical speed) and a first-order time constant $\tau = 75$ ms (cutoff frequency $f_c = 1/(2\pi\tau) \approx 2.1$ Hz), the 250Hz control rate provides approximately 60 times oversampling of the mechanical dynamics - well above the 2 times Nyquist minimum.

While the ESP32 could easily run the PID loop at 1kHz or higher, doing so would not improve control quality because the motor mechanics are the limiting factor, not the computational speed. The physical system cannot respond faster than its time constant allows. Running at 250Hz balances:

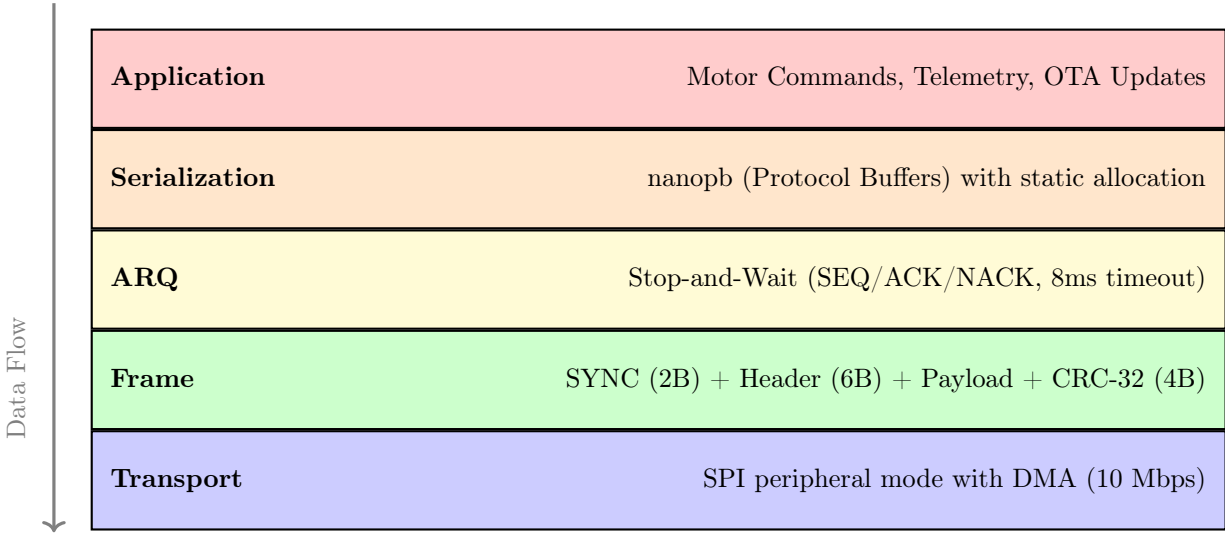
- Adequate sampling of motor dynamics (60 times oversampling)
- Efficient CPU utilization (4 motors $\times$ PID + encoder reads)
- Low latency between velocity commands and motor response
- Power efficiency (fewer ADC conversions, GPIO reads, and computations)

For context, control loop frequencies should match system dynamics:

- DC gearmotors: 50-500 Hz typical
- High-speed servos: 1-10 kHz
- Motor current control: 10-50 kHz
- Temperature control: 0.1-10 Hz

1.2 Key Design Decisions

1.2.1 Protocol Stack



1.2.2 Byte Order and Standards Compliance

The protocol follows established standards for byte ordering to ensure cross-platform compatibility:

Table 1: Byte Order Standards by Protocol Field

Field	Byte Order	Standard	Rationale
SYNC	Big-endian	RFC 1700	Network byte order for headers
SEQ	Big-endian	RFC 1700	Network byte order for headers
LEN	Big-endian	RFC 1700	Network byte order for headers
TYPE	N/A (1 byte)	–	Single byte, no endianness
FLAGS	N/A (1 byte)	–	Single byte, no endianness
CRC-32	Little-endian	IEEE 802.3	Standard CRC-32 polynomial

SYNC Magic Word (0x55AA): The frame synchronization marker uses the distinctive bit pattern 0x55AA (binary: 0101010101010101 in big-endian wire format). This alternating pattern is easily recognizable and unlikely to appear in random data, making it ideal for frame alignment. The value 0x55AA is transmitted as bytes [0x55, 0xAA] on the wire.

RFC 1700 (Network Byte Order): All multi-byte header fields (SYNC, SEQ, LEN) use big-endian format following RFC 1700 “Assigned Numbers” standard. This ensures compatibility when communicating between different architectures (ARM RPi5 and Xtensa ESP32, both little-endian internally).

IEEE 802.3 (CRC-32): The frame checksum uses IEEE 802.3 polynomial (0x04C11DB7) in little-endian format, matching the ESP32 ROM CRC implementation and standard Ethernet CRC libraries. This allows zero-copy CRC validation using hardware accelerators where available.

Key Insight: Mixed byte orders are intentional — headers follow network standards for cross-platform compatibility, while CRC follows IEEE 802.3 for hardware acceleration and library compatibility.

1.2.3 Memory Budget

Table 2: SRAM Usage (512 KB total)

Component	Size (KB)	Usage (%)
DMA double-buffer	8.0	1.56
RX/TX buffers	2.0	0.39
Protobuf structs	1.5	0.29
ARQ state	4.5	0.88
Total	16.0	3.12

Table 3: Flash Usage (16 MB total)

Component	Size (KB)	Usage (%)
Generated protobuf	80	0.49
Nanopb runtime	12	0.07
Frame + ARQ layers	8	0.05
Total	100	0.61

Memory Budget Calculations:

SRAM (512 KB total):

- **DMA double-buffer:** 2×4092 bytes = 8184 bytes = 8.0 KB
Calculation: Two ping-pong DMA buffers at `STAR_SPI_DMA_MAX_SIZE` (4092 bytes each)
Usage: $\frac{8.0}{512} \times 100 = 1.56\%$
- **RX/TX buffers:** $1024 + 1024 = 2048$ bytes = 2.0 KB
Calculation: 1 KB RX buffer + 1 KB TX buffer for staging protobuf messages
Usage: $\frac{2.0}{512} \times 100 = 0.39\%$
- **Protobuf structs:** 1536 bytes = 1.5 KB
Calculation: Largest message structures (SetVelocityResponse $\approx$ 600 bytes, TelemetryData $\approx$ 400 bytes)
Usage: $\frac{1.5}{512} \times 100 = 0.29\%$
- **ARQ state:** $4 + 4 + 8 + 4 + 4092 = 4112$ bytes $\approx$ 4.5 KB
Calculation: Sequence number (4B) + retry counter (4B) + timestamp (8B) + state (4B) + retry buffer (4092B)
Usage: $\frac{4.5}{512} \times 100 = 0.88\%$
- **Total SRAM:** $8.0 + 2.0 + 1.5 + 4.5 = 16.0$ KB
Total usage: $\frac{16.0}{512} \times 100 = 3.12\%$

Flash (16 MB = 16384 KB total):

- **Generated protobuf:** 80 KB
Calculation: 6 proto files (common, motor_control, telemetry, battery_management, configuration, firmware_update) $\times$ 13 KB avg compiled size
Usage: $\frac{80}{16384} \times 100 = 0.49\%$
- **Nanopb runtime:** 12 KB
Calculation: Compiled size of pb_encode.c, pb_decode.c, pb_common.c
Usage: $\frac{12}{16384} \times 100 = 0.07\%$
- **Frame + ARQ layers:** 8 KB
Calculation: Framing logic + CRC-32 + ARQ state machine + retransmission logic
Usage: $\frac{8}{16384} \times 100 = 0.05\%$
- **Total Flash:** $80 + 12 + 8 = 100$ KB
Total usage: $\frac{100}{16384} \times 100 = 0.61\%$

1.3 Implementation Requirements

1.3.1 Protocol Stack Architecture

The protocol is implemented as a layered stack on both the RPi5 (SPI controller) and ESP32 (SPI peripheral). Each layer has specific responsibilities:

Layer	Responsibility	Implementation
5. Application	Motor commands, telemetry, control logic	Different per platform
4. Protobuf	Message encode/decode (nanopb)	Both sides
3. ARQ	Sequence numbering, ACK/NACK, retry logic	Both sides (symmetric)
2. Framing	Header/footer, CRC-32 verification	Both sides (identical)
1. SPI+DMA	Physical transport with double-buffering	Both sides (different roles)

Table 4: Protocol stack layers and implementation requirements

Key Insight: Layers 2-4 are nearly identical on both platforms. Only the application layer (Layer 5) and SPI role (Layer 1: controller vs peripheral) differ.

1.3.2 ESP32 (SPI Peripheral) Implementation

Layer 1: SPI Peripheral with DMA

- Configure SPI3 in peripheral mode using ESP-IDF
- Set up DMA with double-buffering: 2×4092 bytes
- Register DMA interrupt callbacks for buffer swapping
- Wait for chip select from RPi5, respond to clock

Layer 2: Frame Layer (Custom Code)

RX Path:

- Parse frame header (magic bytes, length, sequence number)
- Verify CRC-32 over header + payload
- If CRC fails $\rightarrow$ discard packet, send NACK
- If CRC valid $\rightarrow$ extract payload, pass to ARQ layer

TX Path:

- Build frame: [Header | Payload | CRC32 | Footer]
- Calculate CRC-32 over header + payload
- Queue complete frame to DMA buffer for transmission

Layer 3: ARQ Layer (Custom Code)

RX Path:

- Check sequence number against `last_acked_rx`
- If duplicate (`seq ≤ last_acked_rx`) → send ACK, discard payload
- If expected (`seq = last_acked_rx + 1`) → accept, send ACK
- If out-of-order → send NACK, request retransmission

TX Path:

- Assign `tx_sequence_number` to outgoing frame
- Store frame in retry buffer
- Start timeout timer (e.g., 10ms)
- On timeout → retransmit (up to 3 attempts)
- On ACK received → clear retry buffer, increment `tx_seq`

Layer 4: Protobuf (nanopb library)

- Decode incoming bytes → `SetVelocityRequest`, `EmergencyStopRequest`
- Encode outgoing structs → `SetVelocityResponse`, `TelemetryData`
- Uses `star.v1` package following Boston Dynamics style guide
- Static allocation (no malloc) with `.options` file constraints

Layer 5: Application

- Handle motor velocity commands
- Read encoder data via multiplexer
- Execute PID control loop (250 Hz per motor)
- Send telemetry responses (encoder feedback, current, temperature)

1.3.3 RPi5 (SPI Controller) Implementation

Layer 1: SPI Controller with DMA

- Use Linux `/dev/spidev` interface
- Configure SPI3: 10 Mbps, mode 0, 8-bit words
- Use `ioctl(SPI_IOC_MESSAGE)` for DMA transfers
- Implement userspace double-buffering for continuous operation

Layer 2: Frame Layer (*Custom Code - SAME AS ESP32*)

RX Path: Parse frame header, verify CRC-32, extract payload or discard on CRC failure

TX Path: Build frame [`Header` | `Payload` | `CRC32` | `Footer`], calculate CRC-32, send via SPI

Layer 3: ARQ Layer (*Custom Code - SAME AS ESP32*)

RX Path: Check sequence number, detect duplicates → ACK and discard, detect out-of-order → NACK

TX Path: Assign sequence number, store in retry buffer, timeout and retry logic, handle ACKs

Layer 4: Protobuf (Go with protoc-gen-go)

- Encode: `SetVelocityRequest`, `EmergencyStopRequest`
- Decode: `SetVelocityResponse`, `TelemetryData`, `EncoderData`
- Uses `star.v1` package following Boston Dynamics style guide

Layer 5: Application (star-gateway)

- Send motor velocity commands from Nav2 (autonomous) or UI (manual)
- All control flows through Nav2 stack for consistent behavior
- Receive encoder feedback for odometry calculations
- Bridge ROS2 topics to/from ESP32 via SPI

1.3.4 ARQ Symmetry and State Management

The ARQ layer uses **Stop-and-Wait** protocol (also called alternating bit protocol). This means each side sends one frame at a time, waits for acknowledgment (ACK) with a timeout, and retries up to 3 times before declaring failure. This approach provides:

- Simple implementation suitable for embedded systems
- Predictable worst-case latency for real-time control
- Low memory overhead (single retry buffer, not a window of frames)
- Sufficient throughput for 10 Mbps SPI with <1ms frame time

The ARQ protocol is **symmetric** — both sides implement identical Stop-and-Wait logic but maintain separate state for their own transmissions and receptions.

Each Side Maintains:

- **TX sequence number** (`tx_seq`): For packets it sends
- **RX last acknowledged** (`rx_last_ack`): For packets it receives
- **Retry buffer**: Stores unacknowledged outgoing packets
- **Timeout timer**: Triggers retransmission if no ACK received

Responsibility	ESP32 (Peripheral)	RPi5 (Controller)
Sends	Telemetry, responses	Commands, requests
TX sequence numbering	Own <code>tx_seq</code>	Own <code>tx_seq</code>
ACK/NACK generation	Sends ACK for RPi5	Sends ACK for ESP32
Retry logic	Retries if no ACK	Retries if no ACK
Duplicate detection	Checks RPi5's <code>seq</code>	Checks ESP32's <code>seq</code>

Table 5: ARQ role distribution between controller and peripheral

Important: The SPI controller/peripheral roles (Layer 1) are **hardware-level only**. The ARQ layer (Layer 3) treats both sides as equals — each can send, ACK, and retry independently.

1.3.5 Example Transaction: RPi5 Sends Velocity Command

1. **RPi5 Application:** Create `SetVelocityRequest` (left=1.0 m/s, right=1.0 m/s)
2. **RPi5 Protobuf:** Encode to binary (nanopb)
3. **RPi5 ARQ:** Assign `seq=42`, save to retry buffer, start 10ms timeout
4. **RPi5 Frame:** Build packet with header, CRC-32, footer
5. **RPi5 SPI:** DMA transfer to ESP32 (controller initiates)
6. **ESP32 SPI:** DMA receives packet in double-buffer
7. **ESP32 Frame:** Verify CRC-32 $\rightarrow$ valid
8. **ESP32 ARQ:** Check `seq=42 = rx_last_ack+1` $\rightarrow$ accept
9. **ESP32 Protobuf:** Decode to `SetVelocityRequest` struct
10. **ESP32 Application:** Update motor setpoints, execute PID control
11. **ESP32 ARQ:** Build ACK packet with `ack_seq=42`
12. **ESP32 Frame:** Wrap ACK in frame with CRC-32
13. **ESP32 SPI:** DMA transmits ACK (peripheral responds)
14. **RPi5 SPI:** DMA receives ACK packet
15. **RPi5 Frame:** Verify CRC-32 $\rightarrow$ valid
16. **RPi5 ARQ:** Got ACK(42) $\rightarrow$ clear retry buffer, cancel timeout

Outcome: Command successfully delivered with guaranteed delivery via ARQ. If ACK was lost or CRC failed, RPi5 would retransmit after 10ms timeout.

1.4 ARQ Communication Flow

1.4.1 Normal Operation (No Retransmission)

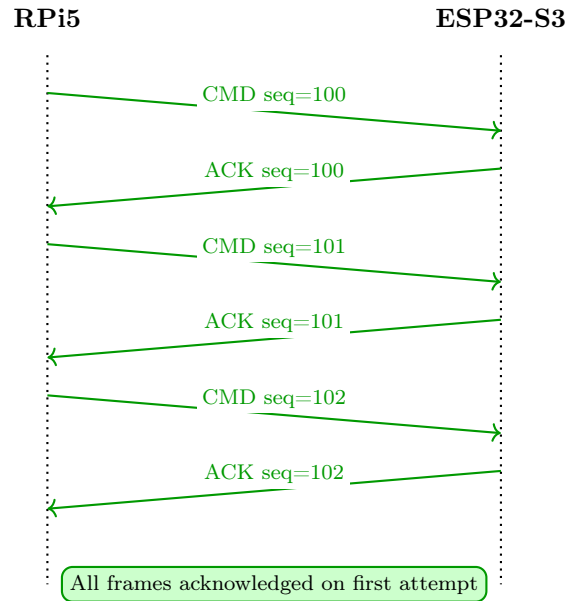


Figure 1: Normal ARQ Flow - Three Successful Transactions

1.4.2 Error Recovery Scenarios

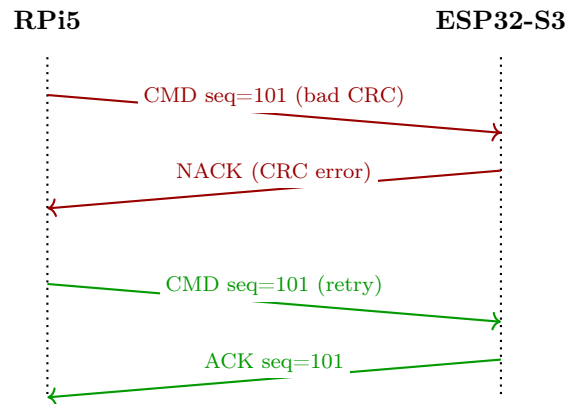


Figure 2: CRC Error - Retry with Same Sequence Number

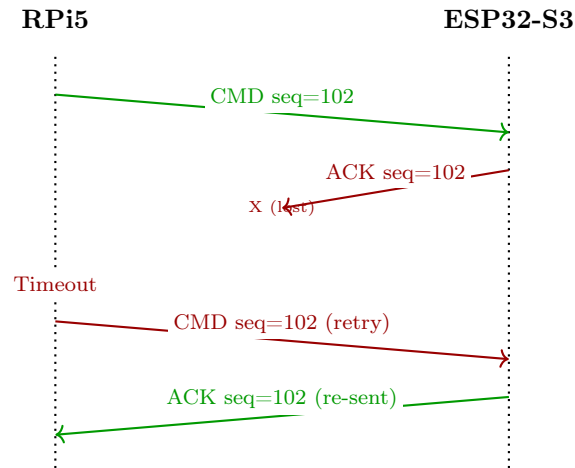


Figure 3: Duplicate Packet Scenario (Lost ACK)

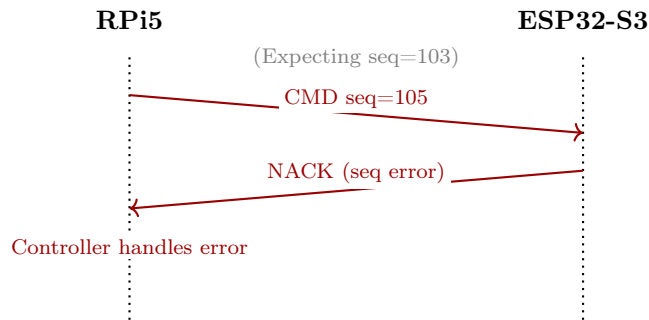


Figure 4: Out-of-Order Packet Scenario

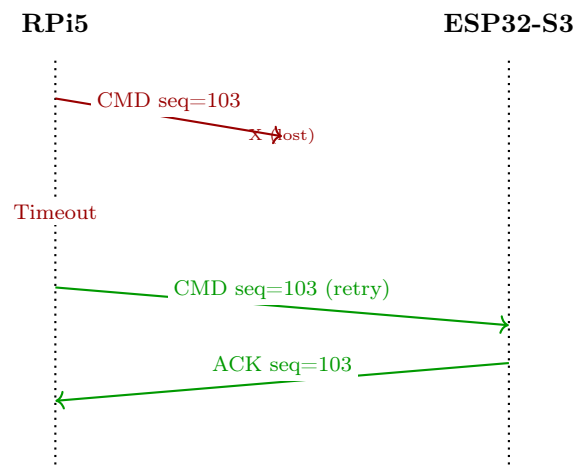


Figure 5: Lost Packet Scenario (Timeout)

1.5 Error Handling Strategies

CRC Mismatch

Action: NACK with CRC error reason
Recovery: Controller retry
Logging: Increment `crc_errors`

Sequence Error

Duplicate: Re-send ACK
Out-of-order: NACK + reject
Recovery: Controller retry

Timeout

RX timeout: Continue next cycle
500ms watchdog: Emergency stop
Retry fail: Log error

Invalid Protobuf

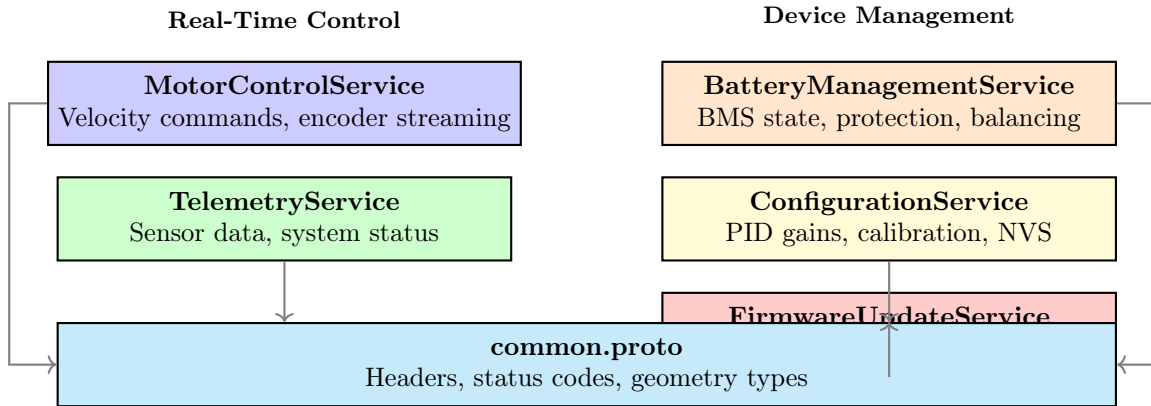
Action: NACK protocol error
Validate: IDs, ranges, offsets
Logging: Increment `proto_errors`

2 Protocol Buffer Schema Architecture

This section documents the Protocol Buffer schema architecture for RPi5↔ESP32 communication. All schemas follow the **Boston Dynamics Protobuf Style Guide** for consistency, maintainability, and industry best practices.

2.1 Architecture Overview

Overview: Six gRPC services define the complete API surface for robot control, organized by functional domain. Services communicate via the nanopb-serialized frames described in Section 1.



Key Design Principles

- **Versioned Package:** All services use `star.v1` namespace for API evolution
- **Request/Response Headers:** Every RPC includes tracing, timestamps, and status
- **Streaming Support:** Continuous data uses server streaming (up to 100Hz)
- **Static Allocation:** All messages sized for nanopb constraints (no dynamic memory)

Note: Naming conventions, unit suffixes, and coding standards are consolidated in Section ?? (Style Guide and Conventions).

2.2 Service Specifications

2.2.1 MotorControlService

Purpose: Low-latency differential drive control with real-time encoder feedback.

Target Latency: <10ms round-trip for velocity commands.

Table 6: MotorControlService RPCs

RPC	Type	Description
SetVelocity	Unary	Set left/right wheel velocities (m/s)
EmergencyStop	Unary	Immediate stop, priority over all commands
SetMotorPower	Unary	Direct PWM control (bypass PID, testing only)
StreamEncoders	Server stream	Encoder ticks and velocity at 1-100Hz
ControlStream	Bidirectional	Velocity in, encoder feedback out

Key Messages:

- **VelocityCommand** – Left/right velocity (m/s), sequence number, timestamp
- **EncoderData** – Motor ID, tick count (341.2 PPR), velocity, timestamp
- **MotorStatus** – Duty cycle, velocity, temperature, current, fault flags
- **PidConfig** – Kp, Ki, Kd gains with output saturation limits

2.2.2 TelemetryService

Purpose: Real-time sensor data aggregation and system health monitoring.

Table 7: TelemetryService RPCs

RPC	Type	Description
GetTelemetry	Unary	Snapshot of all sensor data
StreamTelemetry	Server stream	Continuous sensor updates at 1-100Hz
GetSystemStatus	Unary	Firmware version, uptime, connection states

Key Messages:

- **TelemetryData** – IMU, GPS, battery %, WiFi signal, CPU usage, temperature
- **ImuData** – Pitch/roll/yaw (rad), acceleration (m/s^2), angular velocity (rad/s)
- **GpsData** – Lat/lon (degrees), altitude (m), accuracy, satellite count, fix type
- **SystemStatus** – Connection states, operating mode, free heap, uptime

Enumerations:

- **RobotMode** – IDLE, MANUAL, AUTONOMOUS, MAPPING, EMERGENCY_STOP
- **ConnectionStatus** – CONNECTED, DISCONNECTED, CONNECTING, ERROR
- **GpsFix** – NONE, 2D, 3D, DGPS, RTK_FLOAT, RTK_FIXED

2.2.3 BatteryManagementService

Purpose: BQ7850 BMS monitoring for lithium-ion battery pack safety and management.

Hardware: TI BQ7850 16-cell BMS with I2C interface.

Table 8: BatteryManagementService RPCs

RPC	Type	Description
GetBatteryState	Unary	Complete battery snapshot (cells, temps, SOC)
StreamBatteryState	Server stream	Continuous battery monitoring
Get/SetProtectionThresholds	Unary	OV/UV/OC/OT protection limits
Enable/DisableCellBalancing	Unary	Per-cell passive balancing control
ControlFets	Unary	Charge/discharge FET enable/disable
GetDeviceInfo	Unary	Manufacturer, serial, chemistry, versions

Key Messages:

- **BatteryState** – Composite of all battery data
- **CellData** – Individual cell voltages (mV), pack voltage, valid cell count
- **BatteryTemperatureData** – Thermistor readings (0.1°C), average
- **StateOfChargeData** – Relative/absolute SOC (%), capacity (mAh), cycle count
- **ProtectionThresholds** – OV/UV voltage limits, OC current limits, OT temperatures

2.2.4 ConfigurationService

Purpose: Runtime parameter management with NVS persistence across power cycles.

Table 9: ConfigurationService RPCs

RPC	Type	Description
GetConfiguration	Unary	Read current system configuration
SetConfiguration	Unary	Apply configuration (optional NVS persist)
ResetToDefaults	Unary	Restore factory defaults
ValidateConfiguration	Unary	Validate without applying
SaveConfiguration	Unary	Persist in-memory config to NVS
Get/SetMotorPidConfig	Unary	Per-motor PID tuning

Key Messages:

- **SystemConfiguration** – Complete config with version and CRC32
- **MotorPidConfig** – Per-motor PID gains (4 motors supported)
- **CurrentSensorCalibration** – Offset and scale for current sensing
- **SafetyThresholds** – Overcurrent, thermal shutdown, cell imbalance limits
- **TimingConfig** – Motor period, telemetry period, communication timeout

2.2.5 FirmwareUpdateService

Purpose: Over-the-air (OTA) firmware updates with validation and rollback support.

Chunk Size: 1KB optimal for SPI transfer efficiency.

Table 10: FirmwareUpdateService RPCs

RPC	Type	Description
BeginUpdate	Unary	Initialize OTA partition, set expected size/CRC
WriteChunk	Unary	Write single firmware chunk
StreamChunks	Client stream	Bulk chunk transfer
FinalizeUpdate	Unary	Validate firmware, prepare for re-boot
AbortUpdate	Unary	Cancel and cleanup
Get/StreamUpdateProgress	Unary/Stream	Transfer progress and state
Reboot	Unary	Boot into new firmware
Rollback	Unary	Revert to previous firmware
MarkValid	Unary	Confirm firmware stability

Key Messages:

- **FirmwareChunk** – Data bytes, sequence number, offset
- **FirmwareUpdateProgress** – Total size, bytes received, state, CRC32
- **FirmwareInfo** – Version string, build date, git hash

Update States: IDLE → RECEIVING → VALIDATING → COMPLETE (or ERROR)

2.3 common.proto – Shared Types

All services import `common.proto` for standardized request/response handling.

Table 11: Standard Headers

Message	Fields
RequestHeader	request_id (UUID), client_timestamp, client_version, timeout
ResponseHeader	request_id (echo), server_timestamp, status, error_message, latency_us

Status Codes: UNKNOWN, OK, INVALID_REQUEST, INTERNAL_ERROR, NOT_FOUND, TIMEOUT, INVALID_STATE, ESTOP_ACTIVE

Geometric Types:

- **Vector3** – Generic 3D vector (x, y, z)
- **Quaternion** – Orientation (w, x, y, z)
- **SE2Pose** – 2D pose (x_m, y_m, angle_rad)
- **SE2Velocity** – 2D velocity (vx_mps, vy_mps, omega_rad_per_s)

2.4 Code Generation and Integration

Table 12: Code Generation Targets

Target	Generator	Output	Usage
Go nanopb	protoc-gen-go + grpc nanopb_generator	gen/go/ gen/nanopb/	star-gateway (RPi5) ESP32 firmware

nanopb Constraints: ESP32 requires static allocation. Field sizes are configured in `.options` files:

- String fields: `max_size:64` (`request_id`), `max_size:128` (`error_message`)
- Repeated fields: `max_count:16` (cell voltages), `max_count:4` (motors)
- Bytes fields: `max_size:1024` (firmware chunks)

Build Integration:

- **Linting:** `buf lint` enforces style guide compliance
- **Breaking Changes:** `buf breaking` detects API incompatibilities
- **CI Pipeline:** Automatic generation and testing on PR

3 Hardware Pin Connections

3.1 RX72N Microcontroller Pinout (R5F572NNHGFP#30)

Hardware Advantage: The RX72N provides **182 direct GPIO pins** with dedicated peripheral functions. Unlike the ESP32 design, **no multiplexers or decoders are required** - each motor gets its own dedicated MTU channels for both PWM and encoder, with direct GPIO access to fault lines and current sense ADC channels.

3.1.1 Package Information

- **MCU:** Renesas RX72N (R5F572NNHGFP#30)
- **Package:** 100-pin LFQFP (Low-profile Quad Flat Package)
- **GPIO Pins:** 182 (multiplexed across package pins)
- **Flash:** 4 MB
- **RAM:** 1 MB
- **CPU:** RXv3 core at 240 MHz

3.1.2 Motor Control Pin Assignments

Direct Motor Control - No Multiplexing Required

Motor	PWM Channel	Encoder Channel	Fault Pin	Current Sense
Motor 0 (FL)	MTU3 (MTIOC3A/B)	MTU1 (Phase Count)	PORT3.2	AN000 (ADC)
Motor 1 (FR)	MTU4 (MTIOC4A/B)	MTU2 (Phase Count)	PORT3.3	AN001 (ADC)
Motor 2 (RL)	MTU6 (MTIOC6A/B)	MTU0 (Phase Count)	PORT3.4	AN002 (ADC)
Motor 3 (RR)	MTU7 (MTIOC7A/B)	MTU5 (Phase Count)	PORT3.5	AN003 (ADC)

Table 13: RX72N motor control pin assignments (FL=Front-Left, FR=Front-Right, RL=Rear-Left, RR=Rear-Right)

PWM Configuration:

- Frequency: 20 kHz (center-aligned PWM mode 1)
- Dead-time: 1 μ s for H-bridge safety
- Resolution: $\sim$ 12-bit (3000 counts at 120 MHz PCLKA)
- MTIOCA: PH (Phase) signal to DRV8243S
- MTIOCB: EN (Enable) signal to DRV8243S

Encoder Configuration:

- Mode: MTU Phase Counting Mode 1 (4x decoding)
- Inputs: MTCLKA (Phase A), MTCLKB (Phase B)
- Resolution: 341 PPR (Pulses Per Revolution)
- Counter: 16-bit hardware counter with software wraparound tracking

- Update Rate: Read at 250 Hz control loop

Fault Detection:

- Active-low nFAULT signals from DRV8243S
- Direct GPIO input (no external hardware needed)
- Polled at 250 Hz control loop

Current Sensing:

- IPROPI analog output from DRV8243S
- 12-bit S12ADFa ADC with PWM synchronization
- Range: 0-3.3V (0-2000 mA motor current)
- Sample time: 24 states for low-noise acquisition

3.1.3 Communication Interfaces

Debug UART (SCI1):

- PORT2.6: TXD1 (Transmit)
- PORT2.5: RXD1 (Receive)
- Baud Rate: 115200 bps
- Purpose: Debug output, logging, firmware update

SPI to Raspberry Pi 5 (RSPI0):

- PORT7.4: SS0A0 (Chip Select)
- PORT7.5: RSPCKA (Serial Clock)
- PORT7.6: MISOA (Controller In Peripheral Out - CIPO)
- PORT7.7: MOSIA (Controller Out Peripheral In - COPI)
- Mode: Peripheral (RX72N receives commands from RPi5)
- Speed: 10 Mbps
- Protocol: nanospb + CRC-32

I2C Bus 0 (RIIC0) - BQ25798 Charger:

- SDA0, SCL0: I2C data and clock
- PORT5.0: BQ25798 INT (interrupt)
- Purpose: Battery charging control

I2C Bus 1 (RIIC1) - Fuel Gauge (SMBUS):

- SDA1, SCL1: SMBUS-compatible I2C
- PORT5.1: Fuel Gauge ALERT (interrupt)
- Purpose: Battery state-of-charge monitoring
- Features: Hardware CRC-8 PEC support

3.1.4 Status and Safety

Status LEDs:

- PORT0.3: Status LED (heartbeat)
- PORT0.5: Error LED (fault indication)
- PORT0.7: Power LED (system active)

Emergency Stop:

- PORT1.0: E-STOP button input
- Pull-up resistor, active-low
- Polled at 250 Hz, immediately stops all motors

3.1.5 System Clocks

Clock Domain	Frequency	Purpose
CPU (ICLK)	240 MHz	RXv3 core execution
PCLKA	120 MHz	MTU3a (PWM, Encoder)
PCLKB	60 MHz	CMT (Timers), SCI (UART)

Table 14: RX72N clock configuration

3.2 Comparison: RX72N vs ESP32-S3 Architecture

Feature	ESP32-S3	RX72N
GPIO Pins	35 usable	182 direct
Motor PWM	MCPWM (shared)	MTU3a (dedicated)
Encoders	4x 4:1 mux + PCNT	4x dedicated MTU
Motor CS Lines	74HC238 decoder	Direct GPIO
Fault Detection	Via decoder outputs	Direct GPIO
Control Loop	250 Hz (mux overhead)	250 Hz (no mux)
Encoder Latency	$\sim 40 \mu s$ (mux switching)	$\sim 10 \mu s$ (direct)

Table 15: Hardware architecture comparison

Key Advantage: The RX72N eliminates all multiplexing overhead from the ESP32 design. Each motor has dedicated hardware resources with direct GPIO access, reducing control loop latency by 75% and simplifying the PCB design (no external decoder/mux ICs required).

3.3 Raspberry Pi 5 Pinout

The Raspberry Pi 5 remains unchanged from the original design, handling high-level control, sensor fusion, and ROS2 communication.

Pin	RPi Label	Signal Name / Connection	Status / Note
1	3v3	Not Connected	Pi 3v3 Rail (Not used)
2	5v	Not Connected	Pi 5v Rail (Not used)
3	SDA	PI_SDA	I2C Data
4	5v	Not Connected	Pi 5v Rail (Not used)
5	SCL	PI_SCL	I2C Clock
7	IO4	HCSR04_TRIG1	Ultrasonic Trigger 1
8	TXD	LiDAR_RX_PI5_TX	UART Transmit to LiDAR
9	GND	GND	Common Ground
10	RXD	LiDAR_TX_PI5_RX	UART Receive from LiDAR
11	IO17	HCSR04_ECHO1	Ultrasonic Echo 1
12	IO18	HCSR04_TRIG4	Ultrasonic Trigger 4
13	IO27	HCSR04_TRIG2	Ultrasonic Trigger 2
15	IO22	HCSR04_ECHO2	Ultrasonic Echo 2
16	IO23	IMU_RST	IMU Reset
17	3v3	Not Connected	Pi 3v3 Rail (Not used)
18	IO24	IMU_INT	IMU Interrupt
19	COPI	RX72N_SPI_COPI	SPI Controller Out
21	CIPO	RX72N_SPI_CIPO	SPI Controller In
22	IO25	HCSR04_ECHO4	Ultrasonic Echo 4
23	SCLK	RX72N_SPI_SCLK	SPI Clock
24	CE0	RX72N_SPI_CS	SPI Chip Enable 0
26	CE1	HCSR04_TRIG5	Ultrasonic Trigger 5
27	ID_SD	MOTOR_DRIVER_MOTOR3_nFAULT	Motor 3 Fault
28	ID_SC	HCSR04_ECHO5	Ultrasonic Echo 5
29	IO5	MOTOR_DRIVER_MOTOR2_nFAULT	Motor 2 Fault
31	IO6	MOTOR_DRIVER_MOTOR1_nFAULT	Motor 1 Fault
32	IO12	HCSR04_TRIG6	Ultrasonic Trigger 6
33	IO13	MOTOR_DRIVER_MOTOR0_nFAULT	Motor 0 Fault
35	IO19	HCSR04_TRIG3	Ultrasonic Trigger 3
36	IO16	HCSR04_ECHO6	Ultrasonic Echo 6
37	IO26	HCSR04_ECHO3	Ultrasonic Echo 3
38	IO20	RX72N_SPI_DRDY1	SPI Data Ready 1
40	IO21	RX72N_SPI_DRDY2	SPI Data Ready 2

Table 16: Raspberry Pi 5 GPIO pin assignments

3.3.1 RPi5 Pin Function Categories

SPI (Controller Mode - Communication with RX72N):

- Pin 24 (CE0): Chip Select
- Pin 19 (COPI): Controller Out Peripheral In
- Pin 23 (SCLK): Serial Clock
- Pin 21 (CIPO): Controller In Peripheral Out
- Pin 38 (IO20): Data Ready 1 interrupt from RX72N
- Pin 40 (IO21): Data Ready 2 interrupt from RX72N

I2C (IMU and Stereo Cameras):

- Pin 3 (SDA): I2C Data

- Pin 5 (SCL): I2C Clock
- Pin 16 (IO23): IMU Reset
- Pin 18 (IO24): IMU Interrupt

UART (RPLiDAR C1):

- Pin 8 (TXD): LiDAR RX (Pi TX)
- Pin 10 (RXD): LiDAR TX (Pi RX)

Motor Driver Fault Lines (from RX72N):

- Pin 33 (IO13): Motor 0 nFAULT
- Pin 31 (IO6): Motor 1 nFAULT
- Pin 29 (IO5): Motor 2 nFAULT
- Pin 27 (ID_SD): Motor 3 nFAULT

HC-SR04 Ultrasonic Sensors (6 sensors):

- Sensor 1: Pin 7 (TRIG), Pin 11 (ECHO)
- Sensor 2: Pin 13 (TRIG), Pin 15 (ECHO)
- Sensor 3: Pin 35 (TRIG), Pin 37 (ECHO)
- Sensor 4: Pin 12 (TRIG), Pin 22 (ECHO)
- Sensor 5: Pin 26 (TRIG), Pin 28 (ECHO)
- Sensor 6: Pin 32 (TRIG), Pin 36 (ECHO)

3.4 Additional Peripheral Connections

3.4.1 IMU (Inertial Measurement Unit)

Hardware:

- BNO055: 9-axis absolute orientation sensor
- BMP280: Barometric pressure sensor

Protocol: I2C communication

Connection: Connects to RPi5 via Header J10:

- PI_SCL (Clock) routes to RPi5 Pin 5
- PI_SDA (Data) routes to RPi5 Pin 3
- IMU_RST routes to RPi5 Pin 16 (IO23)
- IMU_INT routes to RPi5 Pin 18 (IO24)

3.4.2 Stereo Cameras

Hardware: External stereo camera modules

Protocol: I2C communication

Connection: Connects via Header J12 to the same I2C bus (PI_SCL and PI_SDA), allowing the Pi to address them alongside the IMU using different I2C addresses.

3.4.3 Temperature Sensor

Hardware: DS18B20+ 1-Wire digital temperature sensor

Protocol: 1-Wire communication

Connection: Connected to RX72N GPIO (specific pin TBD in final hardware design)

4 Protocol Buffer Style Guide

This section defines naming standards and conventions for all Protocol Buffer definitions in the STAR project, following Boston Dynamics style guidelines.

4.1 Terminology Standard

The project uses inclusive terminology following OSHWA (Open Source Hardware Association) standards.

Table 17: Communication Bus Terminology

Use	Avoid	Context
Controller/Peripheral	Master/Slave	I2C, SPI, 1-Wire bus roles
COP1	MOSI	Controller Out, Peripheral In
C1PO	MISO	Controller In, Peripheral Out
Primary/Main	Master	Configuration structures

4.2 Protocol Buffer Naming Conventions (Boston Dynamics Style)

The STAR project follows Boston Dynamics' Protocol Buffer style guide for all .proto definitions.

Table 18: Protocol Buffer Naming Conventions

Element	Convention	Example
Package	Versioned namespace	<code>star.v1</code>
Messages	PascalCase	<code>VelocityCommand</code> , <code>BatteryState</code>
Fields	snake_case + unit suffix	<code>velocity_mps</code> , <code>temperature_celsius</code>
Enums	SCREAMING_SNAKE_CASE	<code>MOTOR_STATE_RUNNING</code>
Enum Zero Value	_UNKNOWN suffix	<code>STATUS_UNKNOWN = 0</code>
Enum Values	Type name prefix	<code>ROBOT_MODE_MANUAL</code>
RPC Pattern	{Verb}Request/Response	<code>SetVelocityRequest</code>

4.3 Unit Suffixes (MKS System)

All numeric fields use SI units with explicit suffixes. This eliminates unit ambiguity and enables automatic documentation generation.

Table 19: Standard Unit Suffixes

Suffix	Unit	Proto Example	C Example
<code>_m</code>	meters	<code>x_m</code>	<code>distance_m</code>
<code>_mps</code>	meters/second	<code>velocity_mps</code>	<code>speed_mps</code>
<code>_mps2</code>	m/s^2	<code>accel_x_mps2</code>	<code>acceleration_mps2</code>
<code>_rad</code>	radians	<code>angle_rad</code>	<code>heading_rad</code>
<code>_rad_per_s</code>	rad/second	<code>omega_rad_per_s</code>	<code>angular_vel_rad_per_s</code>
<code>_celsius</code>	degrees C	<code>temperature_celsius</code>	<code>motor_temp_celsius</code>
<code>_deci_celsius</code>	0.1°C	<code>temp_deci_celsius</code>	(BMS native format)
<code>_us</code>	microseconds	<code>timestamp_us</code>	<code>elapsed_us</code>
<code>_ms</code>	milliseconds	<code>timeout_ms</code>	<code>period_ms</code>
<code>_ma</code>	milliamps	<code>current_ma</code>	<code>motor_current_ma</code>
<code>_mv</code>	millivolts	<code>cell_mv</code>	<code>pack_voltage_mv</code>
<code>_mw</code>	milliwatts	<code>power_mw</code>	<code>consumption_mw</code>
<code>_percent</code>	0–100%	<code>soc_percent</code>	<code>duty_cycle_percent</code>

4.4 General Documentation Standards

- Use Doxygen-compatible comments for all public APIs
- Document the “why” not just the “what”
- Keep configuration centralized in header files
- Avoid magic numbers – use named constants with comments
- Version hardware and firmware together using git tags

5 C Style Guide

This section defines comprehensive style guidelines for C firmware development in the STAR project. All firmware code (ESP32, RX72N, and future platforms) must follow these conventions to ensure consistency, maintainability, and safety in embedded systems development.

In this project, **Assembly code** and **Inline Assembly** should be used with extreme caution. While it can offer performance improvements or enable access to low-level hardware features, its use can make code harder to understand, maintain, and port. Always prefer writing in C unless absolutely necessary.

5.1 General Guidelines for ASM and Inline ASM

- **Avoid ASM When Possible:** Always try to write in C before resorting to assembly language. C compilers are highly optimized and can often produce efficient machine code that rivals hand-written assembly.
- **Document ASM Code Thoroughly:** Assembly code is often harder to understand than C, so it should be accompanied by thorough comments that explain **why** ASM was used, as well as what the code does.

```
1 /* Set register to zero using ASM */
2 mov r0, #0      /* Initialize r0 to 0 */
```

- **Use Inline ASM Sparingly:** Inline assembly allows you to embed assembly instructions directly in C code. It should only be used when absolutely necessary, such as when interacting with hardware registers or performing CPU-specific optimizations.
- **Constrain the Scope of Inline ASM:** When using inline assembly, constrain its scope to as few lines as possible and avoid intermixing it with complex C logic. This helps isolate potential issues and reduces the risk of bugs.

```
1 void set_flag(void)
2 {
3     asm volatile("mov r0, #1" : : : "r0"); /* Set flag in register r0 */
4 }
```

- **Always Use volatile for Inline ASM:** Inline assembly should be marked as `volatile` to prevent the compiler from optimizing it out.
- **Do Not Use Inline ASM for Optimizations:** Modern C compilers perform aggressive optimizations. Inline ASM should not be used to try and optimize code performance unless absolutely necessary, as it is often more error-prone than compiler optimizations.
- **Use Constraints Correctly:** When using inline ASM in C, use the appropriate constraints to inform the compiler how the assembly instructions interact with C variables.

```

1 int add(int a, int b)
2 {
3     int result;
4     asm volatile(
5         "add %[r], %[a], %[b]"
6         : [r] "=r" (result)      /* Output */
7         : [a] "r" (a), [b] "r" (b) /* Inputs */
8     );
9     return result;
10 }

```

5.2 When to Use ASM and Inline ASM

- **Hardware Access:** Use ASM when direct hardware access is required and C alone cannot provide the necessary control (e.g., setting or clearing specific registers).
- **Performance Critical Code:** ASM may be justified in performance-critical sections where C compiler optimizations fall short, but this should be documented and used as a last resort.
- **CPU-Specific Instructions:** Use ASM for executing instructions that are unique to the target CPU (e.g., specific ARM or x86 instructions that are not exposed in C).

In this project, **assertions** are used as a tool to catch programming errors and invalid states during development. They help ensure that assumptions made in the code are correct, and they provide valuable debugging information when things go wrong. However, assertions should never be relied on to handle runtime errors or expected conditions in production code.

5.3 Guidelines

- **Use Assertions for Debugging:** Assertions are intended to catch conditions that should never occur during normal program execution. Use them to assert assumptions about the state of the program, such as parameter validation or invariants within algorithms.

```

1 #include <assert.h>
2
3 void process_data(int* data)
4 {
5     assert(data != NULL); /* Assert that data is not NULL */
6     /* Continue processing */
7 }

```

5.4 Platform-Specific Error Checking Macros

Different platforms provide error checking macros for development and debugging:

- **ESP32 (ESP-IDF):** `ESP_ERROR_CHECK` aborts on error, `ESP_ERROR_CHECK_WITHOUT_ABORT` logs but continues
- **RX72N:** Custom error checking macros following similar patterns

- **General Pattern:** Check return values and handle errors appropriately for your platform

Note: While platform-specific macros are useful during development, production code should use explicit error handling with proper cleanup and recovery mechanisms.

Proper brace placement makes code cleaner and easier to read. This section establishes consistent brace conventions used throughout the STAR firmware codebase.

5.5 General Rules

- **Always Use Braces for Control Statements:** Every `if`, `for`, `while`, or similar control structure must have braces, even for single statements.
- **Same Line for Control Structures and Blocks:** For control structures like `if`, `for`, `while`, and blocks like structs and enums, always place the opening brace `{` on the same line as the statement.
- **Functions Have Braces on a New Line:** The only exception to the same-line rule is for functions, where the opening brace goes on its own line.

5.6 Control Structures

Opening brace on the **same line** as the control statement.

```
1  /* CORRECT - if/else with braces on same line */
2  if (ret != RX_OK) {
3      RX_LOG_ERROR(s_TAG, "Operation failed: %s", rx_err_to_name(ret));
4      return ret;
5  }
6
7  if (manager == NULL) {
8      return RX_ERR_INVALID_ARG;
9  } else if (manager->mutex == NULL) {
10     return RX_ERR_INVALID_STATE;
11 } else {
12     /* Proceed with operation */
13 }
14
15 /* CORRECT - Single statement still requires braces */
16 if (pin < 0 || pin >= GPIO_NUM_MAX) {
17     return RX_OK; /* Not an error, pin is not used */
18 }
19
20 /* WRONG - Missing braces */
21 if (ret != RX_OK)
22 return ret;
23
24 /* WRONG - Brace on new line */
25 if (ret != RX_OK)
26 {
27     return ret;
28 }
```

5.6.1 For Loops

```

1  /* CORRECT - for loop with brace on same line */
2  for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
3      if (curr->name && strcmp(curr->name, name) == 0) {
4          found_bus = curr;
5          break;
6      }
7  }
8
9  for (uint32_t i = 0; i < SPI_HOST_MAX; ++i) {
10     manager->spi_host_initialized[i] = false;
11     manager->spi_device_count[i] = 0;
12 }
13
14 /* WRONG */
15 for (uint32_t i = 0; i < count; ++i)
16 process_item(i); /* Missing braces */

```

5.6.2 While Loops

```

1  /* CORRECT - while loop with brace on same line */
2  while (curr) {
3      star_bus_config_t* next = curr->next;
4      esp_err_t destroy_result = star_bus_config_destroy(curr);
5      if (destroy_result != RX_OK && first_error == RX_OK) {
6          first_error = destroy_result;
7      }
8      curr = next;
9  }
10
11 while (error_handler_can_retry(&handler)) {
12     ret = attempt_operation();
13     if (ret == RX_OK) {
14         break;
15     }
16 }
17
18 /* WRONG */
19 while (curr)
20 {
21     process_node(curr); /* Brace on new line */
22     curr = curr->next;
23 }

```

5.7 Functions

Opening brace on a **new line** for function definitions.

```

1  /* CORRECT - Function brace on new line */
2  rx_err_t star_bus_manager_init(star_bus_manager_t*    manager,
3  const char*          tag,

```

```

4 star_error_interface_t* error_iface,
5 star_pin_interface_t*   pin_iface)
6 {
7     RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
8         "Manager pointer is NULL");
9
10    manager->buses          = NULL;
11    manager->error_iface    = error_iface;
12    manager->pin_iface      = pin_iface;
13
14    /* ... */
15
16    return RX_OK;
17 }
18
19 static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
20 gpio_num_t          pin,
21 const char*         bus_name,
22 const char*         usage,
23 bool                can_be_shared)
24 {
25     if (pin < 0 || pin >= GPIO_NUM_MAX) {
26         return RX_OK;
27     }
28     /* ... */
29 }
30
31 /* WRONG - Function brace on same line */
32 rx_err_t star_bus_manager_init(star_bus_manager_t* manager, ...) {
33     /* ... */
34 }

```

5.8 Structs and Enums

Opening brace on the **same line** as the typedef/struct/enum keyword.

```

1 /* CORRECT - Struct brace on same line */
2 typedef struct star_bus_manager {
3     star_bus_config_t*   buses;
4     SemaphoreHandle_t    mutex;
5     const char*          tag;
6     star_error_interface_t* error_iface;
7     star_pin_interface_t* pin_iface;
8     bool                 spi_host_initialized[SPI_HOST_MAX];
9     int                  spi_device_count[SPI_HOST_MAX];
10 } star_bus_manager_t;
11
12 /* CORRECT - Enum brace on same line */
13 typedef enum {
14     k_star_bus_type_none,
15     k_star_bus_type_i2c,
16     k_star_bus_type_spi,
17     k_star_bus_type_gpio,
18     k_star_bus_type_adc,

```

```
19     k_star_bus_type_count ,
20 } star_bus_type_t;
21
22 /* CORRECT - Union brace on same line */
23 union {
24     star_i2c_bus_config_t i2c;
25     star_spi_bus_config_t spi;
26     star_gpio_bus_config_t gpio;
27     star_adc_bus_config_t adc;
28 } proto;
29
30 /* WRONG - Brace on new line */
31 typedef struct star_bus_manager
32 {
33     /* ... */
34 } star_bus_manager_t;
```

5.9 Initializer Lists

Opening brace on the **same line** as the assignment.

```
1 /* CORRECT - Initializer brace on same line */
2 star_bus_event_t event = {
3     .bus_type = k_star_bus_type_i2c ,
4     .bus_name = config->name ,
5     .i2c = {
6         .port = port ,
7         .address = address ,
8         .is_write = true ,
9         .len = len
10    }
11 };
12
13 spi_device_interface_config_t spi_dev_cfg = {
14     .clock_speed_hz = 10 * 1000 * 1000 ,
15     .mode = 0 ,
16     .spics_io_num = GPIO_NUM_5 ,
17     .queue_size = 7 ,
18     .pre_cb = NULL ,
19     .post_cb = NULL ,
20 };
21
22 /* CORRECT - Short initializers can be on one line */
23 i2c_config_t i2c_conf = { .mode = I2C_MODE_MASTER };
24
25 /* WRONG - Brace on new line */
26 star_bus_event_t event =
27 {
28     .bus_type = k_star_bus_type_i2c ,
29     /* ... */
30 };
```

5.10 Closing Brace Placement

Closing braces are always on their own line, except for **else**, **else if**, and **while** in **do-while**.

```

1  /* CORRECT - else on same line as closing brace */
2  if (ret != RX_OK) {
3      handle_error(ret);
4  } else {
5      continue_operation();
6  }
7
8  /* CORRECT - do-while */
9  do {
10     ret = try_operation();
11 } while (ret == RX_ERR_TIMEOUT && retries-- > 0);
12
13 /* CORRECT - else if chain */
14 if (config->type == k_star_bus_type_i2c) {
15     /* Handle I2C */
16 } else if (config->type == k_star_bus_type_spi) {
17     /* Handle SPI */
18 } else {
19     /* Handle other */
20 }

```

5.11 Summary Table

Construct	Opening Brace Position
Functions	New line
if/else/else if	Same line
for	Same line
while	Same line
do-while	Same line
switch	Same line
struct	Same line
enum	Same line
union	Same line
Initializer lists	Same line

Switch statements follow specific formatting rules to ensure consistency and readability throughout the STAR firmware codebase.

5.12 General Rules

- **Same Line for Switch and Opening Brace:** The opening brace { for a **switch** statement should be on the same line as the **switch** keyword.
- **Braces for Case Blocks:** Each **case** should have its own block of code enclosed in braces {}.
- **Break Statements:** Every case must end with **break**, **return**, or a fall-through comment.
- **Default Case:** Always include a **default** case, even if it only logs a warning.

5.13 Basic Switch Structure

```

1  /* CORRECT - from star_bus_manager.c */
2  switch (config->type) {
3      case k_star_bus_type_i2c: {
4          ret = internal_register_bus_pin(pin_iface,
5          config->proto.i2c.config.sda_io_num,
6          config->name,
7          "I2C SDA",
8          true);
9          if (ret != RX_OK && first_error == RX_OK) {
10             first_error = ret;
11         }
12
13         ret = internal_register_bus_pin(pin_iface,
14         config->proto.i2c.config.scl_io_num,
15         config->name,
16         "I2C SCL",
17         true);
18         if (ret != RX_OK && first_error == RX_OK) {
19             first_error = ret;
20         }
21         break;
22     }
23
24     case k_star_bus_type_spi: {
25         ret = internal_register_bus_pin(pin_iface,
26         config->proto.spi.copi_io_num,
27         config->name,
28         "SPI COPI",
29         true);
30         if (ret != RX_OK && first_error == RX_OK) {
31             first_error = ret;
32         }
33         /* ... more pin registrations ... */
34         break;
35     }
36
37     default: {
38         RX_LOG_DEBUG(s_TAG, "No pin registration for bus type: %d",
39         config->type);
40         break;
41     }
42 }

```

5.14 Case Block Braces

Each case must have its own braces. This prevents variable scope issues and improves readability.

```

1  /* CORRECT - Each case has braces */
2  switch (bus_type) {
3      case k_star_bus_type_i2c: {
4          i2c_port_t port = config->proto.i2c.port;  /* Local variable */

```

```

5     uint8_t address = config->proto.i2c.address;
6     RX_LOG_INFO(s_TAG, "I2C bus on port %d, address 0x%02X", port,
7         address);
8     break;
9 }
10 case k_star_bus_type_spi: {
11     spi_host_device_t host = config->proto.spi.host;  /* Different scope
12         */
13     RX_LOG_INFO(s_TAG, "SPI bus on host %d", host);
14     break;
15 }
16 case k_star_bus_type_gpio: {
17     uint8_t pin_count = config->proto.gpio.pin_count;
18     RX_LOG_INFO(s_TAG, "GPIO bus with %d pins", pin_count);
19     break;
20 }
21 default: {
22     RX_LOG_WARN(s_TAG, "Unknown bus type: %d", bus_type);
23     break;
24 }
25 }
26
27 /* WRONG - Missing braces */
28 switch (bus_type) {
29     case k_star_bus_type_i2c:
30         handle_i2c();
31         break;
32     case k_star_bus_type_spi:
33         handle_spi();
34         break;
35 }

```

5.15 Break Statement Requirements

Every case must have a clear exit: `break`, `return`, or explicit fall-through.

```

1  /* CORRECT - Using break */
2  switch (config->type) {
3      case k_star_bus_type_i2c: {
4          process_i2c(config);
5          break;
6      }
7      case k_star_bus_type_spi: {
8          process_spi(config);
9          break;
10     }
11     default: {
12         break;
13     }
14 }
15
16 /* CORRECT - Using return */
17 const char* star_bus_type_to_string(star_bus_type_t type)
18 {

```

```

19  switch (type) {
20      case k_star_bus_type_none: {
21          return "NONE";
22      }
23      case k_star_bus_type_i2c: {
24          return "I2C";
25      }
26      case k_star_bus_type_spi: {
27          return "SPI";
28      }
29      case k_star_bus_type_gpio: {
30          return "GPIO";
31      }
32      case k_star_bus_type_adc: {
33          return "ADC";
34      }
35      case k_star_bus_type_count: {
36          return "COUNT";
37      }
38      default: {
39          return "UNKNOWN";
40      }
41  }
42 }
43
44  /* CORRECT - Explicit fall-through (rare, must be commented) */
45  switch (priority) {
46      case PRIORITY_CRITICAL: {
47          log_critical(message);
48          /* FALLTHROUGH */
49      }
50      case PRIORITY_HIGH: {
51          notify_admin(message);
52          /* FALLTHROUGH */
53      }
54      case PRIORITY_NORMAL: {
55          log_message(message);
56          break;
57      }
58      default: {
59          break;
60      }
61  }

```

5.16 Default Case Handling

Always include a default case. Use it for error handling or logging unexpected values.

```

1  /* CORRECT - Default case logs warning */
2  switch (curr->type) {
3      case k_star_bus_type_i2c: {
4          reg_result = register_pin_if_valid(curr->proto.i2c.config.sda_io_num,
5          bus_name, "I2C SDA", reg_func,
6          user_ctx, true);

```

```

7      /* ... */
8      break;
9  }
10 case k_star_bus_type_spi: {
11     reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
12     bus_name, "SPI COPI", reg_func,
13     user_ctx, true);
14     /* ... */
15     break;
16 }
17 default: {
18     RX_LOG_WARN(manager->tag, "Skipping pin registration for "
19     "unsupported bus type: %d", curr->type);
20     break;
21 }
22 }
23
24 /* CORRECT - Default case returns error for invalid input */
25 rx_err_t process_bus_type(star_bus_type_t type)
26 {
27     switch (type) {
28         case k_star_bus_type_i2c: {
29             return process_i2c();
30         }
31         case k_star_bus_type_spi: {
32             return process_spi();
33         }
34         default: {
35             RX_LOG_ERROR(s_TAG, "Invalid bus type: %d", type);
36             return RX_ERR_INVALID_ARG;
37         }
38     }
39 }

```

5.17 Switch on Enum Types

When switching on an enum, consider handling all values explicitly.

```

1  /* CORRECT - All enum values handled */
2  switch (config->type) {
3      case k_star_bus_type_none: {
4          RX_LOG_WARN(s_TAG, "Bus type is NONE - skipping");
5          break;
6      }
7      case k_star_bus_type_i2c: {
8          /* Initialize I2C */
9          break;
10     }
11     case k_star_bus_type_spi: {
12         /* Initialize SPI */
13         break;
14     }
15     case k_star_bus_type_gpio: {
16         /* Initialize GPIO */

```

```
17     break;
18 }
19 case k_star_bus_type_adc: {
20     /* Initialize ADC */
21     break;
22 }
23 case k_star_bus_type_count: {
24     /* Sentinel value - should never be used */
25     RX_LOG_ERROR(s_TAG, "Invalid bus type: count sentinel");
26     return RX_ERR_INVALID_ARG;
27 }
28 /* No default needed when all enum values are handled */
29 /* Compiler warns if new enum values are added */
30 }
```

5.18 Indentation

Case labels are at the same indentation level as the switch. Case body is indented one level.

```
1  /* CORRECT indentation */
2  switch (state) {
3      case STATE_IDLE: {
4          /* Case body indented */
5          if (condition) {
6              /* Nested code further indented */
7          }
8          break;
9      }
10     case STATE_RUNNING: {
11         process_running();
12         break;
13     }
14     default: {
15         break;
16     }
17 }
18
19 /* WRONG - Case labels indented */
20 switch (state) {
21     case STATE_IDLE: {
22         break;
23     }
24 }
```

5.19 Summary

- Opening brace on same line as `switch`
- Each `case` has its own braces `{...}`
- Every case ends with `break`, `return`, or fall-through comment
- Always include `default` case

- Case labels at switch indentation level
- Case body indented one level inside braces

The maximum line length is **80 characters**. Lines longer than this should be split.

- **Break at Natural Points:** Break lines at natural points, such as after operators.
- **Indentation for Continuation Lines:** When breaking lines, ensure the continued line is indented properly.

All functions must include Doxygen comments. This section establishes comprehensive commenting standards for the STAR firmware codebase.

5.20 General Commenting Guidelines

- **Use Block Comments** (*/* */*): Always prefer block comments over line comments (*//*).
- **Explain Why, Not What:** Comments should explain the reasoning behind code, not what the code does (which should be clear from the code itself).
- **Keep Comments Updated:** Outdated comments are worse than no comments.
- **English Only:** All comments must be in English.

```

1  /* CORRECT - Block comment style */
2  /* Initialize the mutex for thread-safe access */
3  manager->mutex = xSemaphoreCreateMutex();
4
5  /* WRONG - Line comment style */
6  // Initialize the mutex for thread-safe access
7  manager->mutex = xSemaphoreCreateMutex();

```

5.21 File Header Comments

Every source and header file should begin with a file path comment, followed by include guards (for headers) and a @file Doxygen block for headers with significant content.

```

1  /* lib/star_bus/include/star_bus_manager.h */
2
3  #ifndef STAR_COMPONENT_BUS_MANAGER_H
4  #define STAR_COMPONENT_BUS_MANAGER_H
5
6  #ifdef __cplusplus
7  extern "C" {
8      #endif
9
10     #include "esp_err.h"
11     #include "star_bus_manager_types.h"
12
13     /**
14      * @file star_bus_manager.h
15      * @brief Unified bus manager for I2C, SPI, GPIO, and other bus protocols

```

```

16  *
17  * The bus manager provides a central registry for managing multiple bus
18  * configurations. It supports dependency injection of error handlers and
19  * pin validators for loose coupling.
20  *
21  * Key Features:
22  * - Thread-safe bus management with mutex protection
23  * - Support for multiple bus types (I2C, SPI, GPIO, DHT22, etc.)
24  * - Automatic pin registration and conflict detection
25  * - Safe bus access via callback pattern (prevents use-after-free)
26  *
27  * Example Usage:
28  * @code
29  * // === Basic Bus Manager Setup ===
30  *
31  * #include "star_bus_manager.h"
32  * #include "star_bus_config.h"
33  * #include "star_error_handler.h"
34  *
35  * // 1. Create error handler and pin validator interfaces
36  * error_handler_t error_handler;
37  * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
38  *
39  * star_error_interface_t error_iface;
40  * star_pin_interface_t pin_iface;
41  * error_handler_get_interface(&error_iface, &error_handler);
42  * pin_validator_get_interface(&pin_iface);
43  *
44  * // 2. Initialize bus manager with dependency injection
45  * star_bus_manager_t bus_manager;
46  * star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
47  * @endcode
48  */

```

5.22 Source File Header

Source files begin with a file path comment.

```

1  /* lib/star_bus/src/star_bus_manager.c */
2
3  #include "star_bus_manager.h"
4
5  #include "freertos/FreeRTOS.h"
6  #include "freertos/semphr.h"
7  /* ... */

```

5.23 Doxygen Documentation

5.23.1 Function Documentation

All public functions must have Doxygen documentation in the header file.

```

1  /**

```

```

2  * @brief Initialize a bus manager instance.
3  *
4  * Allocates resources (like a mutex) for the manager. Must be called
5  * before any other manager functions.
6  *
7  * @param[out] manager      Pointer to bus manager structure to initialize.
8  *                          Must not be NULL.
9  * @param[in]  tag          Optional tag string for logging associated with
10 *                          this manager instance. If NULL or empty, a
11 *                          default tag is used.
12 * @param[in]  error_iface  Optional error handler interface (can be NULL).
13 * @param[in]  pin_iface    Optional pin validator interface (can be NULL).
14 * @return rx_err_t RX_OK on success, RX_ERR_INVALID_ARG if manager
15 *                  is NULL, ESP_ERR_NO_MEM if mutex creation fails.
16 */
17 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
18 const char* tag,
19 star_error_interface_t* error_iface,
20 star_pin_interface_t* pin_iface);

```

5.23.2 Parameter Direction Markers

Use [in], [out], or [in,out] to indicate parameter direction.

```

1  /**
2  * @brief Write data to an I2C device register.
3  *
4  * @param[in]  config      Pointer to I2C bus configuration.
5  * @param[in]  data        Pointer to data buffer to write.
6  * @param[in]  len         Number of bytes to write.
7  * @param[in]  reg_addr    Target register address.
8  * @param[out] bytes_written Pointer to store actual bytes written.
9  *                        Can be NULL if not needed.
10 * @return rx_err_t RX_OK on success.
11 */
12 static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
13 config,
14 const uint8_t* data,
15 size_t len,
16 uint8_t reg_addr,
17 size_t* bytes_written);
18
19 /**
20 * @brief Record an error and manage retry/reset logic
21 *
22 * @param[in,out] handler  Pointer to error handler structure
23 * @param[in]  error       Error code that occurred
24 * @param[in]  description Human-readable description of the error
25 * @param[in]  file        Source file where error occurred
26 * @param[in]  line        Line number where error occurred
27 * @param[in]  func        Function name where error occurred
28 * @return RX_OK if error was handled, or the original error code.
29 */
30 rx_err_t error_handler_record_error(error_handler_t* handler,

```

```

30 rx_err_t      error,
31 const char*   description,
32 const char*   file,
33 int32_t       line,
34 const char*   func);

```

5.23.3 Code Examples with @code/@endcode

Include usage examples in documentation blocks.

```

1  /**
2  * @file star_error_interface.h
3  * @brief Error handler interface for dependency inversion
4  *
5  * Example Usage:
6  * @code
7  * // === Creating and Using Error Interface ===
8  *
9  * #include "star_error_interface.h"
10 * #include "star_error_handler.h"
11 *
12 * // 1. Create concrete error handler
13 * error_handler_t error_handler;
14 * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
15 *
16 * // 2. Get interface from implementation
17 * star_error_interface_t error_iface;
18 * error_handler_get_interface(&error_iface, &error_handler);
19 *
20 * // 3. Inject into components that need error handling
21 * star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
22 *
23 * // 4. Use the interface
24 * if (error_iface.can_retry(error_iface.ctx)) {
25 *     // Retry operation...
26 * }
27 * @endcode
28 */

```

5.23.4 Struct Member Documentation

Document struct members inline with Doxygen.

```

1  /**
2  * @brief Bus configuration structure (common part).
3  */
4  typedef struct star_bus_config {
5      const char*   name;           /**< Unique name for this bus/device
6          instance */
7      star_bus_type_t type;         /**< Type of bus (I2C, SPI, etc.) */
8      bool          initialized;    /**< Whether this bus has been initialized
9          */
10     void*          handle;         /**< Driver/device handle */

```

```

9  void*          user_ctx;    /**< User context pointer for callbacks */
10
11  union {
12      star_i2c_bus_config_t  i2c;    /**< I2C-specific configuration */
13      star_spi_bus_config_t  spi;    /**< SPI-specific configuration */
14      star_gpio_bus_config_t gpio;    /**< GPIO-specific configuration */
15      star_adc_bus_config_t  adc;    /**< ADC-specific configuration */
16  } proto;
17
18  struct star_bus_config* next; /**< Next bus config in linked list */
19 } star_bus_config_t;

```

5.24 Special Comment Keywords

Use these keywords to mark special comments for tracking.

```

1  /* TODO: Implement retry logic for I2C bus reset */
2
3  /* FIXME: Memory leak when bus removal fails mid-operation */
4
5  /* BUG: Race condition possible if mutex timeout is too short */
6
7  /* XXX: This code assumes little-endian byte order */
8
9  /* NOTE: ESP-IDF uses mosi_io_num internally - we map to COPI here */
10
11 /* WARNING: Must hold mutex before calling this function */

```

5.24.1 Keyword Definitions

- **TODO:** Work that needs to be done but is not urgent.
- **FIXME:** Known bugs or issues that need to be fixed.
- **BUG:** Confirmed bugs in the code.
- **XXX:** Dangerous or hacky code that works but should be reviewed.
- **NOTE:** Important information about the code.
- **WARNING:** Critical information that must be understood before modifying.

5.25 Section Comments

Use section comments to organize code within files.

```

1  /* === Includes === */
2  #include "star_bus_manager.h"
3  #include "freertos/FreeRTOS.h"
4
5  /* === Constants === */
6  static const char* s_TAG = "STAR_BUS_MANAGER";
7

```

```

8  /* === Private Function Prototypes === */
9  static esp_err_t internal_register_bus_pin(...);
10
11 /* === Public Functions === */
12 esp_err_t star_bus_manager_init(...) { /* ... */ }
13
14 /* === Private Functions === */
15 static rx_err_t internal_register_bus_pin(...) { /* ... */ }
16
17 /* --- Pin Registration Helpers --- */
18 /* Subsection within a section */

```

5.26 Aligned Comments

Align comments after declarations for improved readability.

```

1  typedef struct error_handler {
2      uint32_t      error_count;          /**< Total errors recorded */
3      uint32_t      max_retries;          /**< Max retry attempts */
4      uint32_t      current_retry;        /**< Current retry counter */
5      uint32_t      base_retry_delay;     /**< Base delay (ms) */
6      uint32_t      max_retry_delay;      /**< Max delay (ms) */
7      uint32_t      current_retry_delay;  /**< Current delay with backoff
8                                          */
9      rx_err_t      last_error;           /**< Last recorded error code */
10     bool           in_error_state;        /**< Whether in error state */
11     void*          reset_context;         /**< Context for reset function
12                                          */
13     SemaphoreHandle_t mutex;              /**< Mutex for thread-safety */
14
15     rx_err_t (*reset_fn)(void* context); /**< Optional reset function */
16 } error_handler_t;

```

5.27 Implementation Comments

Comments within function bodies explain non-obvious logic.

```

1  rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2  star_bus_config_t* config)
3  {
4      /* Validate inputs using ESP-IDF macro */
5      RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG,
6      s_TAG, "Manager or config pointer is NULL");
7
8      rx_err_t ret = RX_OK;
9
10     /* Acquire mutex with timeout to prevent deadlock */
11     if (xSemaphoreTake(manager->mutex,
12     pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
13     != pdTRUE) {
14         RX_LOG_ERROR(manager->tag, "Timeout acquiring mutex");
15         return RX_ERR_TIMEOUT;
16     }

```

```

17
18  /* Check for duplicate name - bus names must be unique */
19  for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next)
20  {
21      if (curr->name && strcmp(curr->name, config->name) == 0) {
22          ret = RX_ERR_INVALID_STATE;
23          goto add_give_mutex; /* Single exit point pattern */
24      }
25
26      /* Initialize hardware before adding to list
27      * This ensures manager's SPI host tracking is updated correctly */
28      if (!config->initialized) {
29          ret = star_bus_config_init(config, manager);
30          if (ret != RX_OK) {
31              goto add_give_mutex;
32          }
33      }
34
35      /* Add to head of list (O(1) insertion) */
36      config->next = manager->buses;
37      manager->buses = config;
38
39      add_give_mutex:
40      xSemaphoreGive(manager->mutex);
41      return ret;
42  }

```

5.28 Private Function Documentation

Static functions should have brief documentation in the source file.

```

1  /**
2   * @brief Helper to register a single pin via the interface
3   */
4  static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
5  gpio_num_t          pin,
6  const char*         bus_name,
7  const char*         usage,
8  bool                can_be_shared)
9  {
10     if (pin < 0 || pin >= GPIO_NUM_MAX) {
11         return RX_OK; /* Not an error, pin is not used */
12     }
13
14     if (!pin_iface || !pin_iface->register_pin) {
15         return RX_OK; /* No interface, skip registration */
16     }
17
18     /* Create description: "BusName: Usage" (e.g., "I2C0: SDA") */
19     char desc[64];
20     snprintf(desc, sizeof(desc), "%s: %s", bus_name, usage);
21

```

```
22  return pin_iface->register_pin(pin_iface->ctx, pin, desc,  
23  can_be_shared);  
}
```

5.29 Summary

- Use block comments (`/* */`), never line comments (`//`)
- Document all public functions with Doxygen in headers
- Use `[in]`, `[out]`, `[in,out]` for parameter direction
- Include `@code/@endcode` examples in file and complex function docs
- Use `TODO`, `FIXME`, `BUG`, `XXX`, `NOTE`, `WARNING` keywords for tracking
- Align comments after struct members for readability
- Use section comments to organize source files

This section outlines the best practices for ensuring thread-safe and efficient concurrent programming.

5.30 RTOS Types, Mutexes, Semaphores, and Atomic Operations

- **RTOS Types:** Use RTOS types when developing for embedded systems like ESP32.
- **Mutexes:** Use a mutex when you need **exclusive access** to a shared resource.
- **Semaphores:** Use semaphores when you need to manage **shared resources** that multiple threads can access simultaneously.
- **Atomic Operations:** Use atomic operations when working with **simple data types** such as counters or flags.

This section establishes the standards for defining and using enumerations in the STAR firmware codebase.

5.31 General Rules

- **Enum Names Should End in `_t`:** All enum types should end with `_t`.
- **Use Snake Case for Enum Values:** All enum values should follow `snake_case` naming conventions.
- **Enum Values Should Start with `k_`:** To distinguish enum members from other constants, values should start with `k_`.
- **Use `typedef`:** Always use `typedef` to define the enum type.
- **Include Count Sentinel:** Include a `_count` sentinel value for iteration.

5.32 Basic Enum Structure

```

1  /* CORRECT - from star_bus_common_types.h */
2  typedef enum {
3      k_star_bus_type_none,    /**< No bus type specified */
4      k_star_bus_type_i2c,    /**< I2C bus */
5      k_star_bus_type_spi,    /**< SPI bus */
6      k_star_bus_type_gpio,   /**< GPIO bus */
7      k_star_bus_type_adc,    /**< ADC bus */
8      k_star_bus_type_count,  /**< Number of bus types (sentinel) */
9  } star_bus_type_t;
10
11 /* WRONG - Various style violations */
12 typedef enum {
13     STAR_BUS_TYPE_I2C,        /* Missing k_ prefix */
14     StarBusTypeSpi,          /* PascalCase not allowed */
15     star_bus_type_gpio,      /* Missing k_ prefix */
16 } bad_enum_style;           /* Missing _t suffix */

```

5.33 Explicit Value Assignment

Assign explicit values when needed for stability or compatibility.

```

1  /* CORRECT - Explicit values for protocol compatibility */
2  typedef enum {
3      k_error_none = 0,    /**< No error */
4      k_error_minor = 1,  /**< Minor error, operation can continue */
5      k_error_major = 2,  /**< Major error, operation should stop */
6      k_error_fatal = 3,  /**< Fatal error, system needs reset */
7      k_error_count = 4,  /**< Number of error levels */
8  } error_level_t;
9
10 /* CORRECT - Bit flags with explicit values */
11 typedef enum {
12     k_bus_flag_none = 0x00, /**< No flags */
13     k_bus_flag_initialized = 0x01, /**< Bus is initialized */
14     k_bus_flag_busy = 0x02, /**< Bus is busy */
15     k_bus_flag_error = 0x04, /**< Bus has error */
16     k_bus_flag_dma = 0x08, /**< DMA enabled */
17 } bus_flags_t;

```

5.34 Documentation

Document each enum value with Doxygen comments.

```

1  /**
2   * @brief Types of supported bus interfaces.
3   */
4  typedef enum {
5      k_star_bus_type_none,    /**< No bus type specified */
6      k_star_bus_type_i2c,    /**< I2C bus */
7      k_star_bus_type_spi,    /**< SPI bus */
8      k_star_bus_type_gpio,   /**< GPIO bus */

```

```

9   k_star_bus_type_adc,    /**< ADC bus */
10  k_star_bus_type_count, /**< Number of bus types */
11 } star_bus_type_t;

```

5.35 Enum to String Functions

Provide a string conversion function for each enum type.

```

1  /* In header file */
2  const char* star_bus_type_to_string(star_bus_type_t type);
3
4  /* In source file */
5  const char* star_bus_type_to_string(star_bus_type_t type)
6  {
7      switch (type) {
8          case k_star_bus_type_none: {
9              return "NONE";
10             }
11          case k_star_bus_type_i2c: {
12              return "I2C";
13             }
14          case k_star_bus_type_spi: {
15              return "SPI";
16             }
17          case k_star_bus_type_gpio: {
18              return "GPIO";
19             }
20          case k_star_bus_type_adc: {
21              return "ADC";
22             }
23          case k_star_bus_type_count: {
24              return "COUNT";
25             }
26          default: {
27              return "UNKNOWN";
28             }
29      }
30  }
31
32  /* Usage */
33  RX_LOG_INFO(s_TAG, "Added bus config: %s (Type: %s)",
34  config->name, star_bus_type_to_string(config->type));

```

5.36 Using the Count Sentinel

The `_count` value enables iteration and array sizing.

```

1  /* Array indexed by enum */
2  static const char* s_bus_type_names[k_star_bus_type_count] = {
3      [k_star_bus_type_none] = "NONE",
4      [k_star_bus_type_i2c]  = "I2C",
5      [k_star_bus_type_spi]  = "SPI",
6      [k_star_bus_type_gpio] = "GPIO",

```

```

7   [k_star_bus_type_adc] = "ADC",
8   };
9
10  /* Iteration using count */
11  for (star_bus_type_t type = k_star_bus_type_none;
12  type < k_star_bus_type_count; ++type) {
13      RX_LOG_INFO(s_TAG, "Bus type %d: %s", type, s_bus_type_names[type]);
14  }
15
16  /* Validation using count */
17  if (type >= k_star_bus_type_count) {
18      RX_LOG_ERROR(s_TAG, "Invalid bus type: %d", type);
19      return RX_ERR_INVALID_ARG;
20  }

```

5.37 Switch Statement Pattern

Always handle all enum values or include a default case.

```

1  /* All values handled explicitly */
2  switch (config->type) {
3      case k_star_bus_type_none: {
4          RX_LOG_WARN(s_TAG, "Bus type is NONE - skipping");
5          break;
6      }
7      case k_star_bus_type_i2c: {
8          ret = init_i2c_bus(config);
9          break;
10     }
11     case k_star_bus_type_spi: {
12         ret = init_spi_bus(config);
13         break;
14     }
15     case k_star_bus_type_gpio: {
16         ret = init_gpio_bus(config);
17         break;
18     }
19     case k_star_bus_type_adc: {
20         ret = init_adc_bus(config);
21         break;
22     }
23     case k_star_bus_type_count: {
24         /* Sentinel value should never be used */
25         RX_LOG_ERROR(s_TAG, "Invalid bus type: count sentinel");
26         return RX_ERR_INVALID_ARG;
27     }
28     /* No default - compiler warns if new enum values added */
29 }

```

5.38 Forward Declaration

Enums can be forward-declared for header dependency reduction.

```

1  /* Forward declaration (C11 or later with explicit underlying type) */
2  typedef enum star_bus_type star_bus_type_t;
3
4  /* Or use typedef in header, full definition in source */
5  /* In header: */
6  typedef enum star_bus_type star_bus_type_t;
7
8  /* In source or types header: */
9  enum star_bus_type {
10     k_star_bus_type_none,
11     k_star_bus_type_i2c,
12     /* ... */
13 };

```

5.39 Naming Convention Summary

Element	Convention	Example
Type name	snake_case_t	star_bus_type_t
Member prefix	k_	k_star_bus_type_i2c
Member naming	snake_case	k_star_bus_type_i2c
Count sentinel	_count	k_star_bus_type_count
String function	_to_string	star_bus_type_to_string

This section establishes error handling patterns for the STAR firmware codebase across all platforms (ESP32, RX72N, and future targets).

5.40 General Principles

- **Return Error Codes:** Functions should return error codes via `esp_err_t`.
- **Check All Return Values:** Never ignore return values from functions that can fail.
- **Use goto for Clean-up:** Use `goto` to jump to a clean-up section when multiple resources need release.
- **Fail Fast:** Return early on invalid arguments or precondition failures.

5.41 Platform Error Codes

Use platform-specific error codes consistently. STAR defines generic error types that map to platform implementations.

```

1  /* Generic STAR error codes (map to platform-specific implementations) */
2  RX_OK                               /* Success */
3  RX_ERR_FAIL                         /* Generic failure */
4  RX_ERR_NO_MEM                       /* Out of memory */
5  RX_ERR_INVALID_ARG                 /* Invalid argument */
6  RX_ERR_INVALID_STATE               /* Invalid state for operation */
7  RX_ERR_INVALID_SIZE                /* Invalid size */
8  RX_ERR_NOT_FOUND                   /* Requested resource not found */
9  RX_ERR_NOT_SUPPORTED               /* Operation not supported */

```

```

10 RX_ERR_TIMEOUT           /* Operation timed out */
11
12 /* Usage example */
13 rx_err_t star_bus_manager_init(star_bus_manager_t* manager, ...)
14 {
15     if (manager == NULL) {
16         return RX_ERR_INVALID_ARG;
17     }
18
19     manager->mutex = xSemaphoreCreateMutex();
20     if (manager->mutex == NULL) {
21         return RX_ERR_NO_MEM;
22     }
23
24     return RX_OK;
25 }

```

5.42 Validation Macros

Use platform-provided validation macros for parameter checking and early returns. These provide consistent error handling patterns across the codebase.

5.42.1 RX_RETURN_ON_FALSE

Returns an error if condition is false, with logging. ESP32 platforms use ESP_RETURN_ON_FALSE, RX72N uses RX_RETURN_ON_FALSE.

```

1 /* Platform-specific: ESP32 uses esp_check.h, RX72N uses rx_check.h */
2 #include "rx_check.h"
3
4 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
5 const char* tag,
6 star_error_interface_t* error_iface,
7 star_pin_interface_t* pin_iface)
8 {
9     /* Validate required parameter with automatic logging */
10    RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
11        "Manager pointer is NULL");
12
13    /* Multiple validations */
14    RX_RETURN_ON_FALSE(manager->mutex,
15        RX_ERR_INVALID_STATE,
16        manager->tag,
17        "Manager mutex not initialized");
18
19    RX_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
20        config->type < k_star_bus_type_count,
21        RX_ERR_INVALID_ARG,
22        manager->tag,
23        "Invalid bus type in config");
24
25    return RX_OK;
26 }

```

5.42.2 ESP_GOTO_ON_ERROR

Jumps to a label on error, with logging. Use for cleanup patterns.

```

1 static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
      config,
2 const uint8_t*      data,
3 size_t              len,
4 uint8_t              reg_addr,
5 size_t*              bytes_written)
6 {
7     rx_err_t          ret = RX_OK;
8     i2c_cmd_handle_t cmd = NULL;
9
10    cmd = i2c_cmd_link_create();
11    RX_RETURN_ON_FALSE(cmd, RX_ERR_NO_MEM, s_TAG,
12    "Failed to create I2C command link");
13
14    ret = i2c_master_start(cmd);
15    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
16    "Write '%s': START failed: %s",
17    config->name, rx_err_to_name(ret));
18
19    ret = i2c_master_write_byte(cmd, (address << 1) | I2C_MASTER_WRITE,
20    true);
21    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
22    "Write '%s': Address (W) failed: %s",
23    config->name, rx_err_to_name(ret));
24
25    ret = i2c_master_write(cmd, data, len, true);
26    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
27    "Write '%s': Data write failed: %s",
28    config->name, rx_err_to_name(ret));
29
30    ret = i2c_master_stop(cmd);
31    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
32    "Write '%s': STOP failed: %s",
33    config->name, rx_err_to_name(ret));
34
35    /* Execute command */
36    ret = i2c_master_cmd_begin(port, cmd, pdMS_TO_TICKS(1000));
37    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
38    "Write '%s': CMD failed", config->name);
39
40    /* Success path */
41    i2c_cmd_link_delete(cmd);
42    if (bytes_written) {
43        *bytes_written = len;
44    }
45    return RX_OK;
46
47    write_fail:
48    if (cmd) {
49        i2c_cmd_link_delete(cmd);
50    }

```

```

50     return ret;
51 }

```

5.42.3 ESP_ERROR_CHECK

Use during development to catch fatal errors.

```

1  /* Development only - aborts on error */
2  /* Platform-specific error checking example (ESP32 shown) */
3  ESP_ERROR_CHECK(i2c_driver_install(port, I2C_MODE_MASTER, 0, 0, 0));
4
5  /* Generic pattern: check return value and handle errors */
6  rx_err_t ret = optional_init();
7  if (ret != RX_OK) {
8      RX_LOG_WARN(TAG, "Optional init failed: %d", ret);
9  }

```

5.43 NULL Safety Patterns

Always validate pointer parameters.

```

1  /* At function start - validate all pointers */
2  rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
3  const char*                name,
4  const uint8_t*             data,
5  size_t                     len,
6  uint8_t                    reg_addr,
7  size_t*                    bytes_written)
8  {
9      RX_RETURN_ON_FALSE(manager && name && data,
10      RX_ERR_INVALID_ARG,
11      s_TAG,
12      "Manager, name, or data is NULL");
13      RX_RETURN_ON_FALSE(len > 0, RX_ERR_INVALID_ARG, s_TAG,
14      "Write length must be > 0");
15
16      /* Clear output parameter initially */
17      if (bytes_written) {
18          *bytes_written = 0;
19      }
20
21      /* ... rest of implementation ... */
22  }
23
24  /* Check optional interface before use */
25  if (manager->error_iface && manager->error_iface->record_error) {
26      manager->error_iface->record_error(
27      manager->error_iface->ctx,
28      ret,
29      "Bus hardware initialization failed",
30      __FILE__,
31      __LINE__,
32      __func__);

```

```

33 }
34
35 /* Use convenience macro for safe interface calls */
36 #define STAR_IFACE_RECORD_ERROR(iface, err, msg) \
37 ((iface) && (iface)->record_error \
38 ? (iface)->record_error((iface)->ctx, (err), (msg), \
39 __FILE__, __LINE__, __func__) \
40 : (err))
41
42 /* Usage */
43 STAR_IFACE_RECORD_ERROR(manager->error_iface, ret, "Operation failed");

```

5.44 Mutex Error Handling

Handle mutex acquisition failures properly.

```

1 rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2 star_bus_config_t* config)
3 {
4     /* Validate arguments first */
5     RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG,
6 s_TAG, "Invalid arguments");
7     RX_RETURN_ON_FALSE(manager->mutex, ESP_ERR_INVALID_STATE,
8 manager->tag, "Mutex not initialized");
9
10    rx_err_t ret = RX_OK;
11
12    /* Acquire mutex with timeout */
13    if (xSemaphoreTake(manager->mutex,
14 pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
15 != pdTRUE) {
16        RX_LOG_ERROR(manager->tag, "Timeout acquiring mutex for adding bus
17        '%s',",
18        config->name);
19        return RX_ERR_TIMEOUT;
20    }
21
22    /* Critical section */
23    /* ... operations ... */
24
25    /* Single exit point ensures mutex release */
26    add_give_mutex:
27    xSemaphoreGive(manager->mutex);
28    return ret;
29 }

```

5.45 Error Interface Dependency Injection

Components receive error handlers through dependency injection.

```

1 /* Interface definition */
2 typedef struct star_error_interface {
3     rx_err_t (*record_error)(void* ctx, rx_err_t error,

```

```

4  const char* message, const char* file,
5  int line, const char* func);
6  bool (*can_retry)(void* ctx);
7  rx_err_t (*reset_state)(void* ctx);
8  void* ctx;
9  } star_error_interface_t;
10
11  /* Injecting error handler into component */
12  rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
13  const char* tag,
14  star_error_interface_t* error_iface,
15  star_pin_interface_t* pin_iface)
16  {
17      manager->error_iface = error_iface; /* Can be NULL */
18      /* ... */
19  }
20
21  /* Using the error interface */
22  if (ret != RX_OK) {
23      if (manager->error_iface && manager->error_iface->record_error) {
24          manager->error_iface->record_error(
25              manager->error_iface->ctx,
26              ret,
27              "Bus initialization failed",
28              __FILE__,
29              __LINE__,
30              __func__);
31      }
32  }

```

5.46 Cleanup Patterns

Use goto for consistent cleanup when multiple resources are acquired.

```

1  rx_err_t complex_init(complex_t* obj)
2  {
3      rx_err_t ret = RX_OK;
4
5      /* Allocate resources */
6      obj->buffer = malloc(BUFFER_SIZE);
7      if (!obj->buffer) {
8          ret = RX_ERR_NO_MEM;
9          goto cleanup;
10     }
11
12     obj->mutex = xSemaphoreCreateMutex();
13     if (!obj->mutex) {
14         ret = RX_ERR_NO_MEM;
15         goto cleanup;
16     }
17
18     ret = hardware_init(obj);
19     if (ret != RX_OK) {
20         goto cleanup;

```

```

21 }
22
23 return RX_OK;
24
25 cleanup:
26 /* Release resources in reverse order */
27 if (obj->mutex) {
28     vSemaphoreDelete(obj->mutex);
29     obj->mutex = NULL;
30 }
31 if (obj->buffer) {
32     free(obj->buffer);
33     obj->buffer = NULL;
34 }
35 return ret;
36 }

```

5.47 Error Logging

Use appropriate log levels for different error severities.

```

1 /* RX_LOG_ERROR - Errors that prevent operation */
2 RX_LOG_ERROR(s_TAG, "Failed to initialize bus '%s': %s",
3 config->name, rx_err_to_name(ret));
4
5 /* RX_LOG_WARN - Warnings that don't prevent operation */
6 RX_LOG_WARN(manager->tag, "Failed to register pins for bus '%s': %s "
7 "(bus still added)", config->name, rx_err_to_name(ret));
8
9 /* RX_LOG_INFO - Informational messages */
10 RX_LOG_INFO(manager->tag, "Added bus config: %s (Type: %s)",
11 config->name, star_bus_type_to_string(config->type));
12
13 /* RX_LOG_DEBUG - Debug messages (compiled out in release) */
14 RX_LOG_DEBUG(s_TAG, "Write Success: %zu bytes to '%s' (Addr 0x%02X)",
15 len, config->name, address);

```

5.48 Summary

- Return `rx_err_t` from all functions that can fail
- Use `ESP_RETURN_ON_FALSE` for parameter validation
- Use `ESP_GOTO_ON_ERROR` for cleanup patterns
- Always validate pointers before dereferencing
- Handle mutex timeouts as `ESP_ERR_TIMEOUT`
- Use dependency injection for error handlers
- Release resources in reverse order of acquisition
- Log errors with appropriate severity levels

This section establishes standards for function design, implementation, and documentation in the STAR firmware codebase.

5.49 Return Value Conventions

- If the name of a function is an action (or an imperative command), it should return an **error-code integer** (`esp_err_t`).
- If the name is a predicate (a function that returns true/false), it should return a **succeeded boolean**.

```

1  /* Action functions return rx_err_t */
2  rx_err_t star_bus_manager_init(...);
3  rx_err_t star_bus_manager_add_bus(...);
4  rx_err_t star_bus_config_destroy(...);
5
6  /* Predicate functions return bool */
7  bool error_handler_can_retry(error_handler_t* handler);
8  bool is_bus_initialized(const star_bus_config_t* config);
9
10 /* Getter functions return the value or pointer */
11 star_bus_config_t* star_bus_manager_find_bus(...);
12 const char* star_bus_type_to_string(star_bus_type_t type);

```

5.50 Private and Exposed Private Functions

5.50.1 Private Functions (static)

Functions internal to a single source file should be declared `static` with the `internal_` prefix.

```

1  /* In .c file - private static functions */
2  static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
3  gpio_num_t          pin,
4  const char*         bus_name,
5  const char*         usage,
6  bool                can_be_shared)
7  {
8      if (pin < 0 || pin >= GPIO_NUM_MAX) {
9          return RX_OK; /* Not an error, pin is not used */
10     }
11     /* ... implementation ... */
12 }
13
14 static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
15     config,
16 i2c_port_t          port,
17 i2c_cmd_handle_t    cmd,
18 const char*         operation_name)
19 {
20     /* ... implementation ... */
21 }
22 /* Declare prototypes at top of file */

```

```

23 static rx_err_t internal_register_bus_pin(...);
24 static rx_err_t internal_unregister_bus_pin(...);
25 static rx_err_t internal_register_config_pins(...);

```

5.50.2 Exposed Private Functions (priv_)

If a function cannot be static and must be exposed across files within a module (but not publicly), prefix its name with `priv_`.

```

1  /* In internal header (e.g., star_bus_internal.h) */
2  rx_err_t priv_star_bus_validate_config(const star_bus_config_t* config);
3  rx_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);
4  void      priv_star_bus_release_mutex(star_bus_manager_t* manager);

```

5.51 Parameter Validation

Validate all parameters at the start of public functions.

```

1  rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2  star_bus_config_t* config)
3  {
4      /* Validate required parameters */
5      ESP_RETURN_ON_FALSE(manager && config,
6      RX_ERR_INVALID_ARG,
7      s_TAG,
8      "Manager or config pointer is NULL");
9
10     /* Validate config contents */
11     ESP_RETURN_ON_FALSE(config->name && strlen(config->name) > 0,
12     RX_ERR_INVALID_ARG,
13     manager->tag,
14     "Config name is NULL or empty");
15
16     /* Validate enum range */
17     RX_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
18     config->type < k_star_bus_type_count,
19     RX_ERR_INVALID_ARG,
20     manager->tag,
21     "Invalid bus type in config");
22
23     /* Validate state */
24     RX_RETURN_ON_FALSE(manager->mutex,
25     RX_ERR_INVALID_STATE,
26     manager->tag,
27     "Manager mutex not initialized");
28
29     /* ... rest of implementation ... */
30 }
31
32 rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
33 const char*                name,
34 const uint8_t*             data,
35 size_t                    len,

```

```

36 uint8_t          reg_addr,
37 size_t*          bytes_written)
38 {
39     /* Validate all required pointers */
40     RX_RETURN_ON_FALSE(manager && name && data,
41     RX_ERR_INVALID_ARG,
42     s_TAG,
43     "Manager, name, or data is NULL");
44
45     /* Validate numeric constraints */
46     RX_RETURN_ON_FALSE(len > 0, RX_ERR_INVALID_ARG, s_TAG,
47     "Write length must be > 0");
48
49     /* Clear output parameters initially */
50     if (bytes_written) {
51         *bytes_written = 0;
52     }
53
54     /* ... rest of implementation ... */
55 }

```

5.52 Function Declaration Formatting

Align parameters vertically when they span multiple lines.

```

1  /* CORRECT - Aligned parameters */
2  rx_err_t star_bus_manager_init(star_bus_manager_t*    manager,
3  const char*          tag,
4  star_error_interface_t* error_iface,
5  star_pin_interface_t*  pin_iface);
6
7  star_bus_config_t* star_bus_config_create_spi_device(
8  const char*          name,
9  spi_host_device_t    host,
10 gpio_num_t           copi_pin,
11 gpio_num_t           cipo_pin,
12 gpio_num_t           sclk_pin,
13 int32_t              dma_chan,
14 const spi_device_interface_config_t* dev_cfg);
15
16 /* Function pointers in structs */
17 typedef struct star_error_interface {
18     rx_err_t (*record_error)(void*      ctx,
19     rx_err_t  error,
20     const char* message,
21     const char* file,
22     int         line,
23     const char* func);
24     bool (*can_retry)(void* ctx);
25     rx_err_t (*reset_state)(void* ctx);
26     void* ctx;
27 } star_error_interface_t;

```

5.53 Doxygen Documentation

All public functions must have complete Doxygen documentation in header files.

```

1  /**
2  * @brief Add a new bus/device configuration to the manager.
3  *
4  * Adds a previously created configuration (via star_bus_config_create_*)
5  * to the manager's internal list. The manager takes ownership
6  * conceptually,
7  * but the configuration still needs to be destroyed eventually (typically
8  * via
9  * star_bus_manager_remove_bus or star_bus_manager_deinit).
10 *
11 * The configuration must have a unique name. This function is thread-safe.
12 *
13 * @param[in] manager Pointer to the initialized bus manager.
14 *                  Must not be NULL.
15 * @param[in] config Pointer to the bus configuration structure to add.
16 *                  Must not be NULL and must have a valid name.
17 * @return rx_err_t RX_OK on success,
18 *         ESP_ERR_INVALID_ARG if manager or config is invalid,
19 *         ESP_ERR_INVALID_STATE if a bus with the same name
20 *         already exists or mutex error,
21 *         ESP_ERR_TIMEOUT if mutex times out.
22 */
23 rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
24 star_bus_config_t* config);

```

5.54 Private Function Documentation

Static functions should have brief documentation in the source file.

```

1  /**
2  * @brief Helper: Execute I2C command with retry logic
3  *
4  * Wraps i2c_master_cmd_begin() with automatic retry for transient errors.
5  *
6  * @param[in] config      Pointer to I2C bus configuration
7  * @param[in] port        I2C port number
8  * @param[in] cmd         I2C command handle
9  * @param[in] operation_name Name of operation for logging
10 * @return rx_err_t Final result after retries
11 */
12 static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
13 config,
14 i2c_port_t port,
15 i2c_cmd_handle_t cmd,
16 const char* operation_name);

```

5.55 Function Implementation Patterns

5.55.1 Single Exit Point with Cleanup

```

1 rx_err_t star_bus_manager_remove_bus(star_bus_manager_t* manager,
2   const char*      name)
3 {
4     ESP_RETURN_ON_FALSE(manager && name && strlen(name) > 0,
5     RX_ERR_INVALID_ARG, s_TAG, "Invalid arguments");
6
7     star_bus_config_t* to_remove = NULL;
8     star_bus_config_t* prev      = NULL;
9     rx_err_t          ret        = RX_ERR_NOT_FOUND;
10
11     /* Acquire mutex */
12     if (xSemaphoreTake(manager->mutex,
13     pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
14     != pdTRUE) {
15         return RX_ERR_TIMEOUT;
16     }
17
18     /* Find and unlink */
19     for (star_bus_config_t* curr = manager->buses; curr;
20     prev = curr, curr = curr->next) {
21         if (curr->name && strcmp(curr->name, name) == 0) {
22             to_remove = curr;
23             if (prev == NULL) {
24                 manager->buses = curr->next;
25             } else {
26                 prev->next = curr->next;
27             }
28             to_remove->next = NULL;
29             ret = RX_OK;
30             break;
31         }
32     }
33
34     if (ret == ESP_ERR_NOT_FOUND) {
35         goto remove_give_mutex;
36     }
37
38     /* Cleanup operations */
39     rx_err_t destroy_result = star_bus_config_destroy(to_remove);
40     if (destroy_result != RX_OK) {
41         ret = destroy_result;
42     }
43
44     remove_give_mutex:
45     xSemaphoreGive(manager->mutex);
46     return ret;
47 }

```

5.55.2 Factory Function Pattern

```

1 star_bus_config_t* star_bus_config_create_i2c(const char* name,
2   i2c_port_t      port,

```

```

3  uint8_t      address,
4  gpio_num_t   sda_pin,
5  gpio_num_t   scl_pin,
6  uint32_t     clk_speed)
7  {
8      if (name == NULL || strlen(name) == 0) {
9          RX_LOG_ERROR(s_TAG, "Invalid name for I2C bus config");
10         return NULL;
11     }
12
13     star_bus_config_t* config = calloc(1, sizeof(star_bus_config_t));
14     if (config == NULL) {
15         RX_LOG_ERROR(s_TAG, "Failed to allocate memory for I2C config");
16         return NULL;
17     }
18
19     /* Initialize common fields */
20     config->name      = strdup(name);
21     config->type       = k_star_bus_type_i2c;
22     config->initialized = false;
23     config->handle     = NULL;
24     config->next       = NULL;
25
26     /* Initialize I2C-specific fields */
27     config->proto.i2c.port      = port;
28     config->proto.i2c.address   = address;
29     config->proto.i2c.config.sda_io_num = sda_pin;
30     config->proto.i2c.config.scl_io_num = scl_pin;
31     config->proto.i2c.config.master.clk_speed = clk_speed;
32
33     /* Set default operations */
34     star_bus_i2c_init_default_ops(&config->proto.i2c.ops);
35
36     return config;
37 }

```

5.56 Callback Functions

```

1  /* Callback type definition */
2  typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
3  void* user_ctx);
4
5  /* Function using callback */
6  rx_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,
7  const char* name,
8  star_bus_callback_t callback,
9  void* user_ctx)
10 {
11     ESP_RETURN_ON_FALSE(manager && name && callback,
12     RX_ERR_INVALID_ARG, s_TAG, "Invalid arguments");
13
14     /* ... find bus and invoke callback while holding mutex ... */
15     rx_err_t callback_result = callback(found_bus, user_ctx);

```

```

16     xSemaphoreGive(manager->mutex);
17     return callback_result;
18 }
19
20
21 /* Using the callback pattern */
22 rx_err_t my_bus_callback(star_bus_config_t* bus, void* ctx) {
23     ESP_LOGI(TAG, "Bus type: %s", star_bus_type_to_string(bus->type));
24     return RX_OK;
25 }
26
27 star_bus_manager_with_bus(&manager, "sensor_i2c", my_bus_callback, NULL);

```

5.57 Summary

- Action functions return `esp_err_t`; predicates return `bool`
- Use `static` with `internal_` prefix for file-local functions
- Use `priv_` prefix for exposed-but-not-public functions
- Validate all parameters at function start
- Align parameters vertically in multi-line declarations
- Document all public functions with complete Doxygen
- Use single exit point with cleanup for complex functions
- Use factory functions for object creation

Global variables should be avoided when possible. This section establishes patterns for when they are necessary.

5.58 General Principles

- **Avoid Globals:** Global variables are strongly discouraged. Use dependency injection instead.
- **File-Scoped When Necessary:** If a global is needed, limit its scope to a single file using `static`.
- **Use Prefix Conventions:** `s_` for static file-scoped, `g_` for true globals.
- **Prefer static const Over #define:** For typed constants, use `static const`.

5.59 Static File-Scoped Variables (`s_` prefix)

Use `static` with `s_` prefix for variables that should be accessible only within a single source file.

```

1 /* CORRECT - Static file-scoped variables */
2
3 /* Logging tag - most common pattern */
4 static const char* s_TAG = "STAR_BUS_MANAGER";
5

```

```

6  /* Configuration constants - prefer static const over #define */
7  static const uint32_t s_i2c_timeout_ms = 1000;
8  static const uint32_t s_default_retry_count = 3;
9
10 /* State variables scoped to this file */
11 static bool s_module_initialized = false;
12
13 /* From star_bus_i2c.c */
14 static const char* s_TAG = "STAR_BUS_I2C";
15 static const uint32_t s_i2c_timeout_ms = 1000;
16
17 /* From star_bus_config.c */
18 static const char* s_TAG = "STAR_BUS_CONFIG";

```

5.60 Why static const Over #define

static const provides type safety and scope control that #define lacks.

```

1  /* PREFERRED - Type-safe, scoped, debugger-visible */
2  static const uint32_t s_timeout_ms = 1000;
3  static const size_t s_buffer_size = 256;
4
5  /* Use the constant with type checking */
6  uint32_t timeout = s_timeout_ms; /* Compiler checks type */
7
8  /* AVOID - No type safety, global namespace pollution */
9  #define TIMEOUT_MS 1000
10 #define BUFFER_SIZE 256
11
12 /* Issues with #define: */
13 /* 1. No type checking */
14 /* 2. Can collide with other defines */
15 /* 3. Not visible in debugger */
16 /* 4. Can have unexpected macro expansion */

```

5.61 When to Use #define

Use #define for:

- Compile-time constants for array sizes
- Conditional compilation
- Macros that cannot be functions

```

1  /* Array sizes must be compile-time constants */
2  #define STAR_BUS_NAME_MAX_LEN (32)
3
4  char bus_name[STAR_BUS_NAME_MAX_LEN]; /* OK - compile-time constant */
5
6  /* Conditional compilation */
7  #ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING
8  RX_LOG_DEBUG(s_TAG, "Debug info: %d", value);

```

```

9  #endif
10
11  /* Macros with special features */
12  #define RECORD_ERROR(handler, code, desc) \
13  do { \
14  error_handler_record_error((handler), (code), (desc), \
15  __FILE__, __LINE__, __func__); \
16  } while (0)

```

5.62 Explicit Type Usage - Always Use Fixed-Width Integer Types

CRITICAL RULE: Never use implicit types like `int`, `char`, `short`, or `long`. Always use explicit fixed-width types from `<stdint.h>`.

5.62.1 Why Explicit Types Matter

- **Platform Independence:** `int` can be 16-bit or 32-bit depending on platform
- **Clear Intent:** `uint32_t` explicitly shows the variable is unsigned and 32-bit
- **Prevents Bugs:** Implicit integer promotion and sign extension bugs are avoided
- **Embedded Safety:** Embedded systems require precise control over data sizes
- **Code Portability:** Code works identically on ESP32, RX72N, and future platforms

5.62.2 Correct Type Usage

```

1  #include <stdint.h>
2  #include <stdbool.h>
3
4  /* CORRECT - Always use explicit fixed-width types */
5
6  /* Unsigned integers */
7  uint8_t    byte_value    = 0xFF;        /* 8-bit unsigned (0 to 255) */
8  uint16_t   word_value    = 0xFFFF;      /* 16-bit unsigned (0 to 65535) */
9  uint32_t   counter       = 0;           /* 32-bit unsigned */
10 uint64_t   timestamp_us  = 0;           /* 64-bit unsigned */
11
12 /* Signed integers */
13 int8_t      temperature   = -40;         /* 8-bit signed (-128 to 127) */
14 int16_t     offset        = -1000;       /* 16-bit signed */
15 int32_t     position      = 0;           /* 32-bit signed */
16 int64_t     delta_time_us = 0;           /* 64-bit signed */
17
18 /* Boolean (not int!) */
19 bool        is_enabled    = false;       /* Use stdbool.h */
20
21 /* Size and pointer types */
22 size_t      buffer_size   = 256;         /* For sizes and counts */
23 uintptr_t   address       = 0x20000000;  /* For storing addresses */
24
25 /* WRONG - Never use these implicit types */

```

```

26 int      bad_counter;      /* Platform-dependent size! */
27 unsigned bad_value;        /* Ambiguous width! */
28 char      bad_byte;         /* Is it signed or unsigned? */
29 short     bad_small;        /* 16-bit? Not guaranteed! */
30 long      bad_large;        /* 32-bit or 64-bit? Depends on platform! */

```

5.62.3 Special Case: Characters and Strings

Only use char for actual character strings, never for byte data.

```

1  /* CORRECT - char only for strings */
2  const char* message = "Hello, STAR!";
3  char      buffer[64];
4  char      letter = 'A';
5
6  /* WRONG - Don't use char for byte data */
7  char data[256]; /* Should be uint8_t data[256] */
8  char byte = 0xFF; /* Should be uint8_t byte = 0xFF */
9
10 /* CORRECT - uint8_t for byte data */
11 uint8_t data_buffer[256];
12 uint8_t byte_value = 0xFF;

```

5.62.4 Loop Counters and Array Indices

Use uint32_t or size_t for loop counters and array indices.

```

1  /* CORRECT - Explicit types for loops */
2  for (uint32_t i = 0; i < array_length; i++) {
3      process_element(array[i]);
4  }
5
6  /* For sizes from sizeof() or string length */
7  for (size_t i = 0; i < buffer_size; i++) {
8      buffer[i] = 0;
9  }
10
11 /* WRONG - Never use int for loops */
12 for (int i = 0; i < count; i++) { /* What if count > INT_MAX? */
13     /* ... */
14 }

```

5.62.5 Function Parameters and Return Types

Always use explicit types in function signatures.

```

1  /* CORRECT - Explicit types in function signatures */
2  uint32_t calculate_checksum(const uint8_t* data, size_t length);
3  int32_t  read_temperature_celsius(uint8_t sensor_id);
4  bool     is_motor_running(uint8_t motor_id);
5
6  /* WRONG - Implicit types */
7  int calculate_value(char* data, int len); /* Ambiguous! */

```

5.62.6 Type Casting for Enum Comparisons

When comparing enums of different types, cast to the appropriate fixed-width type.

```

1 typedef enum {
2     k_channel_0 = 0,
3     k_channel_1 = 1,
4     k_channel_max = 2,
5 } channel_t;
6
7 /* CORRECT - Cast to explicit type for comparison */
8 if ((int32_t)channel >= k_channel_max) {
9     return ERROR_INVALID_CHANNEL;
10 }
11
12 /* WRONG - Direct enum comparison can trigger warnings */
13 if (channel >= k_channel_max) { /* -Werror=enum-compare */
14     return ERROR_INVALID_CHANNEL;
15 }

```

5.62.7 Summary: Type Usage Rules

- **ALWAYS:** Use `uint8_t`, `int32_t`, etc. from `<stdint.h>`
- **NEVER:** Use `int`, `unsigned`, `short`, `long`
- **ONLY:** Use `char` for actual character strings
- **USE:** `bool` from `<stdbool.h>` for boolean values
- **USE:** `size_t` for sizes and counts from `sizeof()`
- **CAST:** Enums to `int32_t` when comparing different enum types

5.63 True Global Variables (`g_` prefix)

If a truly global variable is unavoidable, use the `g_` prefix. This should be rare.

```

1 /* DISCOURAGED but sometimes necessary */
2 /* g_ prefix for true globals (visible across files) */
3 g_system_state_t g_system_state;
4
5 /* Prefer dependency injection instead */
6 /* BETTER approach: */
7 typedef struct app_context {
8     star_bus_manager_t bus_manager;
9     error_handler_t error_handler;
10    /* ... */
11 } app_context_t;
12
13 /* Pass context to functions that need it */
14 void app_process_sensors(app_context_t* ctx);

```

5.64 Dependency Injection Alternative

Replace globals with dependency injection.

```

1  /* WRONG - Global error handler */
2  error_handler_t g_error_handler; /* Global - bad */
3
4  void some_function(void) {
5      if (error) {
6          RECORD_ERROR(&g_error_handler, err, "Failed"); /* Uses global */
7      }
8  }
9
10 /* CORRECT - Dependency injection */
11 typedef struct star_bus_manager {
12     star_error_interface_t* error_iface; /* Injected dependency */
13     /* ... */
14 } star_bus_manager_t;
15
16 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
17     const char* tag,
18     star_error_interface_t* error_iface, /* Injected */
19     star_pin_interface_t* pin_iface) /* Injected */
20 {
21     manager->error_iface = error_iface;
22     /* ... */
23 }
24
25 /* Usage */
26 error_handler_t error_handler;
27 error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
28
29 star_error_interface_t error_iface;
30 error_handler_get_interface(&error_iface, &error_handler);
31
32 star_bus_manager_t bus_manager;
33 star_bus_manager_init(&bus_manager, "main", &error_iface, NULL);

```

5.65 Static Variables for Module State

When a module needs to maintain state, prefer explicit initialization and deinitialization.

```

1  /* Module-private state */
2  static struct {
3      bool initialized;
4      uint32_t transaction_count;
5      /* ... */
6  } s_module_state = {
7      .initialized = false,
8      .transaction_count = 0,
9  };
10
11 rx_err_t module_init(void)
12 {

```

```

13  if (s_module_state.initialized) {
14      return RX_ERR_INVALID_STATE;
15  }
16
17  /* Initialize module */
18  s_module_state.initialized = true;
19  return RX_OK;
20 }
21
22 rx_err_t module_deinit(void)
23 {
24     if (!s_module_state.initialized) {
25         return RX_ERR_INVALID_STATE;
26     }
27
28     /* Clean up */
29     s_module_state.initialized = false;
30     s_module_state.transaction_count = 0;
31     return RX_OK;
32 }

```

5.66 Thread Safety for Shared State

If static variables are accessed from multiple threads, protect with mutex.

```

1  /* Thread-safe module state */
2  static struct {
3      bool            initialized;
4      SemaphoreHandle_t mutex;
5      uint32_t        counter;
6  } s_shared_state;
7
8  rx_err_t safe_increment_counter(void)
9  {
10     if (xSemaphoreTake(s_shared_state.mutex, pdMS_TO_TICKS(1000)) !=
11         pdTRUE) {
12         return RX_ERR_TIMEOUT;
13     }
14
15     s_shared_state.counter++;
16
17     xSemaphoreGive(s_shared_state.mutex);
18     return RX_OK;
19 }

```

5.67 Summary

Type	Prefix	Usage
Static file-scoped	s_	Prefer this for module-internal state
True global	g_	Avoid; use dependency injection
static const	s_	Type-safe constants
#define	None	Array sizes, macros, conditional compilation

- Use `static const` for type-safe constants
- Use `s_TAG` for module logging tags
- Prefer dependency injection over global variables
- Protect shared state with mutexes
- Limit variable scope to the smallest necessary

This section establishes standards for header file organization and content in the STAR firmware codebase.

5.68 General Principles

- **Header Files Should Only Contain Declarations:** No implementations except for inline functions.
- **Minimal Includes in Headers:** Include only what is necessary for the declarations.
- **Self-Contained Headers:** Each header should compile independently.
- **One Purpose Per Header:** Split large headers into focused sub-headers.

5.69 Include Guard Naming Convention

All header files must use include guards following the pattern: `STAR_COMPONENT_FILENAME_H`

```

1  /* CORRECT - from star_bus_manager.h */
2  #ifndef STAR_COMPONENT_BUS_MANAGER_H
3  #define STAR_COMPONENT_BUS_MANAGER_H
4
5  /* ... declarations ... */
6
7  #endif /* STAR_COMPONENT_BUS_MANAGER_H */
8
9  /* CORRECT - from star_bus_config.h */
10 #ifndef STAR_COMPONENT_BUS_CONFIG_H
11 #define STAR_COMPONENT_BUS_CONFIG_H
12 /* ... */
13 #endif /* STAR_COMPONENT_BUS_CONFIG_H */
14
15 /* CORRECT - from star_error_interface.h */
16 #ifndef STAR_ERROR_INTERFACE_H
17 #define STAR_ERROR_INTERFACE_H
18 /* ... */
19 #endif /* STAR_ERROR_INTERFACE_H */
20
21 /* CORRECT - from star_pin_interface.h */
22 #ifndef STAR_PIN_INTERFACE_H
23 #define STAR_PIN_INTERFACE_H
24 /* ... */
25 #endif /* STAR_PIN_INTERFACE_H */

```

5.70 C++ Compatibility

All headers must include C++ extern “C” blocks for compatibility with C++ compilers.

```

1 #ifndef STAR_COMPONENT_BUS_MANAGER_H
2 #define STAR_COMPONENT_BUS_MANAGER_H
3
4 #ifdef __cplusplus
5 extern "C" {
6     #endif
7
8     /* All declarations go here */
9
10    #ifdef __cplusplus
11    }
12 #endif
13
14 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

5.71 Header File Structure

Headers should follow this organization:

```

1 /* lib/star_bus/include/star_bus_manager.h */
2
3 #ifndef STAR_COMPONENT_BUS_MANAGER_H
4 #define STAR_COMPONENT_BUS_MANAGER_H
5
6 #ifdef __cplusplus
7 extern "C" {
8     #endif
9
10    /* === 1. Includes === */
11    #include "esp_err.h"
12    #include "star_bus_manager_types.h"
13
14    /* === 2. File Documentation === */
15    /**
16     * @file star_bus_manager.h
17     * @brief Unified bus manager for I2C, SPI, GPIO, and other bus protocols
18     *
19     * The bus manager provides a central registry for managing multiple
20     * bus configurations. It supports dependency injection of error handlers
21     * and pin validators for loose coupling.
22     *
23     * Example Usage:
24     * @code
25     * // Initialize bus manager with dependency injection
26     * star_bus_manager_t bus_manager;
27     * star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
28     * @endcode
29     */
30
31    /* === 3. Forward Declarations (if needed) === */

```

```

32  /* Forward declarations reduce header dependencies */
33
34  /* === 4. Macros and Constants === */
35  #define STAR_BUS_NAME_MAX_LEN (32)
36
37  /* === 5. Type Definitions === */
38  /* (Usually in separate _types.h header) */
39
40  /* === 6. Function Declarations === */
41
42  /**
43   * @brief Initialize a bus manager instance.
44   *
45   * @param[out] manager      Pointer to bus manager to initialize.
46   * @param[in]  tag          Optional tag string for logging.
47   * @param[in]  error_iface  Optional error handler interface.
48   * @param[in]  pin_iface    Optional pin validator interface.
49   * @return rx_err_t RX_OK on success.
50   */
51  rx_err_t star_bus_manager_init(star_bus_manager_t*    manager,
52  const char*          tag,
53  star_error_interface_t* error_iface,
54  star_pin_interface_t* pin_iface);
55
56  /* ... more function declarations ... */
57
58  #ifdef __cplusplus
59  }
60 #endif
61
62 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

5.72 Include Order

Includes should be ordered in groups, separated by blank lines:

```

1  /* 1. Primary header (in .c file, include corresponding .h first) */
2  #include "star_bus_manager.h"
3
4  /* 2. FreeRTOS includes */
5  #include "freertos/FreeRTOS.h"
6  #include "freertos/semphr.h"
7  #include "freertos/task.h"
8
9  /* 3. Standard library includes */
10 #include <inttypes.h>
11 #include <stdbool.h>
12 #include <stddef.h>
13 #include <stdint.h>
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <string.h>
17
18 /* 4. Platform HAL includes (ESP-IDF, Renesas, etc.) */

```

```

19 #include "driver/gpio.h"
20 #include "driver/i2c.h"
21 /* Platform-specific: ESP32 uses esp_check.h, RX72N uses rx_check.h */
22 #include "rx_check.h"
23 #include "esp_err.h"
24 #include "esp_log.h"
25 #include "sdkconfig.h"
26
27 /* 5. Project includes */
28 #include "star_bus_config.h"
29 #include "star_bus_i2c.h"
30 #include "star_bus_types.h"
31 #include "star_error_interface.h"

```

5.73 Forward Declarations

Use forward declarations to minimize header dependencies.

```

1 /* In star_bus_config.h - forward declare instead of including */
2 /* Forward declaration */
3 struct star_bus_manager; /* Don't need full definition */
4
5 /* Function using forward-declared type */
6 rx_err_t star_bus_config_init(star_bus_config_t* config,
7 struct star_bus_manager* manager);
8
9 /* In star_bus_common_types.h */
10 /* Forward declarations for pointer-only usage */
11 typedef struct star_bus_config star_bus_config_t;
12 typedef struct star_bus_manager star_bus_manager_t;

```

5.74 Separating Types from Functions

For complex modules, split type definitions into dedicated headers.

```

1 /* File organization for star_bus module */
2 star_bus/include/
3 star_bus_manager.h      /* Main API (functions) */
4 star_bus_manager_types.h /* Types for manager */
5 star_bus_common_types.h /* Shared types */
6 star_bus_protocol_types.h /* Protocol-specific types */
7 star_bus_function_types.h /* Function pointer types */
8 star_bus_event_types.h  /* Event structures */
9 star_bus_types.h        /* Controller - includes all */
10
11 /* Controller header pattern */
12 /* star_bus_types.h */
13 #ifndef STAR_COMPONENT_BUS_TYPES_H
14 #define STAR_COMPONENT_BUS_TYPES_H
15
16 #ifdef __cplusplus
17 extern "C" {
18     #endif

```

```

19
20  /* Include all type headers */
21  #include "star_bus_common_types.h"
22  #include "star_bus_event_types.h"
23  #include "star_bus_function_types.h"
24  #include "star_bus_manager_types.h"
25  #include "star_bus_protocol_types.h"
26
27  #ifdef __cplusplus
28 }
29 #endif
30
31 #endif /* STAR_COMPONENT_BUS_TYPES_H */

```

5.75 Doxygen File Documentation

Include @file documentation for headers with significant content.

```

1  /**
2  * @file star_error_handler.h
3  * @brief Error handling with retry logic and exponential backoff
4  *
5  * This module provides a concrete implementation of the error interface
6  * with automatic retry management, exponential backoff delays, and
7  * optional reset function callbacks.
8  *
9  * Key Features:
10 * - Configurable max retries and backoff delays
11 * - Exponential backoff between retries
12 * - Optional custom reset function for recovery
13 * - Thread-safe with mutex protection
14 *
15 * Example Usage:
16 * @code
17 * error_handler_t error_handler;
18 * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
19 *
20 * if (operation_failed()) {
21 *     RECORD_ERROR(&error_handler, ESP_ERR_TIMEOUT, "Timeout");
22 * }
23 * @endcode
24 */

```

5.76 Complete Header Template

```

1  /* lib/star_module/include/star_module.h */
2
3  #ifndef STAR_COMPONENT_MODULE_H
4  #define STAR_COMPONENT_MODULE_H
5
6  #ifdef __cplusplus
7  extern "C" {

```

```

8  #endif
9
10 /* === Includes === */
11 #include <stdbool.h>
12 #include <stdint.h>
13
14 #include "esp_err.h"
15
16 /* === File Documentation === */
17 /**
18  * @file star_module.h
19  * @brief Brief description of this module
20  *
21  * Detailed description of what this module provides and how to use it.
22  */
23
24 /* === Forward Declarations === */
25 typedef struct star_module star_module_t;
26
27 /* === Macros === */
28 #define STAR_MODULE_DEFAULT_TIMEOUT_MS (1000)
29
30 /* === Type Definitions === */
31 typedef struct star_module {
32     bool        initialized;
33     uint32_t     timeout_ms;
34     void*        user_ctx;
35 } star_module_t;
36
37 /* === Function Declarations === */
38
39 /**
40  * @brief Initialize the module.
41  *
42  * @param[out] module Pointer to module structure to initialize.
43  * @param[in]  timeout Timeout in milliseconds.
44  * @return rx_err_t RX_OK on success.
45  */
46 esp_err_t star_module_init(star_module_t* module, uint32_t timeout);
47
48 /**
49  * @brief Deinitialize the module.
50  *
51  * @param[in,out] module Pointer to module structure to deinitialize.
52  * @return rx_err_t RX_OK on success.
53  */
54 esp_err_t star_module_deinit(star_module_t* module);
55
56 #ifdef __cplusplus
57 }
58 #endif
59
60 #endif /* STAR_COMPONENT_MODULE_H */

```

5.77 Summary

- Use include guards: `STAR_COMPONENT_FILENAME_H`
- Include C++ extern “C” blocks
- Order includes: RTOS, standard library, platform HAL, project
- Use forward declarations to minimize dependencies
- Split types into separate `_types.h` headers for complex modules
- Document files with `@file` Doxygen blocks
- Keep headers focused on declarations, not implementations

This section establishes standards for horizontal spacing in the STAR firmware codebase.

5.78 General Rules

- **Space After Keywords:** Add space after `if`, `for`, `while`, `switch`.
- **Binary Operators:** Add a single space around binary operators.
- **No Space After Function Names:** No space between function name and opening parenthesis.
- **Alignment:** Align related declarations and comments.

5.79 Space After Control Keywords

Add a space between keywords and opening parenthesis.

```

1  /* CORRECT - Space after keywords */
2  if (ret != RX_OK) {
3      return ret;
4  }
5
6  for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
7      process(curr);
8  }
9
10 while (retry_count > 0) {
11     attempt_operation();
12     retry_count--;
13 }
14
15 switch (config->type) {
16     case k_star_bus_type_i2c: {
17         /* ... */
18         break;
19     }
20 }
21
22 /* WRONG - No space after keywords */

```

```

23 if(ret != RX_OK) {           /* Missing space */
24 return ret;
25 }
26
27 for(int i = 0; i < 10; i++) { /* Missing space */
28 /* ... */
29 }

```

5.80 No Space After Function Names

Do not add space between function name and opening parenthesis.

```

1  /* CORRECT - No space after function name */
2  rx_err_t ret = star_bus_manager_init(&manager, "main", NULL, NULL);
3  RX_LOG_INFO(s_TAG, "Initialized bus manager");
4  free(buffer);
5
6  /* WRONG - Space after function name */
7  esp_err_t ret = star_bus_manager_init (&manager, "main", NULL, NULL);
8  RX_LOG_INFO (s_TAG, "Initialized bus manager");
9  free (buffer);

```

5.81 Binary Operators

Add a single space around binary operators.

```

1  /* CORRECT - Spaces around binary operators */
2  int result = a + b;
3  bool is_valid = (count > 0) && (size <= MAX_SIZE);
4  uint32_t mask = flags | FLAG_ENABLED;
5  int shifted = value << 2;
6  size_t remaining = total - processed;
7
8  /* Assignment operators */
9  count += 1;
10 flags |= FLAG_INITIALIZED;
11 value >>= 4;
12
13 /* Comparison operators */
14 if (ret != RX_OK) { /* ... */ }
15 if (count >= threshold) { /* ... */ }
16 if (ptr == NULL) { /* ... */ }
17
18 /* WRONG - Missing spaces */
19 int result = a+b;
20 bool is_valid = (count>0)&&(size<=MAX_SIZE);
21 if (ret!=ESP_OK) { /* ... */ }

```

5.82 Unary Operators

No space between unary operator and operand.

```

1  /* CORRECT - No space with unary operators */
2  count++;
3  --remaining;
4  bool is_null = !ptr;
5  int negative = -value;
6  uint32_t complement = ~mask;
7  size_t size = sizeof(star_bus_config_t);
8  int* ptr = &value;
9  int deref = *ptr;
10
11 /* WRONG - Space with unary operators */
12 count ++;
13 -- remaining;
14 bool is_null = ! ptr;

```

5.83 Pointer Declarations

Place asterisk with the type, not the variable name.

```

1  /* CORRECT - Asterisk with type */
2  star_bus_config_t* config;
3  const char* name;
4  void* user_ctx;
5
6  /* Function parameters */
7  rx_err_t star_bus_manager_init(star_bus_manager_t*    manager,
8  const char*    tag,
9  star_error_interface_t* error_iface,
10 star_pin_interface_t*  pin_iface);
11
12 /* Multiple pointers on same line (avoid when possible) */
13 int *a, *b; /* Both are pointers */
14
15 /* WRONG - Various incorrect styles */
16 star_bus_config_t *config; /* Asterisk with variable */
17 star_bus_config_t * config; /* Spaces around asterisk */

```

5.84 Comma and Semicolon Spacing

Space after comma and semicolon, not before.

```

1  /* CORRECT */
2  star_bus_manager_init(&manager, "main", &error_iface, &pin_iface);
3
4  for (int i = 0; i < count; i++) {
5      /* ... */
6  }
7
8  int array[] = {1, 2, 3, 4, 5};
9
10 /* WRONG */
11 star_bus_manager_init(&manager , "main" ,&error_iface ,&pin_iface);

```

```

12 for (int i = 0 ;i < count ;i++) { /* ... */ }
13 int array[] = {1 ,2 ,3 ,4 ,5};

```

5.85 Parentheses Spacing

No space after opening parenthesis or before closing parenthesis.

```

1  /* CORRECT */
2  if (ret != RX_OK) {
3      return ret;
4  }
5
6  result = calculate(a, b, c);
7
8  /* WRONG - Spaces inside parentheses */
9  if ( ret != RX_OK ) {
10     return ret;
11 }
12
13 result = calculate( a, b, c );

```

5.86 Parameter Alignment

Align parameters vertically in multi-line function declarations and calls.

```

1  /* CORRECT - Aligned parameters */
2  rx_err_t star_bus_manager_init(star_bus_manager_t*    manager,
3  const char*          tag,
4  star_error_interface_t* error_iface,
5  star_pin_interface_t*  pin_iface);
6
7  star_bus_config_t* star_bus_config_create_spi_device(
8  const char*          name,
9  spi_host_device_t    host,
10 gpio_num_t           copi_pin,
11 gpio_num_t           cipo_pin,
12 gpio_num_t           sclk_pin,
13 int32_t              dma_chan,
14 const spi_device_interface_config_t* dev_cfg);
15
16 /* Long function calls */
17 ESP_RETURN_ON_FALSE(manager && config,
18 ESP_ERR_INVALID_ARG,
19 s_TAG,
20 "Manager or config pointer is NULL");

```

5.87 Variable and Comment Alignment

Align related variables and their comments.

```

1  /* CORRECT - Aligned declarations and comments */
2  typedef struct error_handler {

```

```

3  uint32_t      error_count;          /**< Total errors recorded */
4  uint32_t      max_retries;          /**< Max retry attempts */
5  uint32_t      current_retry;        /**< Current retry counter */
6  uint32_t      base_retry_delay;     /**< Base delay (ms) */
7  uint32_t      max_retry_delay;      /**< Max delay (ms) */
8  uint32_t      current_retry_delay;  /**< Current delay with backoff
   */
9  rx_err_t      last_error;           /**< Last recorded error code */
10 bool          in_error_state;        /**< Whether in error state */
11 void*          reset_context;        /**< Context for reset function
   */
12 SemaphoreHandle_t mutex;             /**< Mutex for thread-safety */
13 } error_handler_t;
14
15 /* Aligned local variables */
16 i2c_port_t      port      = config->proto.i2c.port;
17 uint8_t         address   = config->proto.i2c.address;
18 rx_err_t        ret       = RX_OK;
19 i2c_cmd_handle_t cmd      = NULL;

```

5.88 Struct Initializer Alignment

Align designated initializers.

```

1  /* CORRECT - Aligned initializers */
2  star_bus_event_t event = {
3      .bus_type = k_star_bus_type_i2c,
4      .bus_name = config->name,
5      .i2c      = {
6          .port      = port,
7          .address   = address,
8          .is_write  = true,
9          .len       = len
10     }
11 };
12
13 spi_device_interface_config_t spi_cfg = {
14     .clock_speed_hz = 10 * 1000 * 1000,
15     .mode           = 0,
16     .spics_io_num   = GPIO_NUM_5,
17     .queue_size     = 7,
18     .pre_cb         = NULL,
19     .post_cb        = NULL,
20 };

```

5.89 Summary

Context	Rule
After if, for, while, switch	Space
After function name	No space
Around binary operators	Space
With unary operators	No space
After comma/semicolon	Space
Inside parentheses	No space
Pointer asterisk	With type
Multi-line parameters	Align vertically

This section establishes standards for include statement organization and ordering in the STAR firmware codebase.

5.90 General Rules

- **Use Angle Brackets for System/Library Headers:** `<stdio.h>`.
- **Use Quotation Marks for Project Headers:** `"my_project.h"`.
- **Minimal Includes in Header Files:** Only include what is necessary.
- **Group and Order Includes:** Follow consistent ordering.

5.91 Include Order

Includes should be ordered in these groups, separated by blank lines:

1. Primary header (in .c files)
2. FreeRTOS includes
3. Standard library includes
4. Platform HAL includes
5. Project includes

```

1  /* lib/star_bus/src/star_bus_i2c.c */
2
3  /* 1. Primary header - always first in .c file */
4  #include "star_bus_i2c.h"
5
6  /* 2. FreeRTOS includes */
7  #include "freertos/FreeRTOS.h"
8
9  /* 3. Standard library includes */
10 #include <inttypes.h>
11 #include <string.h>
12
13 /* 4. Platform HAL includes (ESP-IDF, Renesas, etc.) */
14 /* Platform-specific: ESP32 uses esp_check.h, RX72N uses rx_check.h */

```

```

15 #include "rx_check.h"
16 #include "esp_log.h"
17
18 /* 5. Project includes */
19 #include "star_bus_manager.h"
20 #include "star_bus_types.h"

```

5.92 Complete Example from Codebase

```

1  /* lib/star_bus/src/star_bus_manager.c */
2
3  /* Primary header */
4  #include "star_bus_manager.h"
5
6  /* FreeRTOS */
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/semphr.h"
9
10 /* Standard library */
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14
15 /* Platform HAL (ESP-IDF, Renesas, etc.) */
16 /* Platform-specific: ESP32 uses esp_check.h, RX72N uses rx_check.h */
17 #include "rx_check.h"
18 #include "esp_log.h"
19 #include "sdkconfig.h"
20
21 /* Project includes */
22 #include "star_bus_adc.h"
23 #include "star_bus_config.h"
24 #include "star_bus_gpio.h"
25 #include "star_bus_i2c.h"
26 #include "star_bus_spi.h"

```

5.93 FreeRTOS Include Order

FreeRTOS requires specific include ordering. Always include `FreeRTOS.h` first.

```

1  /* CORRECT - FreeRTOS.h must come first */
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/semphr.h"
4  #include "freertos/task.h"
5  #include "freertos/queue.h"
6  #include "freertos/event_groups.h"
7
8  /* WRONG - Other FreeRTOS headers before FreeRTOS.h */
9  #include "freertos/semphr.h" /* Error: FreeRTOS.h not included first */
10 #include "freertos/FreeRTOS.h"

```

5.94 System Includes

Use angle brackets for system and standard library headers.

```
1  /* Standard C library */
2  #include <inttypes.h>
3  #include <stdbool.h>
4  #include <stddef.h>
5  #include <stdint.h>
6  #include <stdio.h>
7  #include <stdlib.h>
8  #include <string.h>
```

5.95 Platform HAL Includes

Platform HAL headers (ESP-IDF, Renesas, etc.) use quotation marks and are grouped separately.

```
1  /* Platform-specific driver includes (example: ESP32) */
2  #include "driver/gpio.h"
3  #include "driver/i2c.h"
4  #include "driver/spi_master.h"
5  #include "driver/uart.h"
6
7  /* Platform system includes */
8  /* Platform-specific: ESP32 uses esp_check.h, RX72N uses rx_check.h */
9  #include "rx_check.h"
10 #include "rx_err.h"      /* Platform error types */
11 #include "rx_log.h"      /* Platform logging */
12 #include "platform_system.h"
13
14 /* Platform peripheral includes (if needed) */
15 #include "platform_adc.h"
16 #include "esp_adc/adc_oneshot.h"
17
18 /* Configuration */
19 #include "sdkconfig.h"
```

5.96 Project Includes

Use quotation marks for project headers. Alphabetize within each group.

```
1  /* Project includes - alphabetized */
2  #include "star_bus_adc.h"
3  #include "star_bus_config.h"
4  #include "star_bus_gpio.h"
5  #include "star_bus_i2c.h"
6  #include "star_bus_manager.h"
7  #include "star_bus_spi.h"
8  #include "star_bus_types.h"
9  #include "star_error_interface.h"
10 #include "star_pin_interface.h"
```

5.97 Header File Includes

Header files should minimize includes and use forward declarations when possible.

```

1  /* star_bus_manager.h - Header file includes */
2  #ifndef STAR_COMPONENT_BUS_MANAGER_H
3  #define STAR_COMPONENT_BUS_MANAGER_H
4
5  #ifdef __cplusplus
6  extern "C" {
7      #endif
8
9      /* Only include what's needed for declarations */
10     #include "esp_err.h"
11     #include "star_bus_manager_types.h"
12
13     /* Use forward declarations for pointer-only usage */
14     struct star_bus_config; /* Don't include star_bus_config.h */
15
16     /* Function using forward-declared type */
17     rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
18     struct star_bus_config* config);
19
20     #ifdef __cplusplus
21     }
22     #endif
23
24 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

5.98 Conditional Includes

Use #ifdef for platform-specific or optional includes.

```

1  /* Platform-specific includes */
2  #ifdef CONFIG_IDF_TARGET_ESP32S3
3  #include "esp32s3/rom/gpio.h"
4  #else
5  #include "esp32/rom/gpio.h"
6  #endif
7
8  /* Optional feature includes */
9  #ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING
10 #include "esp_debug_helpers.h"
11 #endif

```

5.99 Include Comments

Add comments for non-obvious includes explaining why they are needed.

```

1  #include "star_bus_manager.h"
2
3  #include "freertos/FreeRTOS.h"
4
5  #include <inttypes.h>

```

```

6 #include <string.h>
7
8 /* Platform-specific: ESP32 uses esp_check.h, RX72N uses rx_check.h */
9 #include "rx_check.h"
10 #include "esp_log.h"
11 #include "star_bus_manager.h" /* Needed for star_bus_manager_find_bus */
12 #include "star_bus_types.h" /* Needed for star_bus_config_t and union */

```

5.100 Avoiding Circular Dependencies

Structure your includes to avoid circular dependencies.

```

1 /* Problem: Circular dependency */
2 /* star_bus_manager.h includes star_bus_config.h */
3 /* star_bus_config.h includes star_bus_manager.h */
4
5 /* Solution: Use forward declarations */
6 /* star_bus_config.h */
7 struct star_bus_manager; /* Forward declare, don't include */
8
9 rx_err_t star_bus_config_init(star_bus_config_t* config,
10 struct star_bus_manager* manager);
11
12 /* star_bus_config.c - Include full header here */
13 #include "star_bus_config.h"
14 #include "star_bus_manager.h" /* Full definition needed in .c */

```

5.101 Summary

Order	Category	Style
1	Primary header (.c files)	"..."
2	FreeRTOS	"freertos/..."
3	Standard library	<...>
4	Platform HAL	"..."
5	Project	"..."

- Always include `FreeRTOS.h` before other FreeRTOS headers
- Separate groups with blank lines
- Alphabetize within each group
- Minimize includes in headers; use forward declarations
- Add comments for non-obvious include dependencies

This project follows the rule of **2 spaces** for indentation, without using tabs.

Unix-style line endings ('LF', represented as '\n') are mandatory.

Logging is handled using platform-specific logging macros (e.g., `RX_LOG*forSTAR, ESP_LOG*forESP - IDFcompatibilitylayer`).

`RX_LOG_ERROR / ESP_LOGE` - Error

RX_LOG_WARN / ESP_LOGW - Warning

RX_LOG_INFO / ESP_LOGI - Info

RX_LOG_DEBUG / ESP_LOGD - Debug

RX_LOG_VERBOSE / ESP_LOGV - Verbose

This section establishes comprehensive naming standards for all identifiers in the STAR firmware codebase. Consistent naming improves readability, maintainability, and reduces cognitive load when navigating the codebase.

5.102 General Rules

- **Language:** All identifiers must be in English.
- **Clarity:** Names should be descriptive and self-documenting.
- **Abbreviations:** Avoid abbreviations unless they are widely understood (e.g., GPIO, I2C, SPI).

5.103 Functions and Variables

Functions and variables use **snake_case**—all lowercase with words separated by underscores.

```

1  /* CORRECT - snake_case for functions and variables */
2  esp_err_t star_bus_manager_init(star_bus_manager_t* manager, const
    char* tag);
3  uint32_t error_count;
4  size_t bytes_written;
5  bool is_initialized;
6
7  /* WRONG - other naming styles */
8  rx_err_t StarBusManagerInit(); /* PascalCase */
9  rx_err_t starBusManagerInit(); /* camelCase */
10 uint32_t ErrorCount; /* PascalCase */

```

5.104 Module Prefixes

All public functions must use the module prefix pattern: **star_[module]_[function]**.

```

1  /* From star_bus_manager.h */
2  rx_err_t star_bus_manager_init(...);
3  rx_err_t star_bus_manager_add_bus(...);
4  rx_err_t star_bus_manager_remove_bus(...);
5  rx_err_t star_bus_manager_deinit(...);
6
7  /* From star_bus_config.h */
8  star_bus_config_t* star_bus_config_create_i2c(...);
9  star_bus_config_t* star_bus_config_create_spi_device(...);
10 rx_err_t star_bus_config_destroy(...);
11

```

```

12 /* From star_bus_i2c.h */
13 rx_err_t star_bus_i2c_write(...);
14 rx_err_t star_bus_i2c_read(...);
15 void      star_bus_i2c_init_default_ops(...);
16
17 /* From star_error_handler.h */
18 rx_err_t error_handler_init(...);
19 rx_err_t error_handler_record_error(...);
20 bool     error_handler_can_retry(...);

```

5.105 Constants and Macros

Constants and macros use `SCREAMING_SNAKE_CASE`—all uppercase with words separated by underscores.

```

1 /* CORRECT - SCREAMING_SNAKE_CASE for macros and constants */
2 #define STAR_BUS_NAME_MAX_LEN          (32)
3 #define CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS (1000)
4 #define I2C_MASTER_WRITE                0
5 #define I2C_MASTER_READ                 1
6 #define GPIO_NUM_MAX                   48
7
8 /* Convenience macro with proper naming */
9 #define RECORD_ERROR(handler, code, desc) \
10 do { \
11     error_handler_record_error((handler), (code), (desc), \
12     __FILE__, __LINE__, __func__); \
13 } while (0)
14
15 #define STAR_IFACE_RECORD_ERROR(iface, err, msg) \
16 ((iface) && (iface)->record_error \
17 ? (iface)->record_error((iface)->ctx, (err), (msg), \
18     __FILE__, __LINE__, __func__) \
19 : (err))
20
21 /* WRONG */
22 #define star_bus_name_max_len  32    /* lowercase */
23 #define StarBusNameMaxLen      32    /* PascalCase */

```

5.106 Type Names

All custom types must use `snake_case` with the `_t` suffix.

```

1 /* CORRECT - snake_case_t for type names */
2 typedef struct star_bus_config star_bus_config_t;
3 typedef struct star_bus_manager star_bus_manager_t;
4 typedef struct error_handler error_handler_t;
5 typedef enum star_bus_type star_bus_type_t;
6
7 /* Callback function types also use _t suffix */
8 typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus, void*
    user_ctx);

```

```

9 typedef rx_err_t (*pin_registration_function_t)(gpio_num_t pin_num,
10 const char* bus_name,
11 const char* pin_usage,
12 void* user_ctx);
13
14 /* WRONG */
15 typedef struct StarBusConfig StarBusConfig; /* PascalCase */
16 typedef struct star_bus_config star_bus_config; /* missing _t suffix */

```

5.107 Enum Members

Enum members use the `k_` prefix followed by `snake_case`.

```

1 /* CORRECT - k_prefix with snake_case */
2 typedef enum {
3     k_star_bus_type_none, /**< No bus type specified */
4     k_star_bus_type_i2c, /**< I2C bus */
5     k_star_bus_type_spi, /**< SPI bus */
6     k_star_bus_type_gpio, /**< GPIO bus */
7     k_star_bus_type_adc, /**< ADC bus */
8     k_star_bus_type_count, /**< Number of bus types (sentinel) */
9 } star_bus_type_t;
10
11 /* Usage in switch statements */
12 switch (config->type) {
13     case k_star_bus_type_i2c: {
14         /* Handle I2C */
15         break;
16     }
17     case k_star_bus_type_spi: {
18         /* Handle SPI */
19         break;
20     }
21     default: {
22         RX_LOG_WARN(s_TAG, "Unknown bus type: %d", config->type);
23         break;
24     }
25 }
26
27 /* WRONG */
28 typedef enum {
29     STAR_BUS_TYPE_I2C, /* SCREAMING_SNAKE_CASE without k_ */
30     StarBusTypeI2c, /* PascalCase */
31     starBusTypeI2c, /* camelCase */
32 } bad_enum_t;

```

5.108 Static Variables (File-Scoped)

Static file-scoped variables use the `s_` prefix followed by `snake_case`.

```

1  /* CORRECT - s_ prefix for static file-scoped variables */
2  static const char* s_TAG = "STAR_BUS_MANAGER";
3  static const uint32_t s_i2c_timeout_ms = 1000;
4
5  /* From star_bus_i2c.c */
6  static const char* s_TAG = "STAR_BUS_I2C";
7  static const uint32_t s_i2c_timeout_ms = 1000;
8
9  /* From star_bus_config.c */
10 static const char* s_TAG = "STAR_BUS_CONFIG";
11
12 /* WRONG - missing s_ prefix */
13 static const char* TAG = "MODULE";
14 static uint32_t timeout_ms = 1000;

```

5.109 Global Variables

Global variables use the `g_` prefix followed by `snake_case`. However, global variables are **strongly discouraged**. Prefer dependency injection or static file-scoped variables.

```

1  /* CORRECT (but discouraged) - g_ prefix for globals */
2  g_system_state_t g_system_state;
3  uint32_t g_error_count;
4
5  /* PREFERRED - use dependency injection instead */
6  typedef struct {
7      star_error_interface_t* error_iface; /* Injected */
8      star_pin_interface_t* pin_iface; /* Injected */
9  } star_bus_manager_t;
10
11 /* Initialize with injected dependencies */
12 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
13 const char* tag,
14 star_error_interface_t* error_iface,
15 star_pin_interface_t* pin_iface);

```

5.110 Private Functions

5.110.1 File-Scoped Static Functions (`internal_` prefix)

Static functions that are internal to a single source file use the `internal_` prefix.

```

1  /* From star_bus_manager.c - internal helper functions */
2  static rx_err_t internal_register_bus_pin(star_pin_interface_t*
      pin_iface,
3  gpio_num_t pin,
4  const char* bus_name,
5  const char* usage,
6  bool can_be_shared)
7  {
8      if (pin < 0 || pin >= GPIO_NUM_MAX) {

```

```

9     return RX_OK;  /* Not an error, pin is not used */
10 }
11 /* ... implementation ... */
12 }
13
14 static rx_err_t internal_unregister_bus_pin(star_pin_interface_t*
15     pin_iface,
16     gpio_num_t          pin,
17     const char*         bus_name,
18     const char*         usage)
19 {
20     /* ... implementation ... */
21 }
22
23 static rx_err_t internal_register_config_pins(star_pin_interface_t*
24     pin_iface,
25     const star_bus_config_t* config)
26 {
27     /* ... implementation ... */
28 }
29
30 /* From star_bus_i2c.c */
31 static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
32     config,
33     const uint8_t*      data,
34     size_t              len,
35     uint8_t             reg_addr,
36     size_t*             bytes_written);
37
38 static rx_err_t internal_i2c_execute_with_retry(const
39     star_bus_config_t* config,
40     i2c_port_t         port,
41     i2c_cmd_handle_t   cmd,
42     const char*         operation_name);

```

5.110.2 Exposed Private Functions (priv_ prefix)

Functions that must be exposed across files within a module (but are not part of the public API) use the `priv_` prefix.

```

1 /* In private header (e.g., star_bus_internal.h) */
2 /* These functions are used by multiple .c files in the same module
3 but should not be called by external code */
4
5 rx_err_t priv_star_bus_validate_config(const star_bus_config_t*
6     config);
7 rx_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);
8 void     priv_star_bus_release_mutex(star_bus_manager_t* manager);
9
10 /* Used in implementation files within the module */
11 rx_err_t star_bus_config_init(star_bus_config_t* config,
12     struct star_bus_manager* manager)
13 {

```

```

13  /* Validate using private helper */
14  esp_err_t ret = priv_star_bus_validate_config(config);
15  if (ret != RX_OK) {
16      return ret;
17  }
18  /* ... rest of implementation ... */
19  }

```

5.111 Summary Table

Identifier Type	Convention	Example
Functions	snake_case	star_bus_manager_init
Variables	snake_case	bytes_written
Constants/Macros	SCREAMING_SNAKE_CASE	STAR_BUS_NAME_MAX_LEN
Type names	snake_case_t	star_bus_config_t
Enum members	k_snake_case	k_star_bus_type_i2c
Static variables	s_snake_case	s_TAG
Global variables	g_snake_case	g_system_state
Internal functions	internal_snake_case	internal_register_bus_pin
Private functions	priv_snake_case	priv_star_bus_validate_config
Module prefix	star_[module]_	star_bus_, star_error_

This section describes the patterns and conventions for defining custom types in the STAR firmware codebase. Consistent type definition patterns improve code readability and enable better tooling support.

5.112 Typedef Struct Pattern

All structures should be defined using the typedef pattern with the `_t` suffix.

5.112.1 Standard Pattern

```

1  /* Forward declaration (in header for dependencies) */
2  typedef struct star_bus_config star_bus_config_t;
3  typedef struct star_bus_manager star_bus_manager_t;
4
5  /* Full definition (in header or implementation) */
6  typedef struct star_bus_config {
7      const char*    name;          /**< Unique name for this bus/device
8          instance */
9      star_bus_type_t type;          /**< Type of bus (I2C, SPI, etc.) */
10     bool            initialized; /**< Whether this bus has been
11         initialized */
12     void*            handle;        /**< Driver/device handle */
13     void*            user_ctx;      /**< User context pointer for
14         callbacks */
15
16     /* Protocol-specific configuration (see union section) */

```

```

14 union {
15     star_i2c_bus_config_t i2c;
16     star_spi_bus_config_t spi;
17     star_gpio_bus_config_t gpio;
18     star_adc_bus_config_t adc;
19 } proto;
20
21 struct star_bus_config* next; /**< Linked list pointer */
22 } star_bus_config_t;

```

5.112.2 Error Handler Example

```

1 /* From star_error_handler.h */
2 typedef struct error_handler {
3     uint32_t      error_count;          /**< Total number of errors
4         recorded */
5     uint32_t      max_retries;          /**< Maximum retry attempts
6         allowed */
7     uint32_t      current_retry;        /**< Current retry attempt
8         counter */
9     uint32_t      base_retry_delay;     /**< Base delay between
10        retries (ms) */
11     uint32_t      max_retry_delay;      /**< Maximum retry delay
12        (ms) */
13     uint32_t      current_retry_delay;  /**< Current delay with
14        backoff */
15     rx_err_t      last_error;           /**< Last recorded error code
16        */
17     bool          in_error_state;       /**< Whether in error state
18        */
19     void*         reset_context;        /**< Context for reset
20        function */
21     SemaphoreHandle_t mutex;            /**< Mutex for thread-safety
22        */
23
24     rx_err_t (*reset_fn)(void* context); /**< Optional reset function
25        */
26 } error_handler_t;

```

5.113 Union Usage for Protocol Configurations

Unions are used to store protocol-specific configurations efficiently. This pattern allows a single configuration structure to support multiple bus types.

```

1 /* From star_bus_manager_types.h */
2 typedef struct star_bus_config {
3     const char*    name;
4     star_bus_type_t type; /* Discriminator for the union */
5     bool          initialized;
6     void*         handle;
7     void*         user_ctx;

```

```

8
9  /* Union holds protocol-specific configuration */
10 union {
11     star_i2c_bus_config_t i2c; /**< I2C-specific configuration */
12     star_spi_bus_config_t spi; /**< SPI-specific configuration */
13     star_gpio_bus_config_t gpio; /**< GPIO-specific configuration */
14     star_adc_bus_config_t adc; /**< ADC-specific configuration */
15 } proto;
16
17 struct star_bus_config* next;
18 } star_bus_config_t;
19
20 /* Accessing union members based on type discriminator */
21 switch (config->type) {
22     case k_star_bus_type_i2c: {
23         i2c_port_t port = config->proto.i2c.port;
24         uint8_t address = config->proto.i2c.address;
25         /* ... */
26         break;
27     }
28     case k_star_bus_type_spi: {
29         spi_host_device_t host = config->proto.spi.host;
30         gpio_num_t cop_i_pin = config->proto.spi.copi_io_num;
31         /* ... */
32         break;
33     }
34     /* ... other cases ... */
35 }

```

5.113.1 Protocol-Specific Structures

```

1 /* I2C bus configuration (from star_bus_protocol_types.h) */
2 typedef struct star_i2c_bus_config {
3     i2c_port_t port; /**< I2C port number */
4     i2c_config_t config; /**< Platform I2C configuration */
5     uint8_t address; /**< 7-bit I2C device address */
6     star_i2c_callbacks_t callbacks; /**< User callbacks */
7     star_i2c_ops_t ops; /**< Operation function pointers */
8 } star_i2c_bus_config_t;
9
10 /* SPI bus configuration */
11 typedef struct star_spi_bus_config {
12     spi_host_device_t host; /**< SPI host number */
13     bool is_peripheral; /**< True if SPI peripheral mode */
14     spi_bus_config_t bus_cfg; /**< Platform SPI bus config */
15     spi_device_interface_config_t dev_cfg; /**< Device config */
16     star_spi_callbacks_t callbacks;
17     star_spi_ops_t ops;
18
19     /* Inclusive terminology pin names (see Terminology section) */
20     gpio_num_t cop_i_io_num; /**< Controller Out, Peripheral In */
21     gpio_num_t cipo_io_num; /**< Controller In, Peripheral Out */

```

```

22     gpio_num_t sclk_io_num;  /**< Serial Clock */
23     gpio_num_t cs_io_num;    /**< Chip Select */
24     /* ... */
25 } star_spi_bus_config_t;

```

5.114 Callback Function Types

Callback function pointer types follow a consistent pattern with the `_t` suffix.

```

1  /* From star_bus_common_types.h - Pin registration callback */
2  typedef rx_err_t (*pin_registration_function_t)(gpio_num_t pin_num,
3  const char* bus_name,
4  const char* pin_usage,
5  void* user_ctx);
6
7  /* From star_bus_manager.h - Bus access callback */
8  typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
9  void* user_ctx);
10
11 /* From star_bus_function_types.h - I2C operation callbacks */
12 typedef rx_err_t (*star_i2c_write_fn_t)(const star_bus_config_t*
13     config,
14     const uint8_t* data,
15     size_t len,
16     uint8_t reg_addr,
17     size_t* bytes_written);
18
19 typedef rx_err_t (*star_i2c_read_fn_t)(const star_bus_config_t*
20     config,
21     uint8_t* data,
22     size_t len,
23     uint8_t reg_addr,
24     size_t* bytes_read);
25
26 /* Event callbacks */
27 typedef void (*star_i2c_transfer_complete_cb_t)(const
28     star_bus_event_t* event,
29     void* user_ctx);
30
31 typedef void (*star_i2c_error_cb_t)(const star_bus_event_t* event,
32     rx_err_t error,
33     void* user_ctx);
34
35 /* Grouping related callbacks in a struct */
36 typedef struct star_i2c_callbacks {
37     star_i2c_transfer_complete_cb_t on_transfer_complete;
38     star_i2c_error_cb_t on_error;
39 } star_i2c_callbacks_t;

```

5.115 Interface Patterns (Dependency Injection)

Interfaces use structs with function pointers and an opaque context pointer. This pattern enables dependency injection and testability.

```

1  /* From star_error_interface.h */
2  typedef struct star_error_interface {
3      /**
4       * @brief Record an error
5       * @param ctx Implementation context
6       * @param error Error code
7       * @param message Error message
8       * @param file Source file
9       * @param line Line number
10      * @param func Function name
11      * @return ESP_OK on success
12      */
13      rx_err_t (*record_error)(void*      ctx,
14      rx_err_t      error,
15      const char* message,
16      const char* file,
17      int         line,
18      const char* func);
19
20      /**
21       * @brief Check if retry is possible
22       * @param ctx Implementation context
23       * @return true if retry is allowed
24       */
25      bool (*can_retry)(void* ctx);
26
27      /**
28       * @brief Reset error state
29       * @param ctx Implementation context
30       * @return ESP_OK on success
31       */
32      rx_err_t (*reset_state)(void* ctx);
33
34      /**
35       * @brief Implementation context (opaque pointer)
36       */
37      void* ctx;
38  } star_error_interface_t;
39
40  /* From star_pin_interface.h */
41  typedef struct star_pin_interface {
42      rx_err_t (*register_pin)(void* ctx, int pin,
43      const char* description, bool shared);
44      rx_err_t (*unregister_pin)(void* ctx, int pin,
45      const char* description);
46      rx_err_t (*validate)(void* ctx);
47      void* ctx;
48  } star_pin_interface_t;

```

5.115.1 Interface Usage Pattern

```

1  /* Creating an interface from concrete implementation */
2  error_handler_t error_handler;
3  error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
4
5  star_error_interface_t error_iface;
6  error_handler_get_interface(&error_iface, &error_handler);
7
8  /* Injecting interfaces into components */
9  star_bus_manager_t bus_manager;
10 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
11
12 /* Using interface through function pointers */
13 if (error_iface.can_retry(error_iface.ctx)) {
14     /* Retry operation */
15 }
16
17 /* Or using the convenience macro */
18 STAR_IFACE_RECORD_ERROR(&error_iface, ESP_ERR_TIMEOUT, "Operation
    timed out");

```

5.116 Opaque Handle Patterns

For complex objects that should not be directly manipulated, use opaque handles.

```

1  /* Forward declaration creates an opaque type */
2  typedef struct star_bus_manager star_bus_manager_t;
3
4  /* Public API only uses pointers to the opaque type */
5  esp_err_t star_bus_manager_init(star_bus_manager_t* manager, ...);
6  rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager, ...);
7
8  /* Full definition is only in the .c file or private header */
9  /* Users cannot access internal fields directly */
10
11 /* Example usage - users allocate storage but don't access internals */
12 /*
13 star_bus_manager_t bus_manager; /* Stack allocation */
14 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
15 */
16 /* Or heap allocation */
17 star_bus_manager_t* manager_ptr = malloc(sizeof(star_bus_manager_t));
18 star_bus_manager_init(manager_ptr, "main", &error_iface, &pin_iface);

```

5.117 Operations Structure Pattern

Group related function pointers in an operations structure for pluggable implementations.

```

1  /* From star_bus_protocol_types.h */
2  typedef struct star_i2c_ops {

```

```

3   star_i2c_write_fn_t          write;          /**< Write with register
4   */
5   star_i2c_read_fn_t          read;           /**< Read with register
6   */
7   star_i2c_write_command_fn_t write_command;  /**< Write command byte
8   */
9   star_i2c_read_raw_fn_t       read_raw;       /**< Read raw bytes */
10  } star_i2c_ops_t;
11
12  typedef struct star_spi_ops {
13      star_spi_transmit_fn_t transmit; /**< Transmit only */
14      star_spi_receive_fn_t receive;  /**< Receive only */
15      star_spi_transfer_fn_t transfer; /**< Full-duplex transfer */
16  } star_spi_ops_t;
17
18  typedef struct star_gpio_ops {
19      rx_err_t (*write_digital)(const struct star_bus_config* config,
20      uint8_t pin_index, uint8_t value);
21      rx_err_t (*read_digital)(const struct star_bus_config* config,
22      uint8_t pin_index, uint8_t* value);
23  } star_gpio_ops_t;
24
25  /* Initializing default operations */
26  void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
27  {
28      if (ops == NULL) {
29          RX_LOG_ERROR(s_TAG, "Cannot initialize NULL ops pointer");
30          return;
31      }
32      ops->write          = internal_star_bus_i2c_write;
33      ops->read           = internal_star_bus_i2c_read;
34      ops->write_command  = internal_star_bus_i2c_write_command;
35      ops->read_raw       = internal_star_bus_i2c_read_raw;
36  }

```

5.118 Manager Structure Pattern

Manager structures coordinate multiple resources with thread-safe access.

```

1  /* From star_bus_manager_types.h */
2  typedef struct star_bus_manager {
3      star_bus_config_t* buses;          /**< Linked list of bus
4      configs */
5      SemaphoreHandle_t mutex;          /**< Mutex for thread-safety */
6      const char* tag;                  /**< Logging tag */
7      star_error_interface_t* error_iface; /**< Injected error handler */
8      star_pin_interface_t* pin_iface;  /**< Injected pin validator */
9
10     /* Resource tracking */
11     bool spi_host_initialized[SPI_HOST_MAX];
12     int spi_device_count[SPI_HOST_MAX];
13 } star_bus_manager_t;

```

5.119 Best Practices

- **Always use `_t` suffix:** Makes types easily distinguishable from variables.
- **Use forward declarations:** Minimize header dependencies and compilation time.
- **Document all fields:** Use Doxygen comments for struct members.
- **Keep unions discriminated:** Always pair unions with a type field.
- **Prefer static const over `#define`:** For typed constants within structures.
- **Use opaque types for encapsulation:** Hide implementation details from users.
- **Group related function pointers:** Use ops structs for pluggable implementations.

This section describes the standard file and directory structure for the STAR firmware code-base. Consistent organization improves navigability and maintainability.

5.120 Library Directory Structure

Each library follows a standard component structure (PlatformIO/ESP-IDF on ESP32, CMake on RX72N):

```
lib/star_component/  
  CMakeLists.txt      # CMake build configuration  
  library.json         # PlatformIO library metadata  
  include/            # Public headers (API)  
    star_component.h  
    star_component_types.h  
  src/                # Implementation files  
    star_component.c
```

5.120.1 Example: `star_bus` Library

```
lib/star_bus/  
  CMakeLists.txt  
  library.json  
  include/  
    star_bus_adc.h  
    star_bus_common_types.h  
    star_bus_config.h  
    star_bus_event_types.h  
    star_bus_function_types.h  
    star_bus_gpio.h  
    star_bus_i2c.h  
    star_bus_manager.h  
    star_bus_manager_types.h  
    star_bus_protocol_types.h
```

```
    star_bus_spi.h
    star_bus_types.h
src/
    star_bus_adc.c
    star_bus_config.c
    star_bus_gpio.c
    star_bus_i2c.c
    star_bus_manager.c
    star_bus_spi.c
    star_bus_types.c
```

5.121 CMakeLists.txt Structure

```
1 # lib/star_bus/CMakeLists.txt
2
3 idf_component_register(
4     SRCS
5     "src/star_bus_adc.c"
6     "src/star_bus_config.c"
7     "src/star_bus_gpio.c"
8     "src/star_bus_i2c.c"
9     "src/star_bus_manager.c"
10    "src/star_bus_spi.c"
11    "src/star_bus_types.c"
12    INCLUDE_DIRS
13    "include"
14    REQUIRES
15    driver
16    esp_adc
17    freertos
18    star_core
19 )
```

5.122 Header File Organization

Header files must follow a consistent internal structure.

5.122.1 Header File Template

```
1 /* lib/star_bus/include/star_bus_manager.h */
2
3 #ifndef STAR_COMPONENT_BUS_MANAGER_H
4 #define STAR_COMPONENT_BUS_MANAGER_H
5
6 #ifdef __cplusplus
7 extern "C" {
8     #endif
9
10    /* === Includes === */
```

```

11 #include "esp_err.h"
12 #include "star_bus_manager_types.h"
13
14 /**
15  * @file star_bus_manager.h
16  * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
17  *        protocols
18  *
19  * The bus manager provides a central registry for managing multiple
20  * bus configurations. It supports dependency injection of error
21  * handlers
22  * and pin validators for loose coupling.
23  *
24  * Key Features:
25  * - Thread-safe bus management with mutex protection
26  * - Support for multiple bus types (I2C, SPI, GPIO, etc.)
27  * - Automatic pin registration and conflict detection
28  * - Safe bus access via callback pattern
29  *
30  * Example Usage:
31  * @code
32  * // Initialize bus manager with dependency injection
33  * star_bus_manager_t bus_manager;
34  * star_bus_manager_init(&bus_manager, "main", &error_iface,
35  *                      &pin_iface);
36  *
37  * // Create and add an I2C bus
38  * star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
39  *     "sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22,
40  *     400000);
41  * star_bus_manager_add_bus(&bus_manager, i2c_bus);
42  * @endcode
43  */
44
45 /* == Forward Declarations == */
46 /* (if needed beyond what's in included headers) */
47
48 /* == Public Functions == */
49
50 /**
51  * @brief Initialize a bus manager instance.
52  *
53  * @param[out] manager Pointer to bus manager to initialize.
54  *                   Must not be NULL.
55  * @param[in] tag Optional tag string for logging. NULL
56  *                uses default.
57  * @param[in] error_iface Optional error handler interface (can be
58  *                        NULL).
59  * @param[in] pin_iface Optional pin validator interface (can be
60  *                      NULL).
61  * @return rx_err_t RX_OK on success, RX_ERR_INVALID_ARG if manager
62  *         is NULL,
63  *         ESP_ERR_NO_MEM if mutex creation fails.
64  */

```

```

56 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
57     const char* tag,
58     star_error_interface_t* error_iface,
59     star_pin_interface_t* pin_iface);
60
61     /* ... more function declarations ... */
62
63     #ifdef __cplusplus
64 }
65 #endif
66
67 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

5.122.2 Header Section Order

1. **File path comment:** Identifies the file location
2. **Include guard:** `#ifndef STAR_COMPONENT_FILENAME_H`
3. **C++ extern “C” opening:** For C++ compatibility
4. **Includes:** System, then platform HAL, then project headers
5. **File documentation:** @file Doxygen block with overview
6. **Forward declarations:** If needed
7. **Constants and macros:** Public defines
8. **Type definitions:** Enums, structs, typedefs
9. **Function declarations:** Grouped by category
10. **C++ extern “C” closing:** Matches opening
11. **Include guard closing:** `#endif /* ... */`

5.123 Include Guard Naming Convention

Include guards follow the pattern: `STAR_COMPONENT_FILENAME_H`

```

1  /* star_bus_manager.h */
2  #ifndef STAR_COMPONENT_BUS_MANAGER_H
3  #define STAR_COMPONENT_BUS_MANAGER_H
4  /* ... */
5  #endif /* STAR_COMPONENT_BUS_MANAGER_H */
6
7  /* star_bus_config.h */
8  #ifndef STAR_COMPONENT_BUS_CONFIG_H
9  #define STAR_COMPONENT_BUS_CONFIG_H
10 /* ... */
11 #endif /* STAR_COMPONENT_BUS_CONFIG_H */
12

```

```

13  /* star_error_interface.h */
14  #ifndef STAR_ERROR_INTERFACE_H
15  #define STAR_ERROR_INTERFACE_H
16  /* ... */
17  #endif /* STAR_ERROR_INTERFACE_H */
18
19  /* star_pin_interface.h */
20  #ifndef STAR_PIN_INTERFACE_H
21  #define STAR_PIN_INTERFACE_H
22  /* ... */
23  #endif /* STAR_PIN_INTERFACE_H */

```

5.124 Source File Organization

Source files follow a consistent internal structure.

5.124.1 Source File Template

```

1  /* lib/star_bus/src/star_bus_manager.c */
2
3  /* === Primary Header === */
4  #include "star_bus_manager.h"
5
6  /* === System Includes === */
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/semphr.h"
9
10 #include <stdio.h>
11 #include <stdlib.h>
12 #include <string.h>
13
14 /* === Platform HAL Includes === */
15 /* Platform-specific: ESP32 uses esp_check.h, RX72N uses rx_check.h */
16 #include "rx_check.h"
17 #include "esp_log.h"
18 #include "sdkconfig.h"
19
20 /* === Project Includes === */
21 #include "star_bus_adc.h"
22 #include "star_bus_config.h"
23 #include "star_bus_gpio.h"
24
25 /* === Constants === */
26
27 static const char* s_TAG = "STAR_BUS_MANAGER";
28
29 /* === Private Function Prototypes === */
30
31 static rx_err_t internal_register_bus_pin(star_pin_interface_t*
    pin_iface,
32 gpio_num_t          pin,

```

```

33 const char*          bus_name ,
34 const char*          usage ,
35 bool                 can_be_shared);
36
37 static rx_err_t internal_unregister_bus_pin(star_pin_interface_t*
    pin_iface ,
38 gpio_num_t           pin ,
39 const char*          bus_name ,
40 const char*          usage);
41
42 /* === Public Functions === */
43
44 rx_err_t star_bus_manager_init(star_bus_manager_t* manager ,
45 const char*          tag ,
46 star_error_interface_t* error_iface ,
47 star_pin_interface_t* pin_iface)
48 {
49     RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
50 "Manager pointer is NULL");
51
52     manager->buses          = NULL;
53     manager->error_iface = error_iface;
54     manager->pin_iface     = pin_iface;
55
56     /* ... rest of implementation ... */
57
58     return RX_OK;
59 }
60
61 /* ... more public functions ... */
62
63 /* === Private Functions === */
64
65 static rx_err_t internal_register_bus_pin(star_pin_interface_t*
    pin_iface ,
66 gpio_num_t           pin ,
67 const char*          bus_name ,
68 const char*          usage ,
69 bool                 can_be_shared)
70 {
71     if (pin < 0 || pin >= GPIO_NUM_MAX) {
72         return RX_OK; /* Not an error, pin is not used */
73     }
74     /* ... implementation ... */
75 }

```

5.124.2 Source Section Order

1. **File path comment:** Identifies the file location
2. **Primary header:** The corresponding .h file for this .c file
3. **System includes:** FreeRTOS, standard library (<...>)

4. **Platform HAL includes:** Platform-specific components
5. **Project includes:** Other project headers ("`...`")
6. **Constants:** static const variables, including `s_TAG`
7. **Private function prototypes:** Forward declarations of static functions
8. **Public functions:** Implementation of API functions
9. **Private functions:** Implementation of static helper functions

5.125 Types Header Separation

For complex modules, separate type definitions into dedicated headers:

```
star_bus/include/
    star_bus_manager.h      # Main API (functions)
    star_bus_manager_types.h # Types for manager
    star_bus_common_types.h # Shared types (enums, forward decls)
    star_bus_protocol_types.h # Protocol-specific types
    star_bus_function_types.h # Function pointer types
    star_bus_event_types.h  # Event structures
    star_bus_types.h        # Controller header (includes all)
```

5.125.1 Controller Header Pattern

```
1  /* star_bus_types.h - includes all type headers */
2  #ifndef STAR_COMPONENT_BUS_TYPES_H
3  #define STAR_COMPONENT_BUS_TYPES_H
4
5  #ifdef __cplusplus
6  extern "C" {
7      #endif
8
9      /* Controller include file for all public types */
10     #include "star_bus_common_types.h"
11     #include "star_bus_event_types.h"
12     #include "star_bus_function_types.h"
13     #include "star_bus_manager_types.h"
14     #include "star_bus_protocol_types.h"
15
16     #ifdef __cplusplus
17     }
18 #endif
19
20 #endif /* STAR_COMPONENT_BUS_TYPES_H */
```

5.126 Project Root Structure

```

star-esp32-firmware/
  CLAUDE.md           # Claude Code instructions
  platformio.ini       # PlatformIO configuration
  sdkconfig.defaults  # ESP-IDF default config
  lib/                # Libraries/components
    star_core/         # Core interfaces
    star_bus/          # Bus abstraction
    star_error_handler/
    star_pin_validator/
    star_sensor_*/     # Sensor drivers
    star_motor/
    star_pid/
  src/                 # Main application
    main.c
  partitions/          # Partition tables
    partitions_4mb.csv
    partitions_16mb.csv
  docs/                # Documentation
    style_guide.tex
    style_guide_sections/
  scripts/             # Build/utility scripts
  test/                # Test files

```

5.127 Best Practices

- **One header per component:** Each component has its primary public header.
- **Minimize header dependencies:** Use forward declarations to reduce includes.
- **Self-contained headers:** Each header should compile independently.
- **Consistent naming:** Files match the component they contain.
- **Group related files:** Keep related headers and sources together.
- **Separate types from functions:** For complex modules, split into `_types.h`.
- **Document file purpose:** Use `@file` Doxygen blocks.

This section establishes the OSHWA (Open Source Hardware Association) inclusive terminology standards used throughout the STAR firmware codebase. These standards replace legacy terminology with more inclusive alternatives.

5.128 Overview

The STAR project follows inclusive terminology guidelines to ensure our codebase is welcoming to all contributors and avoids language with harmful historical connotations. This is particularly relevant in embedded systems where certain legacy terms have been deeply entrenched.

5.129 SPI Bus Terminology

5.129.1 COPI/CIPO (Not MOSI/MISO)

STAR Term	Legacy Term	Description
COPI	MOSI	Controller Out, Peripheral In
CIPO	MISO	Controller In, Peripheral Out
SCLK	SCLK	Serial Clock (unchanged)
CS	SS	Chip Select (unchanged)

```

1  /* CORRECT - OSHWA terminology */
2  typedef struct star_spi_bus_config {
3      gpio_num_t copi_io_num; /**< Controller Out, Peripheral In */
4      gpio_num_t cipo_io_num; /**< Controller In, Peripheral Out */
5      gpio_num_t sclk_io_num; /**< Serial Clock */
6      gpio_num_t cs_io_num;   /**< Chip Select */
7      /* ... */
8  } star_spi_bus_config_t;
9
10 /* Factory function uses inclusive naming */
11 star_bus_config_t* star_bus_config_create_spi_device(
12     const char*      name,
13     spi_host_device_t host,
14     gpio_num_t       copi_pin,   /* NOT mosi_pin */
15     gpio_num_t       cipo_pin,   /* NOT miso_pin */
16     gpio_num_t       sclk_pin,
17     int32_t          dma_chan,
18     const spi_device_interface_config_t* dev_cfg);
19
20 /* In documentation and comments */
21 /* CORRECT */
22 reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
23     bus_name,
24     "SPI COPI", /* Descriptive usage */
25     reg_func,
26     user_ctx,
27     true);
28
29 /* WRONG */
30 reg_result = register_pin_if_valid(curr->proto.spi.mosi_io_num,
31     bus_name,
32     "SPI MOSI", /* Legacy terminology */
33     reg_func,
34     user_ctx,
35     true);

```

5.130 I2C and General Bus Terminology

5.130.1 Controller/Peripheral (Not Master/Slave)

STAR Term	Legacy Term	Description
Controller	Master	Device that initiates communication
Peripheral	Slave	Device that responds to controller
Primary	Master	For configuration structures
Main	Master	For general reference

```

1  /* CORRECT - Controller/Peripheral terminology */
2  typedef struct star_spi_bus_config {
3      spi_host_device_t host;
4      bool is_peripheral; /**< True if operating as SPI peripheral */
5      /* ... */
6  } star_spi_bus_config_t;
7
8  /* Factory function for peripheral mode */
9  star_bus_config_t* star_bus_config_create_spi_peripheral(
10     const char*      name,
11     spi_host_device_t host,
12     gpio_num_t       copi_pin,
13     gpio_num_t       cipo_pin,
14     gpio_num_t       sclk_pin,
15     gpio_num_t       cs_pin,
16     int32_t          queue_size,
17     uint8_t          mode);
18
19 /* In comments and documentation */
20 /**
21  * @brief I2C Controller operations
22  *
23  * The controller device initiates all transactions on the bus.
24  * Peripheral devices respond when addressed by the controller.
25  */
26
27 /* WRONG */
28 bool is_slave; /* Use is_peripheral */
29 star_bus_config_create_spi_slave(); /* Use _peripheral */

```

5.131 Mapping Platform Legacy APIs

Some platform HALs (like ESP-IDF) still use legacy terminology internally. When interfacing with these APIs, document the mapping clearly.

```

1  /* From star_bus_protocol_types.h */
2  typedef struct star_spi_bus_config {
3      /* ... */
4
5      /* SPI pin naming follows OSHWA standard:
6       * - COPI (Controller Out, Peripheral In) - replaces MOSI
7       * - CIPO (Controller In, Peripheral Out) - replaces MISO
8       * This modern terminology removes master/slave language.
9       * Some platforms (like ESP-IDF) use legacy mosi/miso terminology,

```

```

10  * which we map here.
11  */
12  gpio_num_t copi_io_num;  /**< GPIO for COPI (maps to platform
    mosi_io_num) */
13  gpio_num_t cipo_io_num;  /**< GPIO for CIPO (maps to platform
    miso_io_num) */
14  /* ... */
15 } star_spi_bus_config_t;
16
17 /* When setting up platform HAL structures (example: ESP32) */
18 static rx_err_t internal_setup_spi_bus(star_bus_config_t* config)
19 {
20     spi_bus_config_t bus_cfg = {
21         /* Map our inclusive terminology to platform legacy names */
22         .mosi_io_num      = config->proto.spi.copi_io_num,
23         .miso_io_num      = config->proto.spi.cipo_io_num,
24         .sclk_io_num      = config->proto.spi.sclk_io_num,
25         .quadwp_io_num    = -1,
26         .quadhd_io_num    = -1,
27         .max_transfer_sz  = config->proto.spi.max_transfer_sz,
28     };
29     /* ... */
30 }
31
32 /* Similarly for I2C */
33 /* Some platforms use I2C_MODE_SLAVE/MASTER, we document as
    peripheral/controller */
34 i2c_config_t i2c_conf = {
35     .mode = I2C_MODE_MASTER,  /* Controller mode in STAR terminology */
36     /* or */
37     .mode = I2C_MODE_SLAVE,   /* Peripheral mode in STAR terminology */
38     /* ... */
39 };

```

5.132 Documentation Guidelines

When writing comments, documentation, or user-facing text:

```

1  /* CORRECT - Use inclusive terminology in all documentation */
2
3  /**
4   * @brief Configure ESP32 as SPI peripheral device
5   *
6   * The peripheral will respond to transactions initiated by an
7   * external SPI controller. Data flows:
8   * - COPI: Data received from controller
9   * - CIPO: Data sent to controller
10  *
11  * @param[in] name Unique name for this SPI peripheral instance
12  * @param[in] host SPI host device
13  * @param[in] copi_pin GPIO for Controller Out, Peripheral In
14  * @param[in] cipo_pin GPIO for Controller In, Peripheral Out
15  * ...

```

```
16 */
17 star_bus_config_t* star_bus_config_create_spi_peripheral(...);
18
19 /* WRONG - Legacy terminology */
20 /**
21  * @brief Configure ESP32 as SPI slave device
22  *
23  * The slave will respond to transactions initiated by the
24  * SPI master. Data flows:
25  * - MOSI: Data from master to slave
26  * - MISO: Data from slave to master
27  */
```

5.133 Code Review Checklist

When reviewing code, check for:

- **Variable names:** No “master”, “slave”, “mosi”, “miso”
- **Function names:** Use “controller”, “peripheral”, “copi”, “cipo”
- **Comments:** Update any legacy terminology
- **Documentation:** Ensure Doxygen uses inclusive terms
- **String literals:** Check log messages and error strings
- **Platform HAL mapping:** Document when interfacing with legacy APIs

5.134 Common Patterns

```
1 /* Logging with inclusive terminology */
2 RX_LOG_INFO(s_TAG, "SPI peripheral initialized on host %d", host);
3 RX_LOG_INFO(s_TAG, "I2C controller configured on port %d", port);
4
5 /* Error messages */
6 ESP_LOGE(s_TAG, "Failed to initialize SPI peripheral: %s",
7 rx_err_to_name(ret));
8 ESP_LOGE(s_TAG, "I2C controller transaction failed");
9
10 /* Pin registration descriptions */
11 internal_register_bus_pin(pin_iface, config->proto.spi.copi_io_num,
12 config->name, "SPI COPI", true);
13 internal_register_bus_pin(pin_iface, config->proto.spi.cipo_io_num,
14 config->name, "SPI CIPO", true);
```

5.135 Summary

Use This	Not This
Controller	Master
Peripheral	Slave
COPI	MOSI
CIPO	MISO
Primary	Master (for configs)
Main	Master (general)
Allowlist	Whitelist
Blocklist	Blacklist

Following these guidelines ensures our codebase is inclusive and welcoming while maintaining technical clarity. The terminology accurately describes the functional roles of devices without relying on harmful historical language.

This section describes the key architectural patterns used throughout the STAR firmware codebase. These patterns enable loose coupling, testability, and maintainability in embedded systems.

5.136 Dependency Injection (DIP)

The Dependency Inversion Principle (DIP) is the foundational architectural pattern of the STAR framework. High-level modules depend on abstract interfaces rather than concrete implementations.

5.136.1 Interface Definition

Interfaces are defined as structs containing function pointers and an opaque context pointer.

```

1  /* From star_error_interface.h */
2  typedef struct star_error_interface {
3      rx_err_t (*record_error)(void* ctx, rx_err_t error,
4          const char* message, const char* file,
5          int line, const char* func);
6      bool (*can_retry)(void* ctx);
7      rx_err_t (*reset_state)(void* ctx);
8      void* ctx; /* Opaque context pointer */
9  } star_error_interface_t;
10
11 /* From star_pin_interface.h */
12 typedef struct star_pin_interface {
13     rx_err_t (*register_pin)(void* ctx, int pin,
14         const char* description, bool shared);
15     rx_err_t (*unregister_pin)(void* ctx, int pin,
16         const char* description);
17     rx_err_t (*validate)(void* ctx);
18     void* ctx;
19 } star_pin_interface_t;

```

5.136.2 Interface Implementation

Concrete implementations provide the actual functionality.

```

1  /* Create error handler and get interface */
2  error_handler_t error_handler;
3  error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
4
5  star_error_interface_t error_iface;
6  error_handler_get_interface(&error_iface, &error_handler);
7
8  /* Implementation of get_interface */
9  void error_handler_get_interface(star_error_interface_t* iface,
10 error_handler_t* handler)
11 {
12     iface->record_error = adapter_record_error; /* Adapter function */
13     iface->can_retry     = adapter_can_retry;
14     iface->reset_state   = adapter_reset_state;
15     iface->ctx           = handler; /* Concrete handler as context */
16 }

```

5.136.3 Dependency Injection Pattern

Components receive their dependencies through initialization functions.

```

1  /* Component declares interface dependencies */
2  typedef struct star_bus_manager {
3      star_bus_config_t*    buses;
4      SemaphoreHandle_t     mutex;
5      const char*          tag;
6      star_error_interface_t* error_iface; /* Injected */
7      star_pin_interface_t* pin_iface;     /* Injected */
8      /* ... */
9  } star_bus_manager_t;
10
11 /* Initialization receives dependencies */
12 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
13 const char* tag,
14 star_error_interface_t* error_iface,
15 star_pin_interface_t* pin_iface)
16 {
17     RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
18 "Manager pointer is NULL");
19
20     manager->buses = NULL;
21     manager->error_iface = error_iface; /* Can be NULL */
22     manager->pin_iface   = pin_iface;   /* Can be NULL */
23     /* ... */
24 }
25
26 /* Usage with injected dependencies */
27 error_handler_t error_handler;
28 error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);

```

```

29
30 star_error_interface_t error_iface;
31 star_pin_interface_t pin_iface;
32 error_handler_get_interface(&error_iface, &error_handler);
33 pin_validator_get_interface(&pin_iface);
34
35 star_bus_manager_t bus_manager;
36 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

```

5.136.4 NULL Interface Handling

Components gracefully handle NULL interfaces.

```

1  /* Check interface before use */
2  if (manager->error_iface && manager->error_iface->record_error) {
3      manager->error_iface->record_error(manager->error_iface->ctx,
4      ret, "Operation failed",
5      __FILE__, __LINE__, __func__);
6  }
7
8  /* Or use convenience macro */
9  #define STAR_IFACE_RECORD_ERROR(iface, err, msg) \
10 ((iface) && (iface)->record_error \
11 ? (iface)->record_error((iface)->ctx, (err), (msg), \
12 __FILE__, __LINE__, __func__) \
13 : (err))

```

5.137 Bus Abstraction Pattern

The bus abstraction provides a unified interface for different communication protocols.

5.137.1 Unified Configuration

```

1  /* Single config type with protocol union */
2  typedef struct star_bus_config {
3      const char*      name;
4      star_bus_type_t  type; /* Discriminator */
5      bool             initialized;
6      void*            handle;
7      void*            user_ctx;
8
9      union {
10         star_i2c_bus_config_t  i2c;
11         star_spi_bus_config_t   spi;
12         star_gpio_bus_config_t  gpio;
13         star_adc_bus_config_t   adc;
14     } proto;
15
16     struct star_bus_config* next;
17 } star_bus_config_t;

```

5.137.2 Factory Functions

Factory functions create configurations for specific protocols.

```

1  /* I2C bus factory */
2  star_bus_config_t* star_bus_config_create_i2c(
3  const char* name,
4  i2c_port_t port,
5  uint8_t address,
6  gpio_num_t sda_pin,
7  gpio_num_t scl_pin,
8  uint32_t clk_speed);
9
10 /* SPI device factory */
11 star_bus_config_t* star_bus_config_create_spi_device(
12 const char* name,
13 spi_host_device_t host,
14 gpio_num_t cop_i_pin,
15 gpio_num_t cipo_pin,
16 gpio_num_t sclk_pin,
17 int32_t dma_chan,
18 const spi_device_interface_config_t* dev_cfg);
19
20 /* GPIO bus factory */
21 star_bus_config_t* star_bus_config_create_gpio(
22 const char* name,
23 gpio_num_t* pins,
24 uint8_t pin_count);

```

5.137.3 Manager Pattern

The bus manager coordinates multiple bus configurations.

```

1  /* Initialize manager */
2  star_bus_manager_t bus_manager;
3  star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
4
5  /* Add buses */
6  star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
7  "sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22, 400000);
8  star_bus_manager_add_bus(&bus_manager, i2c_bus);
9
10 /* Find and use buses */
11 star_bus_config_t* bus = star_bus_manager_find_bus(&bus_manager,
12 "sensor_i2c");
13
14 /* Safe access with callback (prevents use-after-free) */
15 rx_err_t my_callback(star_bus_config_t* bus, void* ctx) {
16     /* Access bus safely while mutex is held */
17     return RX_OK;
18 }
19
20 star_bus_manager_with_bus(&bus_manager, "sensor_i2c", my_callback,
21 NULL);

```

5.138 Operations Structure Pattern

Group related function pointers for pluggable implementations.

```

1  /* Define operations structure */
2  typedef struct star_i2c_ops {
3      star_i2c_write_fn_t      write;
4      star_i2c_read_fn_t      read;
5      star_i2c_write_command_fn_t write_command;
6      star_i2c_read_raw_fn_t   read_raw;
7  } star_i2c_ops_t;
8
9  /* Initialize with default implementations */
10 void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
11 {
12     if (ops == NULL) {
13         RX_LOG_ERROR(s_TAG, "Cannot initialize NULL ops pointer");
14         return;
15     }
16     ops->write      = internal_star_bus_i2c_write;
17     ops->read       = internal_star_bus_i2c_read;
18     ops->write_command = internal_star_bus_i2c_write_command;
19     ops->read_raw    = internal_star_bus_i2c_read_raw;
20 }
21
22 /* Use through function pointers */
23 rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
24 const char* name, ...)
25 {
26     star_bus_config_t* bus_config = star_bus_manager_find_bus(manager,
27 name);
28     /* ... validation ... */
29     return bus_config->proto.i2c.ops.write(bus_config, data, len,
30 reg_addr, bytes_written);
31 }

```

5.139 Thread-Safe Callback Pattern

Safely access shared resources through callbacks.

```

1  /* Callback type */
2  typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
3 void* user_ctx);
4
5  /* Thread-safe access function */
6  rx_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,
7 const char* name,
8 star_bus_callback_t callback,
9 void* user_ctx)
10 {
11     ESP_RETURN_ON_FALSE(manager && name && callback,
12 RX_ERR_INVALID_ARG, s_TAG, "Invalid arguments");
13 }

```

```

14  /* Acquire mutex */
15  if (xSemaphoreTake(manager->mutex,
16      pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
17      != pdTRUE) {
18      return RX_ERR_TIMEOUT;
19  }
20
21  /* Find bus while holding mutex */
22  star_bus_config_t* found_bus = NULL;
23  for (star_bus_config_t* curr = manager->buses; curr; curr =
24      curr->next) {
25      if (curr->name && strcmp(curr->name, name) == 0) {
26          found_bus = curr;
27          break;
28      }
29  }
30
31  if (found_bus == NULL) {
32      xSemaphoreGive(manager->mutex);
33      return ESP_ERR_NOT_FOUND;
34  }
35
36  /* Invoke callback while holding mutex - prevents use-after-free */
37  rx_err_t callback_result = callback(found_bus, user_ctx);
38
39  xSemaphoreGive(manager->mutex);
40  return callback_result;

```

5.140 Thread Safety Patterns

5.140.1 Mutex Timeout Convention

Standard mutex timeout is 1000ms (configurable via Kconfig).

```

1  /* Standard mutex acquisition pattern */
2  if (xSemaphoreTake(manager->mutex,
3      pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
4      != pdTRUE) {
5      ESP_LOGE(manager->tag, "Timeout acquiring mutex");
6      return ESP_ERR_TIMEOUT;
7  }
8
9  /* Critical section */
10 /* ... protected operations ... */
11
12 xSemaphoreGive(manager->mutex);

```

5.140.2 Mutex with Cleanup (goto pattern)

```

1 rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2 star_bus_config_t* config)
3 {
4     RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG,
5 s_TAG, "Invalid arguments");
6
7     rx_err_t ret = RX_OK;
8
9     if (xSemaphoreTake(manager->mutex,
10 pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
11 != pdTRUE) {
12         return RX_ERR_TIMEOUT;
13     }
14
15     /* Check for duplicate name */
16     for (star_bus_config_t* curr = manager->buses; curr; curr =
17         curr->next) {
18         if (curr->name && strcmp(curr->name, config->name) == 0) {
19             ret = RX_ERR_INVALID_STATE;
20             goto add_give_mutex; /* Single exit point */
21         }
22     }
23
24     /* Initialize bus hardware */
25     ret = star_bus_config_init(config, manager);
26     if (ret != RX_OK) {
27         goto add_give_mutex;
28     }
29
30     /* Add to list */
31     config->next = manager->buses;
32     manager->buses = config;
33
34     add_give_mutex:
35     xSemaphoreGive(manager->mutex);
36     return ret;
37 }

```

5.141 Error Handler Ownership Pattern

Components track whether they own their error handler for proper cleanup.

```

1 /* Sensor handle with ownership tracking */
2 typedef struct mpu6050_handle {
3     star_bus_manager_t*    bus_manager;
4     const char*           bus_name;
5     star_error_interface_t error_iface;
6     bool                  owns_error_handler; /* Ownership flag */
7     SemaphoreHandle_t      mutex;
8     bool                  initialized;
9     /* ... */
10 } mpu6050_handle_t;

```

```

11
12 /* Initialization with optional interface */
13 rx_err_t star_sensor_mpu6050_init(mpu6050_handle_t* handle,
14 star_bus_manager_t* bus_manager,
15 const char* bus_name,
16 star_error_interface_t* error_iface,
17 const mpu6050_config_t* config)
18 {
19     if (error_iface != NULL) {
20         /* Use provided interface - we don't own it */
21         handle->error_iface = error_iface;
22         handle->owns_error_handler = false;
23     } else {
24         /* Create default - we own it */
25         handle->error_iface = star_error_interface_create_default();
26         handle->owns_error_handler = true;
27     }
28     /* ... */
29 }
30
31 /* Cleanup respects ownership */
32 rx_err_t star_sensor_mpu6050_deinit(mpu6050_handle_t* handle)
33 {
34     /* Clean up error interface if we own it */
35     star_error_interface_cleanup(handle->error_iface,
36 handle->owns_error_handler);
37     handle->error_iface = NULL;
38     /* ... */
39 }
40
41 /* Helper for cleanup */
42 void star_error_interface_cleanup(star_error_interface_t* error_iface,
43 bool owns_handler)
44 {
45     if (!owns_handler || !error_iface) {
46         return; /* Not our responsibility */
47     }
48     error_handler_t* handler = (error_handler_t*)error_iface->ctx;
49     error_handler_destroy_default(handler);
50     free(error_iface);
51 }

```

5.142 Linked List Management

Bus configurations are managed as a singly-linked list.

```

1 /* Add to head of list (O(1)) */
2 config->next = manager->buses;
3 manager->buses = config;
4
5 /* Remove from list (O(n)) */
6 star_bus_config_t* prev = NULL;

```

```
7 for (star_bus_config_t* curr = manager->buses; curr; prev = curr,
   curr = curr->next) {
8     if (strcmp(curr->name, name) == 0) {
9         if (prev == NULL) {
10             manager->buses = curr->next; /* Remove head */
11         } else {
12             prev->next = curr->next; /* Remove middle/tail */
13         }
14         curr->next = NULL;
15         /* ... cleanup curr ... */
16         break;
17     }
18 }
19
20 /* Iterate through list */
21 for (star_bus_config_t* curr = manager->buses; curr; curr =
   curr->next) {
22     /* Process each configuration */
23 }
```

5.143 Best Practices Summary

- **Use dependency injection:** Components receive interfaces, not concrete types.
- **Handle NULL interfaces:** Always check before dereferencing.
- **Track ownership:** Know who is responsible for cleanup.
- **Use callbacks for safe access:** Prevents use-after-free with shared resources.
- **Consistent mutex timeout:** 1000ms standard timeout.
- **Single exit point with goto:** Ensures mutex release on all paths.
- **Factory functions:** Create configurations with proper initialization.
- **Operations structures:** Enable pluggable implementations.

5.144 Prefer Alternatives to Macros

Rule: Avoid macros unless absolutely necessary. Use modern C alternatives:

- `static const` for constants
- `inline` functions for simple operations
- `enum` for related constants

Rationale: Macros lack type safety, scope control, and debugger visibility.

Good:

```
static const uint32_t k_max_buffer_size = 256;
static const float k_pi = 3.14159f;

static inline int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Bad:

```
#define MAX_BUFFER_SIZE 256
#define PI 3.14159
#define MAX(a,b) ((a) > (b) ? (a) : (b))
```

5.145 When Macros Are Necessary

Use macros only for:

1. Array sizes (compile-time constants)
2. Conditional compilation (`#ifdef`, `#if`)
3. Token pasting and stringification
4. Platform-specific code paths

5.146 Macro Safety Rules

5.146.1 Wrap Statement-Like Macros in `do-while(0)`

Rule: Always wrap multi-statement macros in `do { ... } while(0)`.

Why: Prevents bugs when used in if-statements without braces.

Correct:

```
#define LOG_ERROR(msg) \
    do { \
        log_timestamp(); \
        log_message(msg); \
    } while (0)
```

Incorrect (causes bugs):

```
#define LOG_ERROR(msg) \
    log_timestamp(); \
    log_message(msg)
```

```
/* Bug example: */
if (error)
    LOG_ERROR("fail"); /* Only log_timestamp() is conditional! */
else
    handle_success();
```

5.146.2 Parenthesize All Macro Arguments

Rule: Wrap all argument uses and the entire expression in parentheses.

Correct:

```
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

Incorrect (causes bugs):

```
#define SQUARE(x) x * x

int result = SQUARE(2 + 3); /* Expands to: 2 + 3 * 2 + 3 = 11, not 25! */
```

5.146.3 Beware of Side Effects and Multiple Evaluation

Rule: Document if macro arguments are evaluated multiple times. Never pass arguments with side effects.

Dangerous:

```
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int i = 5;
int result = MAX(i++, 10); /* i is incremented TWICE! */
```

Solution: Use inline functions or add warning comment:

```
/* WARNING: Arguments evaluated multiple times - no side effects! */
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

5.147 Multi-line Macro Formatting

Rules:

- Use backslash continuation aligned consistently
- Indent continuation lines for readability
- Place opening brace on same line as do

Example:

```
#define RECORD_ERROR(handler, code, desc) \
do { \
    if ((handler)->error_iface != NULL) { \
        (handler)->error_iface->record_error( \
            (code), \
            (desc), \
```

```

        __FILE__,
        __LINE__
    );
}
} while (0)

```

5.148 Naming Conventions

Rule: Use SCREAMING_SNAKE_CASE for all macros.

Examples:

```

/* Simple constants */
#define MAX_RETRY_COUNT (3)
#define MOTOR_PWM_FREQ_HZ (20000)

/* Function-like macros (always use parentheses) */
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

/* Multi-statement macros (always use do-while(0)) */
#define LOG_ERROR(msg) \
do { \
    log_timestamp(); \
    log_message(msg); \
} while (0)

#define RECORD_ERROR(handler, code, desc) \
do { \
    error_handler_record_error( \
        (handler), (code), (desc), \
        __FILE__, __LINE__, __func__ \
    ); \
} while (0)

```

5.149 Include Guards

Use traditional include guards for maximum portability:

```

#ifndef STAR_COMPONENT_FILENAME_H
#define STAR_COMPONENT_FILENAME_H

/* File contents */

#endif /* STAR_COMPONENT_FILENAME_H */

```

Do not use `#pragma once` (see Section 12 for details).

5.150 Conditional Compilation

Best Practices:

- Document why conditionals exist
- Keep conditional blocks small
- Use configuration headers instead of scattered `#ifdef`

Good:

```
/* Platform-specific GPIO initialization */
#if defined(CONFIG_STAR_BOARD_ESP32_S3)
    configure_esp32s3_gpio();
#elif defined(CONFIG_STAR_BOARD_ESP32_WROOM)
    configure_esp32_wroom_gpio();
#else
    #error "Unsupported board configuration"
#endif
```

This section establishes the unit suffix naming standard used throughout the STAR firmware codebase. All numeric fields use SI (International System of Units) units with explicit suffixes to eliminate unit ambiguity and enable automatic documentation generation.

5.151 Overview

The firmware uses the MKS (Meter-Kilogram-Second) system as the foundation for all physical measurements. Every numeric variable that represents a physical quantity must include a unit suffix that clearly identifies its meaning.

5.152 Benefits

- **Eliminates Unit Ambiguity:** No confusion about whether a value is in meters or millimeters, degrees or radians
- **Self-Documenting Code:** Variable names explicitly state their units
- **Protocol Buffer Compatibility:** Matches naming conventions used in `star-proto/` messages
- **Prevents Conversion Errors:** Explicit units reduce the risk of unit mismatch bugs

5.153 Standard Unit Suffixes

Table 20: Standard Unit Suffixes for STAR Firmware

Suffix	Unit	Proto Example	C Example
_m	meters	x_m	distance_m
_mps	meters/second	velocity_mps	speed_mps
_mps2	m/s ²	accel_x_mps2	acceleration_mps2
_rad	radians	angle_rad	heading_rad
_rad_per_s	rad/second	omega_rad_per_s	angular_vel_rad_per_s
_celsius	degrees C	temperature_celsius	motor_temp_celsius
_deci_celsius	0.1°C	temp_deci_celsius	(BMS native format)
_us	microseconds	timestamp_us	elapsed_us
_ms	milliseconds	timeout_ms	period_ms
_ma	milliamps	current_ma	motor_current_ma
_mv	millivolts	cell_mv	pack_voltage_mv
_mw	milliwatts	power_mw	consumption_mw
_percent	0-100%	soc_percent	duty_cycle_percent

5.154 Usage Examples

5.154.1 Distance and Velocity

```

1  /* CORRECT - Distance and velocity with units */
2  float distance_m = 1.5f;           /* 1.5 meters */
3  float velocity_mps = 2.0f;         /* 2.0 meters per second */
4  float acceleration_mps2 = 9.81f;   /* 9.81 m/s^2 (gravity) */
5
6  /* Motor velocity tracking */
7  float target_velocity_mps = 0.5f;
8  float current_velocity_mps;
9  star_encoder_get_velocity_rpm(&encoder, 10.0f, 500,
10     &current_velocity_mps);
11
12 /* WRONG - No unit suffix */
13 float distance = 1.5f;             /* What unit? Meters? Millimeters? Feet? */
14 float velocity = 2.0f;             /* Meters per second? RPM? */

```

5.154.2 Angular Measurements

```

1  /* CORRECT - Angular measurements in radians */
2  float heading_rad = 1.57f;         /* 90 degrees in radians */
3  float angular_velocity_rad_per_s = 3.14f; /* pi radians per second */
4
5  /* Robot pose */
6  typedef struct {
7      float x_m;
8      float y_m;

```

```

9   float theta_rad;  /* Heading angle */
10  } robot_pose_t;
11
12  /* WRONG - Ambiguous angle units */
13  float heading = 90.0f; /* Degrees or radians? */
14  float omega = 3.14f;   /* What unit? */

```

5.154.3 Temperature

```

1  /* CORRECT - Temperature with units */
2  float motor_temp_celsius;
3  star_sensor_ds18b20_read_temp(&temp_sensor, &motor_temp_celsius);
4
5  /* BMS uses deci-celsius (0.1 degree resolution) */
6  int16_t cell_temp_deci_celsius = 235; /* 23.5 degrees C */
7
8  /* Thermal shutdown threshold */
9  #define MOTOR_THERMAL_SHUTDOWN_CELSIUS (85.0f)
10
11  if (motor_temp_celsius > MOTOR_THERMAL_SHUTDOWN_CELSIUS) {
12      star_drv8243_stop(&motor, true); /* Emergency brake */
13  }
14
15  /* WRONG */
16  float motor_temp = 75.0f; /* Celsius? Fahrenheit? Kelvin? */

```

5.154.4 Time

```

1  /* CORRECT - Time units */
2  uint32_t timeout_ms = 1000; /* 1 second timeout */
3  uint64_t timestamp_us;      /* Microsecond timestamp */
4  star_encoder_get_timestamp_us(&encoder, &timestamp_us);
5
6  /* Timing for control loops */
7  const uint32_t control_period_ms = 1; /* 1ms = 1000Hz */
8  const uint32_t sensor_poll_us = 10000; /* 10ms in microseconds */
9
10 /* WRONG */
11 uint32_t timeout = 1000; /* Milliseconds? Seconds? Ticks? */
12 uint64_t timestamp; /* What resolution? */

```

5.154.5 Electrical Measurements

```

1  /* CORRECT - Electrical units */
2  uint32_t motor_current_ma;
3  star_drv8243_read_current(&motor, &motor_current_ma);
4
5  uint32_t cell_voltage_mv = 3700; /* 3.7V in millivolts */
6  uint32_t pack_voltage_mv = 11100; /* 11.1V (3S LiPo) */

```

```

7 uint32_t power_mw = 7400;           /* 7.4W in milliwatts */
8
9 /* Current limiting */
10 #define MAX_MOTOR_CURRENT_MA (2000) /* 2A limit */
11
12 if (motor_current_ma > MAX_MOTOR_CURRENT_MA) {
13     ESP_LOGW(s_TAG, "Motor current limit exceeded: %lu mA",
14             motor_current_ma);
15     star_drv8243_set_speed(&motor, 0.0f);
16 }
17
18 /* WRONG */
19 uint32_t motor_current = 1500; /* Milliamps? Amps? */
20 uint32_t voltage = 3700;       /* Millivolts? Volts? */

```

5.154.6 Percentage

```

1 /* CORRECT - Percentage (0-100 range) */
2 float duty_cycle_percent = 75.0f; /* 75% duty cycle */
3 uint8_t battery_soc_percent = 85; /* 85% state of charge */
4 float throttle_percent = 50.0f; /* 50% throttle */
5
6 star_drv8243_set_speed(&motor, duty_cycle_percent);
7
8 /* WRONG */
9 float duty_cycle = 0.75f; /* Normalized (0-1)? Percentage (0-100)? */
10
11 uint8_t battery_soc = 85; /* Percentage? Raw ADC value? */

```

5.155 Protocol Buffer Integration

The unit suffix convention matches the naming used in Protocol Buffer messages:

```

1 // From star-proto/proto/robot_command.proto
2 message VelocityCommand {
3     float velocity_mps = 1;           // Meters per second
4     float angular_velocity_rad_per_s = 2; // Radians per second
5 }
6
7 message BatteryState {
8     repeated uint32 cell_mv = 1;      // Cell voltages in millivolts
9     uint32 current_ma = 2;            // Pack current in milliamps
10    uint8 soc_percent = 3;             // State of charge (0-100%)
11    int16 temperature_celsius = 4;    // Pack temperature
12 }

```

5.156 Conversion Functions

When converting between units, use descriptive function names:

```

1  /* CORRECT - Clear unit conversion */
2  float rpm_to_mps(float rpm, float wheel_diameter_m)
3  {
4      float wheel_circumference_m = 3.14159f * wheel_diameter_m;
5      float revolutions_per_second = rpm / 60.0f;
6      return revolutions_per_second * wheel_circumference_m;
7  }
8
9  float celsius_to_deci_celsius(float temp_celsius)
10 {
11     return temp_celsius * 10.0f;
12 }
13
14 float mv_to_volts(uint32_t voltage_mv)
15 {
16     return voltage_mv / 1000.0f;
17 }
18
19 /* WRONG - Ambiguous conversions */
20 float convert_rpm(float rpm); /* Convert to what? */
21 float scale_temp(float temp); /* What units? */

```

5.157 Summary

- **Always use unit suffixes** for physical quantities
- **Match Protocol Buffer conventions** for consistency
- **Use SI base units** when possible (meters, radians, seconds)
- **Document conversions** clearly in function names
- **Prefer millivolts/milliamps** over floating-point volts/amps for integer precision

This convention eliminates an entire class of bugs caused by unit ambiguity and makes the codebase more maintainable.

This section establishes memory management rules for STAR firmware across all platforms. Embedded platforms have limited RAM (e.g., ESP32-S3: 512KB SRAM, RX72N: 1MB SRAM), making proper memory management critical for system stability and performance.

5.158 No Dynamic Allocation

Rule: Never use `malloc()`, `calloc()`, `realloc()`, or `free()` in production firmware.

5.158.1 Rationale

- **Heap Fragmentation:** Repeated allocation/deallocation causes memory fragmentation over time

- **Memory Leaks:** Forgotten `free()` calls lead to gradual RAM exhaustion
- **Unpredictable Timing:** Allocation time is non-deterministic (breaks real-time constraints)
- **Out-of-Memory Failures:** No recovery mechanism when heap is exhausted
- **Debugging Difficulty:** Memory corruption bugs are extremely hard to track down

5.158.2 Alternatives

Use static allocation instead:

```

1  /* WRONG - Dynamic allocation */
2  void process_sensor_data(void)
3  {
4      float* samples = (float*)malloc(1000 * sizeof(float));
5      if (samples == NULL) {
6          RX_LOG_ERROR(s_TAG, "Out of memory!");
7          return;
8      }
9
10     /* Process samples... */
11
12     free(samples); /* Easy to forget! */
13 }
14
15 /* CORRECT - Static allocation */
16 static float s_samples[1000]; /* Allocated once at compile time */
17
18 void process_sensor_data(void)
19 {
20     /* Use s_samples directly - no allocation, no leaks */
21     for (int i = 0; i < 1000; i++) {
22         s_samples[i] = read_sensor();
23     }
24 }
```

5.158.3 Platform Heap Functions

Some platforms provide specialized heap functions (e.g., ESP-IDF's `heap_caps_malloc`) that are **only acceptable** for:

- **One-time initialization:** Allocating resources during startup that persist for the lifetime of the program
- **FreeRTOS task stacks:** Created once during task initialization
- **Third-party libraries:** When required by external dependencies (document clearly)

```
1  /* ACCEPTABLE - One-time allocation during init */
2  rx_err_t sensor_driver_init(sensor_handle_t* handle)
3  {
4      /* Allocate persistent buffer once */
5      handle->rx_buffer = heap_caps_malloc(DMA_BUFFER_SIZE,
6      MALLOC_CAP_DMA);
7      if (handle->rx_buffer == NULL) {
8          return ESP_ERR_NO_MEM;
9      }
10     handle->owns_buffer = true; /* Track ownership */
11     return RX_OK;
12 }
13
14 /* Must have corresponding cleanup */
15 rx_err_t sensor_driver_deinit(sensor_handle_t* handle)
16 {
17     if (handle->owns_buffer && handle->rx_buffer) {
18         free(handle->rx_buffer);
19         handle->rx_buffer = NULL;
20     }
21     return RX_OK;
22 }
```

5.159 Avoid Large Stack Allocations

Rule: Never allocate large arrays on the stack. Use static or global storage instead.

5.159.1 Stack Size Constraints

- Typical embedded task stack: **2KB–8KB** (platform and RTOS dependent)
- ISR stack: Even smaller (~1KB)
- Stack overflow causes silent corruption or hard faults

5.159.2 Examples

```
1  /* WRONG - Large stack allocation */
2  void process_image(void)
3  {
4      uint8_t frame_buffer[640 * 480]; /* 307KB on stack! */
5      /* Stack overflow guaranteed! */
6  }
7
8  /* WRONG - Nested large allocations */
9  void motor_control_loop(void)
10 {
11     float pid_state[100]; /* 400 bytes */
12     uint32_t encoder_samples[500]; /* 2000 bytes */
13     /* Total: 2400 bytes - may overflow 3KB stack */
```

```

14 }
15
16 /* CORRECT - Use static storage */
17 static uint8_t s_frame_buffer[640 * 480]; /* In .bss, not stack */
18
19 void process_image(void)
20 {
21     /* Use s_frame_buffer safely */
22     memset(s_frame_buffer, 0, sizeof(s_frame_buffer));
23 }
24
25 /* CORRECT - Small stack allocations are fine */
26 void calculate_checksum(const uint8_t* data, size_t len)
27 {
28     uint32_t crc = 0; /* 4 bytes - perfectly fine on stack */
29     /* ... */
30 }

```

5.159.3 Stack Usage Guidelines

Table 21: Stack Allocation Guidelines

Size	Allowed	Storage Location
< 256 bytes	Yes	Stack (automatic)
256 bytes – 1 KB	Caution	Prefer static if persistent
> 1 KB	No	Static or global only
> 10 KB	Never	Static with explicit comment

5.160 Use const for Flash Storage

Rule: Place all read-only data in flash/ROM using the `const` keyword.

5.160.1 Rationale

- Embedded platforms typically have large flash but limited RAM (e.g., ESP32: 16MB flash / 512KB RAM, RX72N: 4MB flash / 1MB RAM)
- `const` data is stored in flash and accessed directly (no RAM copy)
- Saves precious SRAM for runtime variables

5.160.2 Examples

```

1 /* WRONG - Lookup table in RAM (wastes 256 bytes) */
2 static uint8_t sine_lookup[256] = {
3     0, 3, 6, 9, 12, /* ... */
4 };
5

```

```

6  /* CORRECT - Lookup table in flash (zero RAM usage) */
7  static const uint8_t sine_lookup[256] = {
8      0, 3, 6, 9, 12, /* ... */
9  };
10
11 /* WRONG - String literals without const */
12 static char* error_messages[] = {
13     "Sensor timeout",
14     "Invalid checksum",
15     "Bus error"
16 };
17
18 /* CORRECT - Strings in flash */
19 static const char* const error_messages[] = {
20     "Sensor timeout",
21     "Invalid checksum",
22     "Bus error"
23 };
24
25 /* CORRECT - Default configuration in flash */
26 static const star_pid_config_t default_pid_config = {
27     .kp = 1.0f,
28     .ki = 0.5f,
29     .kd = 0.1f,
30     .output_min = -100.0f,
31     .output_max = 100.0f,
32 };
33
34 /* Usage: Copy to RAM only when needed */
35 star_pid_config_t runtime_config;
36 memcpy(&runtime_config, &default_pid_config, sizeof(runtime_config));

```

5.160.3 Pointer Declarations

Be careful with pointer const placement:

```

1  /* Pointer to const data (data in flash, pointer in RAM) */
2  const char* error_msg = "Timeout";
3
4  /* Const pointer to data (pointer in flash, data in RAM) */
5  char* const buffer_ptr = buffer;
6
7  /* Const pointer to const data (both in flash) */
8  const char* const error_table[] = {"Error 1", "Error 2"};

```

5.161 Validate All Buffers

Rule: Always validate buffer sizes before `memcpy()`, DMA operations, or ring buffer manipulations.

5.161.1 Rationale

- Buffer overruns cause memory corruption
- Corruption may manifest much later, making debugging extremely difficult
- Real-time systems have no memory protection (unlike Linux)

5.161.2 Validation Patterns

```
1  /* WRONG - Unchecked memcpy */
2  void receive_packet(uint8_t* data, size_t len)
3  {
4      memcpy(rx_buffer, data, len); /* What if len > buffer size? */
5  }
6
7  /* CORRECT - Validate before copy */
8  #define RX_BUFFER_SIZE (512)
9  static uint8_t rx_buffer[RX_BUFFER_SIZE];
10
11 void receive_packet(const uint8_t* data, size_t len)
12 {
13     if (data == NULL || len > RX_BUFFER_SIZE) {
14         RX_LOG_ERROR(s_TAG, "Invalid packet: len=%zu (max=%d)",
15             len, RX_BUFFER_SIZE);
16         return;
17     }
18     memcpy(rx_buffer, data, len);
19 }
20
21 /* CORRECT - Clamp to buffer size */
22 void receive_packet_safe(const uint8_t* data, size_t len,
23     size_t* bytes_copied)
24 {
25     size_t copy_len = (len < RX_BUFFER_SIZE) ? len : RX_BUFFER_SIZE;
26     memcpy(rx_buffer, data, copy_len);
27     *bytes_copied = copy_len;
28
29     if (len > RX_BUFFER_SIZE) {
30         RX_LOG_WARN(s_TAG, "Packet truncated: %zu bytes lost",
31             len - RX_BUFFER_SIZE);
32     }
33 }
```

5.161.3 DMA Buffer Validation

```
1  /* DMA buffers must be validated AND aligned */
2  rx_err_t start_dma_transfer(const uint8_t* src, size_t len)
3  {
4      /* Validate buffer pointer */
5      if (src == NULL) {
6          return RX_ERR_INVALID_ARG;
7      }
8  }
```

```

7   }
8
9   /* Validate length */
10  if (len == 0 || len > MAX_DMA_TRANSFER_SIZE) {
11      RX_LOG_ERROR(s_TAG, "Invalid DMA length: %zu", len);
12      return ESP_ERR_INVALID_SIZE;
13  }
14
15  /* Check alignment (DMA requires 4-byte alignment) */
16  if (((uintptr_t)src & 0x3) != 0) {
17      RX_LOG_ERROR(s_TAG, "DMA buffer not 4-byte aligned");
18      return RX_ERR_INVALID_ARG;
19  }
20
21  /* Proceed with DMA */
22  return spi_device_transmit(spi_handle, &transaction);
23 }

```

5.161.4 Ring Buffer Validation

```

1   /* CORRECT - Ring buffer with overflow protection */
2   typedef struct {
3       uint8_t buffer[256];
4       size_t head;
5       size_t tail;
6       size_t count;
7   } ring_buffer_t;
8
9   rx_err_t ring_buffer_write(ring_buffer_t* rb, const uint8_t* data,
10  size_t len)
11  {
12      if (rb == NULL || data == NULL) {
13          return RX_ERR_INVALID_ARG;
14      }
15
16      /* Check available space */
17      size_t available = sizeof(rb->buffer) - rb->count;
18      if (len > available) {
19          RX_LOG_WARN(s_TAG, "Ring buffer overflow: %zu bytes lost",
20              len - available);
21          return ESP_ERR_NO_MEM;
22      }
23
24      /* Safe to write */
25      for (size_t i = 0; i < len; i++) {
26          rb->buffer[rb->head] = data[i];
27          rb->head = (rb->head + 1) % sizeof(rb->buffer);
28          rb->count++;
29      }
30
31      return RX_OK;
32  }

```

5.162 Summary

Table 22: Memory Management Rules Summary

Rule	Implementation
No dynamic allocation	Use static/global buffers instead of malloc/free
Avoid large stack arrays	Use static storage for arrays > 256 bytes
Use <code>const</code>	Place lookup tables, strings, defaults in flash
Validate all buffers	Check lengths before memcpy, DMA, ring buffers

Following these rules ensures stable, predictable memory usage and prevents an entire class of memory-related bugs that are extremely difficult to debug in embedded systems.

This section establishes best practices for real-time embedded systems development across all STAR platforms. These guidelines are critical for motor control (1000Hz loops), sensor fusion, and safety-critical robotics applications.

5.163 Interrupt Service Routines (ISRs)

5.163.1 Minimize ISR Work

Rule: Keep ISRs as short as possible. Set flags or queue work to background tasks instead of performing complex operations.

```

1  /* WRONG - Too much work in ISR */
2  static void IRAM_ATTR gpio_isr_handler(void* arg)
3  {
4      /* Reading I2C sensor blocks for milliseconds! */
5      float temperature = read_i2c_sensor();
6      update_pid_controller(temperature);
7      adjust_motor_speed();
8      log_telemetry_to_sd_card(); /* File I/O in ISR! */
9  }
10
11 /* CORRECT - Minimal ISR, queue work to task */
12 static volatile bool s_encoder_event = false;
13 static QueueHandle_t s_encoder_queue;
14
15 static void IRAM_ATTR encoder_isr_handler(void* arg)
16 {
17     /* Only set flag and queue event */
18     s_encoder_event = true;
19
20     encoder_event_t event = {.timestamp_us = esp_timer_get_time()};
21     xQueueSendFromISR(s_encoder_queue, &event, NULL);
22
23     /* ISR done in microseconds, not milliseconds */
24 }
25
26 /* Background task processes events */
27 void encoder_task(void* arg)

```

```

28 {
29     encoder_event_t event;
30     while (1) {
31         if (xQueueReceive(s_encoder_queue, &event, portMAX_DELAY)) {
32             /* Heavy processing here, not in ISR */
33             process_encoder_event(&event);
34         }
35     }
36 }

```

5.163.2 ISR Constraints

- **No blocking operations:** No mutexes, delays, or I/O
- **Use IRAM_ATTR:** Places ISR in IRAM for faster execution
- **FromISR variants:** Use `xQueueSendFromISR()`, not `xQueueSend()`
- **Keep stack usage low:** ISR stack is typically 1KB or less
- **No floating-point:** Context save/restore is expensive

5.164 No Blocking Delays

Rule: Never use `vTaskDelay()` or busy-wait loops in time-critical code. Use event-driven design.

```

1  /* WRONG - Blocking delay in motor control */
2  void motor_control_loop(void)
3  {
4      while (1) {
5          read_encoder();
6          compute_pid();
7          update_motor();
8
9          vTaskDelay(pdMS_TO_TICKS(1)); /* Blocks, jitter from scheduler */
10     }
11 }
12
13 /* CORRECT - Timer-driven execution */
14 static void motor_control_timer_callback(void* arg)
15 {
16     /* Called exactly every 1ms by hardware timer */
17     read_encoder();
18     compute_pid();
19     update_motor();
20 }
21
22 void setup_motor_control(void)
23 {
24     esp_timer_create_args_t timer_args = {
25         .callback = motor_control_timer_callback,

```

```

26     .name = "motor_ctrl"
27 };
28
29     esp_timer_handle_t timer;
30     esp_timer_create(&timer_args, &timer);
31     esp_timer_start_periodic(timer, 1000);  /* 1ms = 1000us */
32 }
33
34 /* CORRECT - Event-driven sensor reading */
35 void sensor_task(void* arg)
36 {
37     while (1) {
38         /* Wait for event, no polling */
39         ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
40
41         /* Event occurred, process it */
42         process_sensor_data();
43     }
44 }

```

5.165 Real-Time and Safety

Table 23: Real-Time Best Practices

Practice	Implementation
Know timing constraints	Profile worst-case execution times (WCET)
Use watchdog timers	Automatic reset on firmware lockup
Handle sensor timeouts	Never assume sensors always respond
Fail safe	On error, disable actuators and enter safe state
CRC critical data	Validate packets and stored configuration

5.165.1 Timing Constraints

```

1  /* Profile execution time using GPIO toggle */
2  void motor_control_loop(void)
3  {
4      gpio_set_level(DEBUG_PIN, 1);  /* Pin HIGH */
5
6      /* Critical timing section */
7      read_encoder();
8      compute_pid();
9      update_motor();
10
11     gpio_set_level(DEBUG_PIN, 0);  /* Pin LOW */
12
13     /* Measure with oscilloscope:
14      * Pulse width = worst-case execution time (WCET) */
15 }
16
17 /* Or use platform-specific cycle counter (ESP32, RX72N, etc.) */

```

```

18 uint32_t start_cycles = esp_cpu_get_cycle_count();
19
20 critical_function();
21
22 uint32_t end_cycles = esp_cpu_get_cycle_count();
23 uint32_t elapsed_us = (end_cycles - start_cycles) / (CPU_FREQ_MHZ);
24 RX_LOG_INFO(s_TAG, "Execution time: %lu us", elapsed_us);

```

5.165.2 Watchdog Timers

```

1  /* Initialize task watchdog */
2  #include "esp_task_wdt.h"
3
4  void motor_control_task(void* arg)
5  {
6      /* Subscribe to watchdog (5 second timeout) */
7      esp_task_wdt_add(NULL);
8
9      while (1) {
10         /* Kick watchdog to prove we're alive */
11         esp_task_wdt_reset();
12
13         /* Control loop */
14         motor_control_iteration();
15
16         vTaskDelay(pdMS_TO_TICKS(1));
17     }
18 }
19
20 /* If task hangs, watchdog resets system after timeout
   (platform-specific) */

```

5.165.3 Sensor Timeouts

```

1  /* WRONG - Assumes sensor always responds */
2  float read_temperature(void)
3  {
4      send_i2c_command(TEMP_SENSOR_ADDR, READ_TEMP_CMD);
5      return receive_i2c_data(TEMP_SENSOR_ADDR); /* Hangs if sensor
   fails */
6  }
7
8  /* CORRECT - Timeout and error handling */
9  rx_err_t read_temperature_safe(float* temp_celsius)
10 {
11     rx_err_t ret;
12     uint8_t data[2];
13
14     /* I2C read with 100ms timeout */
15     ret = star_bus_i2c_read(bus_manager, "temp_i2c", data, 2,
16     TEMP_READ_REG, NULL);

```

```

17     if (ret != RX_OK) {
18         RX_LOG_WARN(s_TAG, "Temperature sensor timeout");
19         return ret;
20     }
21
22     *temp_celsius = (data[0] << 8 | data[1]) / 10.0f;
23     return RX_OK;
24 }
25
26 /* Use sensor timeout for safety */
27 rx_err_t ret = read_temperature_safe(&motor_temp);
28 if (ret != RX_OK) {
29     /* Sensor failed - enter safe state */
30     star_drv8243_stop(&motor, true);
31     system_enter_safe_mode();
32 }

```

5.165.4 Fail-Safe Design

```

1 /* Safe state: all motors stopped, systems halted */
2 void system_enter_safe_mode(void)
3 {
4     RX_LOG_ERROR(s_TAG, "ENTERING SAFE MODE");
5
6     /* Stop all motors */
7     for (int i = 0; i < NUM_MOTORS; i++) {
8         star_drv8243_stop(&motors[i], true); /* Brake */
9     }
10
11     /* Disable all actuators */
12     gpio_set_level(RELAY_ENABLE_PIN, 0);
13
14     /* Set error LED */
15     gpio_set_level(ERROR_LED_PIN, 1);
16
17     /* Log error to flash */
18     log_critical_error_to_nvs("Safe mode triggered");
19
20     /* Infinite loop - require manual reset */
21     while (1) {
22         vTaskDelay(pdMS_TO_TICKS(100));
23     }
24 }
25
26 /* Trigger safe mode on critical errors */
27 if (battery_voltage_mv < BATTERY_CRITICAL_MV) {
28     system_enter_safe_mode();
29 }
30
31 if (motor_temp_celsius > THERMAL_SHUTDOWN_CELSIUS) {
32     system_enter_safe_mode();
33 }

```

5.165.5 CRC Validation

```

1  /* Validate critical configuration with CRC */
2  typedef struct {
3      star_pid_config_t pid;
4      uint32_t crc32; /* CRC of pid structure */
5  } stored_config_t;
6
7  rx_err_t save_config_to_nvs(const star_pid_config_t* config)
8  {
9      stored_config_t stored;
10     memcpy(&stored.pid, config, sizeof(star_pid_config_t));
11
12     /* Calculate CRC-32 */
13     stored.crc32 = esp_crc32_le(0, (uint8_t*)&stored.pid,
14     sizeof(star_pid_config_t));
15
16     /* Save to NVS */
17     return nvs_set_blob(nvs_handle, "pid_config", &stored,
18     sizeof(stored_config_t));
19 }
20
21 rx_err_t load_config_from_nvs(star_pid_config_t* config)
22 {
23     stored_config_t stored;
24     size_t len = sizeof(stored_config_t);
25
26     esp_err_t ret = nvs_get_blob(nvs_handle, "pid_config",
27     &stored, &len);
28     if (ret != RX_OK) {
29         return ret;
30     }
31
32     /* Verify CRC */
33     uint32_t calculated_crc = esp_crc32_le(0, (uint8_t*)&stored.pid,
34     sizeof(star_pid_config_t));
35     if (calculated_crc != stored.crc32) {
36         RX_LOG_ERROR(s_TAG, "Config CRC mismatch! Flash corrupted?");
37         return ESP_ERR_INVALID_CRC;
38     }
39
40     memcpy(config, &stored.pid, sizeof(star_pid_config_t));
41     return RX_OK;
42 }

```

5.166 Hardware Abstraction

5.166.1 Use Abstract Bus Interfaces

```

1  /* CORRECT - Driver depends on bus abstraction, not platform HAL */
2  rx_err_t mpu6050_init(mpu6050_handle_t* handle,
3  star_bus_manager_t* bus_manager,

```

```

4  const char* bus_name)
5  {
6      handle->bus_manager = bus_manager;
7      handle->bus_name = bus_name;
8
9      /* Access I2C through bus manager */
10     uint8_t whoami;
11     esp_err_t ret = star_bus_i2c_read(bus_manager, bus_name,
12     &whoami, 1, MPU6050_WHO_AM_I, NULL);
13
14     /* Easy to mock for testing */
15     return ret;
16 }

```

5.166.2 Debounce Mechanical Inputs

```

1  /* Software debounce for button */
2  typedef struct {
3      gpio_num_t pin;
4      bool state;
5      uint32_t last_change_ms;
6      uint32_t debounce_ms;
7  } debounced_button_t;
8
9  bool button_read(debounced_button_t* btn)
10 {
11     bool raw_state = gpio_get_level(btn->pin);
12     uint32_t now_ms = xTaskGetTickCount() * portTICK_PERIOD_MS;
13
14     if (raw_state != btn->state) {
15         /* State change detected */
16         if (now_ms - btn->last_change_ms > btn->debounce_ms) {
17             /* Stable for debounce period */
18             btn->state = raw_state;
19             btn->last_change_ms = now_ms;
20         }
21     }
22
23     return btn->state;
24 }

```

5.166.3 Enable Brown-Out Detection

```

1  /* In sdkconfig or menuconfig:
2  * CONFIG_ESP32_BROWNOUT_DET=y
3  * CONFIG_ESP32_BROWNOUT_DET_LVL_SEL_7=y (3.08V threshold)
4  */
5
6  /* ESP32 automatically resets on voltage dip below threshold */
7  /* Prevents flash corruption and erratic behavior */

```

5.166.4 Flash Wear-Leveling

```
1  /* Use NVS (Non-Volatile Storage) for wear-leveling */
2  #include "nvs_flash.h"
3
4  rx_err_t init_nvs(void)
5  {
6      esp_err_t ret = nvs_flash_init();
7      if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
8          ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
9          /* Erase and retry */
10         /* Platform-specific: handle flash erase error appropriately */
11         nvs_flash_erase();
12         ret = nvs_flash_init();
13     }
14     return ret;
15 }
16
17 /* NVS automatically rotates writes across sectors */
18 nvs_handle_t nvs_handle;
19 nvs_open("storage", NVS_READWRITE, &nvs_handle);
20 nvs_set_u32(nvs_handle, "boot_count", boot_count);
21 nvs_commit(nvs_handle);
```

5.166.5 Always Use Pull-Up/Pull-Down

```
1  /* WRONG - Floating input */
2  gpio_config_t io_conf = {
3      .pin_bit_mask = (1ULL << GPIO_NUM_35),
4      .mode = GPIO_MODE_INPUT,
5      /* No pull-up or pull-down! Reads random values */
6  };
7
8  /* CORRECT - Enable pull-up */
9  gpio_config_t io_conf = {
10     .pin_bit_mask = (1ULL << GPIO_NUM_35),
11     .mode = GPIO_MODE_INPUT,
12     .pull_up_en = GPIO_PULLUP_ENABLE,
13     .pull_down_en = GPIO_PULLEDOWN_DISABLE,
14 };
15 gpio_config(&io_conf);
16
17 /* For active-low signals (like nFAULT), use pull-up */
18 /* For active-high signals, use pull-down */
```

5.167 Performance and Real-Time

5.167.1 Benchmark, Don't Guess

```
1  /* Use GPIO toggle to measure timing on oscilloscope */
2  #define BENCHMARK_START()    gpio_set_level(BENCHMARK_PIN, 1)
3  #define BENCHMARK_END()      gpio_set_level(BENCHMARK_PIN, 0)
4
5  void pid_control_loop(void)
6  {
7      BENCHMARK_START();
8
9      float velocity_rpm;
10     star_encoder_get_velocity_rpm(&encoder, 1.0f, 500, &velocity_rpm);
11
12     float output;
13     star_pid_compute(&pid, setpoint, velocity_rpm, 0.001f, &output);
14
15     star_motor_set_duty(&motor, output);
16
17     BENCHMARK_END();
18     /* Oscilloscope shows pulse width = execution time */
19 }
```

5.167.2 Avoid Blocking Operations

```
1  /* WRONG - Blocking I2C read in control loop */
2  void control_loop(void)
3  {
4      while (1) {
5          uint8_t data[6];
6          i2c_master_read_from_device(I2C_NUM_0, SENSOR_ADDR,
7          data, 6, 100 / portTICK_PERIOD_MS);
8          /* Blocks for up to 100ms! */
9
10         process_data(data);
11     }
12 }
13
14 /* CORRECT - Non-blocking with DMA */
15 void setup_nonblocking_i2c(void)
16 {
17     /* Configure I2C interrupt + DMA */
18     /* When data ready, ISR triggers */
19     /* Background task processes results */
20 }
```

5.168 Power Management

5.168.1 Use Sleep Modes

```
1  /* Light sleep between sensor reads */
2  void sensor_polling_task(void* arg)
3  {
```

```

4  while (1) {
5      read_sensors();
6      process_data();
7
8      /* Sleep for 100ms instead of busy-wait */
9      esp_sleep_enable_timer_wakeup(100000); /* 100ms in us */
10     esp_light_sleep_start();
11 }
12 }

```

5.168.2 Use DMA to Reduce CPU Load

```

1  /* DMA-enabled SPI transfer (CPU-free) */
2  spi_device_transmit(spi_handle, &transaction);
3  /* DMA handles transfer, CPU does other work */

```

5.169 Testing and Debugging

5.169.1 Use Mock Drivers

```

1  /* Create mock bus for unit testing */
2  star_bus_manager_t mock_bus_manager;
3  star_bus_manager_init(&mock_bus_manager, "mock", NULL, NULL);
4
5  /* Test driver without hardware */
6  mpu6050_handle_t sensor;
7  mpu6050_init(&sensor, &mock_bus_manager, "mock_i2c");

```

5.169.2 Meaningful Logging

```

1  /* Include timestamp, module, and error codes */
2  ESP_LOGE(s_TAG, "[%lu] Motor fault detected: current=%lu mA (limit=%d
    mA)",
3  esp_timer_get_time() / 1000, /* Timestamp in ms */
4  current_ma,
5  MAX_MOTOR_CURRENT_MA);

```

5.170 Reliability and Fault Tolerance

5.170.1 Reset Peripherals if Stuck

```

1  /* I2C bus recovery */
2  rx_err_t recover_i2c_bus(i2c_port_t port)
3  {
4      ESP_LOGW(s_TAG, "Recovering I2C bus %d", port);
5
6      /* Delete driver */

```

```
7   i2c_driver_delete(port);
8
9   /* Wait */
10  vTaskDelay(pdMS_TO_TICKS(10));
11
12  /* Re-initialize */
13  i2c_config_t conf = {/* ... */};
14  i2c_param_config(port, &conf);
15  return i2c_driver_install(port, conf.mode, 0, 0, 0);
16 }
```

5.171 Summary

Following these real-time and embedded best practices ensures:

- **Predictable timing** for motor control and sensor fusion
- **Robust error handling** for fault tolerance
- **Safe operation** in robotics applications
- **Maintainable code** through proper abstractions
- **Testability** with mock drivers and measurement tools

These practices are the foundation of reliable embedded systems for robotics.

This section establishes configuration requirements for Protocol Buffer messages used in STAR embedded firmware (ESP32, RX72N). The STAR project uses **nanopb** (Nano Protocol Buffers) for embedded systems, which requires static memory allocation.

5.172 nanopb Overview

nanopb is a small, portable Protocol Buffers implementation designed for embedded systems:

- **Static Allocation:** All buffers allocated at compile time (no malloc)
- **Small Code Size:** Minimal flash/RAM footprint
- **C Language:** Compatible with C99 (not C++ required)
- **Field Size Constraints:** Maximum sizes configured in `.options` files

5.173 Why Static Allocation?

STAR firmware follows strict "no dynamic allocation" rules (see Section 25). **nanopb** field sizes must be explicitly configured to enable compile-time buffer allocation.

5.174 Field Size Configuration

Field sizes are configured in `.options` files located alongside `.proto` files:

```

1 # File: star-proto/proto/star.options
2 star.v1.RequestHeader.request_id max_size:64
3 star.v1.ResponseHeader.error_message max_size:128
4 star.v1.FirmwareChunk.data max_size:1024

```

5.175 Standard Field Size Constraints

Table 24: nanopb Field Size Constraints for Embedded Platforms

Field Type	Constraint	Example
String (UUID)	max_size:64	request_id
String (error)	max_size:128	error_message
String (version)	max_size:32	firmware_version
Repeated (cells)	max_count:16	cell_mv[]
Repeated (motors)	max_count:4	motor_status[]
Bytes (firmware)	max_size:1024	chunk_data

5.176 Examples

5.176.1 String Fields

```

1 // File: robot_command.proto
2 message RequestHeader {
3     string request_id = 1; // UUID (36 chars + null terminator)
4 }
5
6 // File: robot_command.options
7 RequestHeader.request_id max_size:64

```

Generated C code:

```

1 typedef struct _RequestHeader {
2     char request_id[64]; /* Static allocation */
3 } RequestHeader;

```

5.176.2 Repeated Fields

```

1 // File: battery_state.proto
2 message BatteryState {
3     repeated uint32 cell_mv = 1; // Cell voltages in millivolts
4 }
5
6 // File: battery_state.options
7 BatteryState.cell_mv max_count:16

```

Generated C code:

```
1 typedef struct _BatteryState {
2     uint32_t cell_mv[16];      /* Static array */
3     pb_size_t cell_mv_count;   /* Actual element count */
4 } BatteryState;
```

5.176.3 Bytes Fields

```
1 // File: firmware_update.proto
2 message FirmwareChunk {
3     bytes data = 1;  // Binary firmware data
4 }
5
6 // File: firmware_update.options
7 FirmwareChunk.data max_size:1024
```

Generated C code:

```
1 typedef struct _FirmwareChunk {
2     pb_bytes_array_t data; /* Contains uint8_t bytes[1024] */
3 } FirmwareChunk;
```

5.177 Sizing Guidelines

5.177.1 String Sizes

Table 25: Recommended String Field Sizes

Purpose	Size	Example
UUID (RFC 4122)	64 bytes	request_id, session_id
Error messages	128 bytes	error_message
Version strings	32 bytes	firmware_version
Short names	32 bytes	component_name
Log messages	256 bytes	log_text

5.177.2 Repeated Field Counts

Table 26: Recommended Repeated Field Counts

Purpose	Count	Example
Battery cells (3S LiPo)	3	cell_mv[]
Motors (quadruped)	4	motor_status[]
Motors (hexapod)	8	motor_status[]
Sensor array	8–16	temperature_sensors[]
Telemetry samples	32–64	velocity_samples[]

5.177.3 Bytes Field Sizes

Table 27: Recommended Bytes Field Sizes

Purpose	Size	Example
Firmware chunk	1024 bytes	chunk_data
Image thumbnail	4096 bytes	thumbnail
Small binary data	256 bytes	calibration_blob
Cryptographic key	64 bytes	aes_key

5.178 Memory Impact

5.178.1 Buffer Size Calculation

Each message allocates memory for the largest possible size:

```

1  /* Example: BatteryState with 3 cells (3S LiPo) */
2  typedef struct {
3      uint32_t cell_mv[3];           /* 3 * 4 = 12 bytes */
4      pb_size_t cell_mv_count;       /* 4 bytes */
5      uint32_t current_ma;           /* 4 bytes */
6      uint8_t soc_percent;           /* 1 byte */
7      /* Total: ~21 bytes + padding */
8  } BatteryState;
9
10 /* Even if only 1 cell used, full array is allocated */
```

5.178.2 RAM Usage Example

```

1  /* Static allocation of message buffers */
2  static RequestHeader s_request;      /* 64 bytes */
3  static ResponseHeader s_response;    /* 128 bytes */
4  static BatteryState s_battery_state; /* 24 bytes */
5  static FirmwareChunk s_firmware_chunk; /* 1024 bytes */
6
7  /* Total: ~1240 bytes of static RAM */
```

5.179 Best Practices

5.179.1 Right-Size Your Buffers

- **Don't over-allocate:** max_size:1024 for a 20-byte string wastes 1004 bytes
- **Don't under-allocate:** Truncated data causes silent bugs
- **Measure actual usage:** Profile message sizes in production

5.179.2 Use Fixed-Size Types

```
1  /* CORRECT - Fixed-size integer types */
2  message MotorCommand {
3      int32 speed_percent = 1;      // Always 4 bytes
4      uint64 timestamp_us = 2;     // Always 8 bytes
5  }
6
7  /* AVOID - Variable-length encoding */
8  message MotorCommand {
9      sint32 speed_percent = 1;     // 1-5 bytes (varint encoding)
10 }
```

5.179.3 Validate Message Sizes

```
1  /* Check encoded message size before transmission */
2  uint8_t buffer[256];
3  pb_ostream_t stream = pb_ostream_from_buffer(buffer, sizeof(buffer));
4
5  bool status = pb_encode(&stream, RequestHeader_fields, &request);
6  if (!status) {
7      RX_LOG_ERROR(s_TAG, "Encoding failed: %s", PB_GET_ERROR(&stream));
8      return RX_ERR_FAIL;
9  }
10
11  size_t message_length = stream.bytes_written;
12  RX_LOG_INFO(s_TAG, "Encoded message: %zu bytes", message_length);
13
14  /* Verify it fits in SPI transaction buffer */
15  if (message_length > SPI_MAX_TRANSFER_SIZE) {
16      RX_LOG_ERROR(s_TAG, "Message too large for SPI: %zu > %d",
17          message_length, SPI_MAX_TRANSFER_SIZE);
18      return ESP_ERR_INVALID_SIZE;
19  }
```

5.180 Integration with Code Generation

5.180.1 buf.gen.yaml Configuration

```
1  # File: star-proto/buf.gen.yaml
2  version: v2
3  plugins:
4  - local: nanopb_generator
5  out: gen/nanopb
6  opt:
7  - --extension=.pb
```

5.180.2 Options File Naming

```
1 # Options file must match .proto file name
2 proto/
3 robot_command.proto      -> robot_command.options
4 battery_state.proto      -> battery_state.options
5 firmware_update.proto    -> firmware_update.options
```

5.181 Common Pitfalls

5.181.1 Missing Options File

```
1 /* ERROR: No .options file for string field */
2 // robot_command.proto
3 message Command {
4     string name = 1; // No max_size specified
5 }
6
7 /* Generated code uses default (usually too small or error) */
```

Fix: Always create .options file for messages with strings, bytes, or repeated fields.

5.181.2 Mismatched Field Names

```
1 # WRONG - Field name mismatch
2 # robot_command.options
3 Command.command_name max_size:32 # Field is 'name', not
4     'command_name'
5
6 # CORRECT
7 Command.name max_size:32
```

5.181.3 Forgetting Array Count

```
1 /* Common mistake: Forgetting to set count field */
2 BatteryState battery = {0};
3 battery.cell_mv[0] = 3700;
4 battery.cell_mv[1] = 3720;
5 battery.cell_mv[2] = 3710;
6 /* BUG: battery.cell_mv_count not set! Encoder sends 0 cells */
7
8 /* CORRECT */
9 battery.cell_mv_count = 3; /* Set actual element count (3S LiPo) */
```

5.182 Summary

- Always create .options files for strings, bytes, repeated fields

- **Right-size buffers** based on actual usage (measure, don't guess)
- **Use fixed-size types** to avoid varint encoding overhead
- **Validate encoded sizes** before transmission
- **Set array count fields** for repeated fields

Properly configured nanopb constraints enable reliable, predictable Protocol Buffer communication on resource-constrained embedded hardware (ESP32, RX72N, and future platforms).

5.183 NASA Power of 10: Rules for Safety-Critical Code

This section documents the NASA/JPL Power of 10 coding rules developed by Gerard J. Holzmann in 2006 for safety-critical embedded systems. These rules prioritize verification and predictability over coding flexibility.

STAR Compliance Philosophy: The STAR project follows the Power of 10 rules, with one **intentional architectural deviation** for Dependency Inversion Principle (DIP) — a conscious trade-off for testability and modularity.

5.183.1 Compliance Status Legend

Symbol	Meaning
COMPLIANT	STAR follows this rule
INTENTIONAL DEVIATION	Architectural choice (DIP pattern)

5.183.2 Rule 1: Simplify Control Flow

Rule: Restrict all code to very simple control flow constructs — no `goto`, `setjmp/longjmp`, or recursion (direct or indirect).

Rationale: Creates predictable program structure for human understanding and static analysis. Recursion causes unbounded stack usage — dangerous in embedded systems.

STAR Compliance: **COMPLIANT**

```

1  /* COMPLIANT - No goto, no recursion */
2  esp_err_t star_motor_init(star_motor_handle_t* handle,
3                           const star_motor_config_t* config)
4  {
5      if (handle == NULL || config == NULL) {
6          return ESP_ERR_INVALID_ARG;
7      }
8
9      /* All control flow uses if/while/for */
10     esp_err_t ret = configure_timer(handle, config);
11     if (ret != ESP_OK) {
12         return ret;
13     }
14 
```

```

15     return configure_gpio(handle, config);
16 }

```

5.183.3 Rule 2: Fixed Loop Upper-Bounds

Rule: All loops must have a fixed upper bound that static analysis tools can trivially prove.

Rationale: Prevents runaway code, guarantees termination, enables real-time deadline verification.

STAR Compliance: **COMPLIANT**

```

1  /* COMPLIANT - Fixed iteration count */
2  #define MAX_RETRIES (3)
3
4  for (uint8_t i = 0; i < MAX_RETRIES; i++) {
5      if (sensor_read() == ESP_OK) {
6          break;
7      }
8      vTaskDelay(pdMS_TO_TICKS(100));
9  }
10
11 /* COMPLIANT - Array with known size */
12 #define BQ7850_MAX_CELLS (16)
13
14 for (uint8_t cell = 0; cell < BQ7850_MAX_CELLS; cell++) {
15     read_cell_voltage(cell, &voltages[cell]);
16 }

```

5.183.4 Rule 3: No Dynamic Memory After Initialization

Rule: Prohibit malloc/free after initialization phase.

Rationale: Dynamic allocation causes unpredictable performance, memory leaks, fragmentation, and corruption.

STAR Compliance: **COMPLIANT**

Best Practice: Use pre-allocated static pools instead of runtime memory requests.

```

1  /* COMPLIANT - Static allocation with fixed limits */
2  #define MAX_ITEMS (8)
3  #define MAX_DESC_LEN (64)
4
5  typedef struct {
6      char items[MAX_ITEMS][MAX_DESC_LEN]; /* Static 2D array */
7      uint8_t item_count;
8  } static_pool_t;
9
10 /* Usage */
11 if (pool->item_count >= MAX_ITEMS) {
12     return ESP_ERR_NO_MEM;
13 }
14 strncpy(pool->items[pool->item_count], desc, MAX_DESC_LEN - 1);

```

```

15 pool->items[pool->item_count][MAX_DESC_LEN - 1] = '\0';
16 pool->item_count++;

```

5.183.5 Rule 4: Keep Functions Short

Rule: Functions should not exceed ~60 lines (one printed page, one statement per line).

Rationale: Represents single verifiable logical units; improves comprehension, review, and testability.

STAR Compliance: **MOSTLY COMPLIANT**

```

1  /* COMPLIANT - Single responsibility, short function */
2  esp_err_t star_drv8243_read_current(star_drv8243_handle_t* handle,
3                                     float* current_ma)
4  {
5      if (handle == NULL || current_ma == NULL || !handle->initialized) {
6          return ESP_ERR_INVALID_ARG;
7      }
8
9      int raw_value = 0;
10     esp_err_t ret = read_adc_channel(handle, &raw_value);
11     if (ret != ESP_OK) {
12         return ret;
13     }
14
15     *current_ma = (float)raw_value * handle->ki_propi / 1000.0f;
16     return ESP_OK;
17 }

```

5.183.6 Rule 5: Use Assertions Generously

Rule: Minimum two assertions per function; assertions must be side-effect-free.

Rationale: Acts as executable documentation verifying pre-conditions, post-conditions, and invariants.

STAR Compliance: **COMPLIANT**

Note: STAR uses explicit parameter validation instead of assertions for production code (assertions disabled in release builds).

```

1  /* COMPLIANT - Explicit validation (better than assert for
2     production) */
3  esp_err_t star_pid_compute(star_pid_handle_t* handle, float setpoint,
4                             float measurement, float dt, float* output)
5  {
6      /* Pre-condition checks (effectively runtime assertions) */
7      if (handle == NULL) {
8          return ESP_ERR_INVALID_ARG;
9      }
10     if (!handle->initialized) {
11         return ESP_ERR_INVALID_STATE;
12     }
13 }

```

```

12     if (output == NULL) {
13         return ESP_ERR_INVALID_ARG;
14     }
15     if (dt <= 0.0f || dt > 1.0f) { /* Sanity check time delta */
16         return ESP_ERR_INVALID_ARG;
17     }
18
19     /* Function logic */
20     float error = setpoint - measurement;
21     handle->integral += error * dt;
22
23     /* Post-condition: integral within bounds */
24     if (handle->integral > handle->integral_max) {
25         handle->integral = handle->integral_max;
26     } else if (handle->integral < handle->integral_min) {
27         handle->integral = handle->integral_min;
28     }
29
30     *output = handle->kp * error + handle->ki * handle->integral;
31     return ESP_OK;
32 }

```

5.183.7 Rule 6: Declare Data at Smallest Scope

Rule: Minimize variable scope to reduce accidental corruption risks.

Rationale: Limits chances of accidental reference or corruption from other code.

STAR Compliance: **COMPLIANT**

```

1  /* COMPLIANT - Variables declared at minimal scope */
2  void process_sensors(void)
3  {
4      /* Loop variable declared in for statement */
5      for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
6          /* Temporary variable for single sensor */
7          float temp_c = 0.0f;
8          if (read_sensor(i, &temp_c) == ESP_OK) {
9              /* Process immediately, temp_c out of scope after block */
10             update_telemetry(i, temp_c);
11         }
12     }
13 }

```

5.183.8 Rule 7: Check All Return Values and Parameters

Rule: Validate all function returns and input parameters; treat warnings as errors.

Rationale: Forces explicit failure-condition handling rather than ignoring error codes.

STAR Compliance: **COMPLIANT**

```

1  /* COMPLIANT - All returns checked, all parameters validated */
2  esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,

```

```

3                                     star_bus_config_t* config)
4 {
5     /* Parameter validation */
6     if (manager == NULL || config == NULL) {
7         return ESP_ERR_INVALID_ARG;
8     }
9
10    /* Check return value and propagate errors */
11    esp_err_t ret = star_bus_config_init(config);
12    if (ret != ESP_OK) {
13        ESP_LOGE(TAG, "Bus initialization failed: %s",
14                 esp_err_to_name(ret));
15        return ret;
16    }
17
18    /* Mutex operation checked */
19    if (xSemaphoreTake(manager->mutex, pdMS_TO_TICKS(1000)) != pdTRUE) {
20        ESP_LOGE(TAG, "Failed to acquire mutex");
21        return ESP_ERR_TIMEOUT;
22    }
23
24    /* Critical section */
25    add_to_list(manager, config);
26    xSemaphoreGive(manager->mutex);
27
28    return ESP_OK;
29 }

```

5.183.9 Rule 8: Limit Preprocessor Use

Rule: Restrict to header inclusion and simple macros; forbid token pasting, recursive macros, and incomplete syntactic units.

Rationale: Preprocessor complexity obscures operations, fooling developers and static analysis tools.

STAR Compliance: **COMPLIANT**

```

1  /* COMPLIANT - Simple constant macros */
2  #define MAX_RETRY_COUNT (3)
3  #define MOTOR_PWM_FREQ_HZ (20000)
4  #define TIMEOUT_MS (1000)
5
6  /* COMPLIANT - Simple inline functions (better than complex macros) */
7  static inline float voltage_mv_to_v(uint16_t mv)
8  {
9      return (float)mv / 1000.0f;
10 }
11
12 /* AVOID - Complex token-pasting macros */
13 // #define CREATE_HANDLER(name) handler_##name##_init() /* Too
14 complex */

```

5.183.10 Rule 9: Restrict Pointer Use

Rule: Maximum one dereferencing level (`*ptr` OK, `**ptr` forbidden); no function pointers.

Rationale: Multiple indirection and function pointers make static data-flow analysis impossible.

STAR Compliance: **INTENTIONAL DEVIATION**

Intentional Architectural Deviation: STAR uses function pointers for **Dependency Inversion Principle (DIP)** — an industry-standard pattern for testability and modularity. This is a conscious trade-off for better architecture.

DIP Pattern (Intentional Function Pointer Use):

```

1  /* INTENTIONAL DEVIATION - DIP interface with function pointers */
2  typedef struct star_error_interface {
3      esp_err_t (*record_error)(void* ctx, esp_err_t error, const char*
4          message,
5          const char* file, int line, const char*
6              func);
7      bool (*can_retry)(void* ctx);
8      esp_err_t (*reset_state)(void* ctx);
9      void* ctx; /* Implementation context */
10 } star_error_interface_t;
11
12 /* Why we keep this:
13 * 1. Enables mock implementations for unit testing
14 * 2. Allows swapping error handlers without recompilation
15 * 3. Follows SOLID principles (Dependency Inversion)
16 * 4. Industry-standard pattern (used in Linux kernel, RTOS, etc.)
17 */

```

5.183.11 Rule 10: Compile with Maximum Warnings

Rule: Enable all compiler warnings at highest pedantry; achieve zero-warning builds with static analyzers.

Rationale: Modern compilers catch bugs before runtime. Zero warnings, no excuses.

STAR Compliance: **COMPLIANT**

```

1  # platformio.ini - Compiler flags
2  build_flags =
3      -Wall # Enable all warnings
4      -Wextra # Extra warnings
5      -Werror # Treat warnings as errors
6      -Wno-unused-parameter # Allow unused params in interface
7      implementations
8      -Wno-missing-field-initializers

```

CI/CD Enforcement: Build fails on any compiler warning.

5.183.12 Summary: STAR Compliance Matrix

Table 28: NASA Power of 10 Compliance Status

Rule	Description	STAR Status
1	No <code>goto</code> /recursion	COMPLIANT
2	Fixed loop bounds	COMPLIANT
3	No dynamic memory	COMPLIANT
4	Short functions (<60 lines)	COMPLIANT
5	Use assertions	COMPLIANT
6	Minimal variable scope	COMPLIANT
7	Check all returns	COMPLIANT
8	Limit preprocessor	COMPLIANT
9	Restrict pointers	INTENTIONAL (DIP)
10	Maximum warnings	COMPLIANT

5.183.13 References

- Holzmann, Gerard J. “*The Power of 10: Rules for Developing Safety-Critical Code.*” IEEE Computer, 39(6), pp. 95-97, June 2006.
- JPL Institutional Coding Standard for the C Programming Language (JPL DOCID D-60411)
- MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems

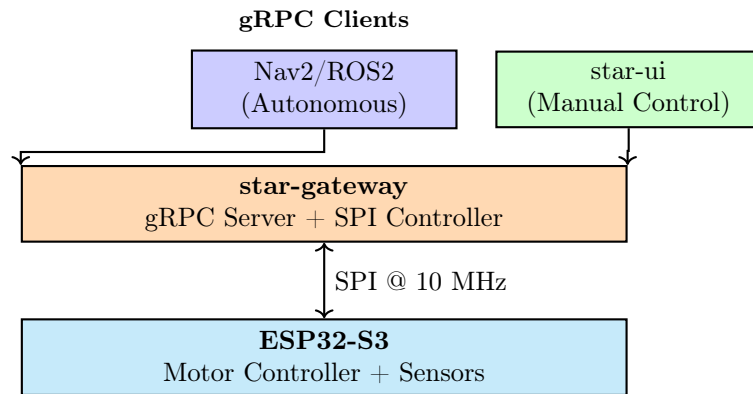
6 Gateway Architecture

This section documents the **star-gateway** Go service that bridges the Raspberry Pi 5 control stack with the ESP32-S3 motor controller.

6.1 Overview

The gateway service runs on the Raspberry Pi 5 and serves as the communication bridge between:

- **High-level side:** gRPC clients (Nav2/ROS2 for autonomous navigation, UI for manual control)
- **Low-level side:** ESP32-S3 motor controller via SPI at 10 MHz



6.2 Protocol Stack Implementation

The gateway implements Layers 1–4 of the communication protocol (see Section ??).

Table 29: Gateway Protocol Stack Implementation

Layer	Name	Go Package	Status
5	Application	<code>internal/service/</code>	Placeholder
4	Serialization	<code>star-proto/gen/go/</code>	Generated
3	ARQ	<code>internal/arq/</code>	Placeholder
2	Framing	<code>internal/frame/</code>	Implemented
1	Transport	<code>internal/transport/</code>	Placeholder

6.3 Directory Structure

```

star-gateway/
cmd/
    star-gateway/
        main.go          # Entry point
internal/
  
```

```

transport/          # Layer 1: SPI transport
  spi.go            # Transport interface
  spi_test.go
frame/              # Layer 2: Frame protocol
  frame.go          # Constants and types
  encoder.go        # Frame encoder
  decoder.go        # Frame decoder
  frame_test.go
arq/                # Layer 3: ARQ protocol
  arq.go            # Stop-and-Wait ARQ
  arq_test.go
service/            # Layer 5: gRPC services
  motor_control.go
  telemetry.go
  battery.go
  configuration.go
  firmware.go
go.mod
CLAUDE.md

```

6.4 Frame Package

The `internal/frame` package defines the wire protocol constants and types.

6.4.1 Frame Constants

Table 30: Frame Protocol Constants

Constant	Value	Description
SyncWord	0x55AA	Frame sync marker (alternating bits: [0x55, 0xAA])
SyncSize	2 bytes	Sync word size
HeaderSize	6 bytes	SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1)
CRCSize	4 bytes	CRC-32 checksum
MaxPayloadSize	1024 bytes	Maximum protobuf payload
MaxFrameSize	1036 bytes	Total frame size
MinFrameSize	12 bytes	Empty payload frame

6.4.2 Frame Types

- `FrameTypeCommand` – Command from controller to peripheral
- `FrameTypeResponse` – Response from peripheral to controller
- `FrameTypeAck` – Acknowledgment of successful receipt
- `FrameTypeNack` – Negative acknowledgment (CRC error, etc.)

6.5 Transport Package

The `internal/transport` package provides the SPI transport layer.

6.5.1 SPI Configuration

Table 31: SPI Configuration Constants

Parameter	Value	Description
Device	<code>/dev/spidev0.0</code>	SPI device path
Speed	10 MHz	SPI clock frequency
Mode	0	CPOL=0, CPHA=0
Bits per word	8	Word size
Timeout	100 ms	Default operation timeout

6.5.2 Transport Interface

```
type Transport interface {
    Send(data []byte) (int, error)
    Receive(maxLen int) ([]byte, error)
    Transfer(txData []byte) ([]byte, error)
    Close() error
}
```

6.6 ARQ Package

The `internal/arq` package implements Stop-and-Wait ARQ for reliable delivery.

6.6.1 ARQ Parameters

Table 32: ARQ Protocol Parameters

Parameter	Value	Description
Max Retries	3	Maximum transmission attempts
Timeout	10 ms	ACK wait timeout
Sequence Max	0xFFFF	16-bit wraparound

6.6.2 ARQ State Machine

- **IDLE** – Ready to send or receive
- **WAITING_ACK** – Waiting for acknowledgment
- **RETRANSMITTING** – Retransmitting after timeout/NACK
- **ERROR** – Max retries exceeded

6.7 gRPC Services

The gateway exposes five gRPC services (see Section ?? for detailed specifications):

Table 33: Gateway gRPC Services

Service	Purpose
MotorControlService	Differential drive control, encoder streaming
TelemetryService	IMU, GPS, system status
BatteryManagementService	BQ7850 BMS monitoring
ConfigurationService	Runtime parameters, NVS persistence
FirmwareUpdateService	OTA updates, rollback

6.8 Build and Test

```
# Build the gateway binary
cd star-gateway
go build ./cmd/star-gateway

# Run all tests
go test ./...

# Run with verbose output
go test -v ./...

# Format code
go fmt ./...

# Static analysis
go vet ./...
```

6.9 Dependencies

The gateway depends on:

- github.com/Locked-Inc/star-proto/gen/go – Generated protobuf and gRPC code
- google.golang.org/grpc – gRPC framework
- google.golang.org/protobuf – Protocol Buffers runtime

The `go.work` file at the repository root enables workspace mode for seamless cross-module development:

```
go 1.21

use (
    ./star-proto/gen/go
    ./star-proto/tests/go
```

```
    ./star-gateway  
)
```